คณะเทคโนโลยีสารสนเทศ
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

Chapter 10: Practical Implementation of Convolutional Neural Networks

บรรยายโดย ผศ.ดร.ธราวิเชษฐ์ ธิติจรูญโรจน์

คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง



หัวข้อ

- หลักการและการสร้างโมเดลสำหรับ Multiclass Image Classification
- หลักการและการสร้างโมเดลสำหรับ Multilabel Image Classification
- หลักการและการสร้างโมเดลสำหรับ Image Regression



หัวข้อ

- หลักการและการสร้างโมเดลสำหรับ Multiclass Image Classification
- หลักการและการสร้างโมเดลสำหรับ Multilabel Image Classification
- หลักการและการสร้างโมเดลสำหรับ Image Regression



การพัฒนาโปรแกรม

การโปรแกรมโครงข่ายประสาทแบบคอนโวลูชัน ด้วยภาษาไพธอนจากชุดคำสั่ง PyTorch ตอนที่ 1

Python สำหรับ Deep Learning

ความสัมพันธ์ของ Keras 3, TensorFlow, JAX, PyTorch และ OpenVINO

โครงสร้างที่ถูกต้องของ Keras 3

Keras 3
High-level API for Deep Learning
เขียนโครงสร้างโมเดล
และกำหนด training workflow
แล้วเลือก backend สำหรับการคำนวณ

เลือก backend

ตัวเลือก computational backend ของ Keras 3 — เลือกใช้หนึ่งตัวในแต่ละโปรแกรม

- **TensorFlow** — ML framework • Training + Inference
- **JAX** — High-performance array computing; NumPy-style • Autodiff • JIT • CPU/GPU/TPU
- **PyTorch** — ML framework • Training + Inference
- **OpenVINO** — Intel toolkit สำหรับ optimize และ deploy; Keras รองรับเฉพาะ Inference

ต้องเข้าใจว่า

- TensorFlow และ PyTorch เป็น framework ที่ใช้งานโดยตรงได้
- JAX ก็ใช้งานโดยตรงได้ หรือใช้เป็น computational backend ของ Keras

Keras เป็นชั้น API ที่อยู่เหนือ backend — ไม่ได้ครอบ TensorFlow หรือ PyTorch และทั้งสอง framework ไม่จำเป็นต้องใช้ Keras

Keras 3

ใช้งานง่าย • Multi-backend • Open source

- API ระดับสูงสำหรับสร้าง ฝึก ประเมิน และทำนายด้วยโมเดล Deep Learning
- ออกแบบโดย François Chollet ในปี 2015 ซึ่งต่อมาเป็นนักวิจัยและวิศวกรด้าน AI ที่ Google
- Keras 3 รองรับ TensorFlow, JAX และ PyTorch รวมถึง OpenVINO สำหรับ inference เท่านั้น

จุดเด่น: เริ่มต้นเร็ว ลด boilerplate และเปลี่ยน backend ได้

TensorFlow

End-to-end Machine Learning Platform

- Framework สำหรับสร้าง ฝึก และ deploy โมเดล ML
- รองรับ server, web, mobile, edge และงานแบบกระจาย
- ใช้ API ของ TensorFlow โดยตรง หรือใช้ผ่าน Keras ได้
- พัฒนาโดย Google Brain และเปิดเป็น open source ปี 2015

จุดเด่น: ecosystem ครบ ตั้งแต่การทดลองจนถึง production

PyTorch

Flexible Python-first ML Framework

- Framework open source สำหรับ Machine/Deep Learning
- เขียน Python ได้เป็นธรรมชาติและปรับ training loop ยืดหยุ่น
- ใช้งานโดยตรง หรือเลือกเป็น backend ของ Keras 3 ได้
- เริ่มจาก Facebook AI Research; ปัจจุบันอยู่ภายใต้ PyTorch Foundation ซึ่ง hosted by Linux Foundation

จุดเด่น: ควบคุมได้ละเอียด เหมาะกับงานวิจัยและ production

แหล่งข้อมูลทางการ: [1] keras.io/keras_3 [2] tensorflow.org/about [3] docs.jax.dev [4] intel.com/openvino [5] pytorch.org/foundation

หมายเหตุ: ความสามารถข้าม backend ของ custom component ขึ้นอยู่กับ API ที่ใช้ • OpenVINO backend ใน Keras รองรับการทำนาย ไม่รองรับการฝึกโมเดล

torch/

การเปรียบเทียบเครื่องมือ Deep Learning ใน Python

Keras 3, JAX, TensorFlow, OpenVINO และ PyTorch มีบทบาทต่างกัน จึงไม่ควรสรุปว่าเครื่องมือใด “เร็วกว่า” ในทุกกรณี

ความเร็วขึ้นกับโมเดล backend ฮาร์ดแวร์ ขนาด batch การคอมไพล์ และรูปแบบการนำไปใช้งาน

หัวข้อ	Keras 3 Multi-backend API	JAX Array computing	TensorFlow End-to-end ML	OpenVINO Inference toolkit	PyTorch Python-first ML
ประเภทและบทบาทหลัก	API ระดับสูงสำหรับสร้างและฝึกโมเดล โดยเลือก backend ได้ ออกแบบโดย François Chollet ปี 2015 • ต่อมาเป็นนักวิจัยที่ Google	ไลบรารีคำนวณอาร์เรย์และแปลงโปรแกรมเพื่อการคำนวณสมรรถนะสูง พัฒนาโดย Google	Framework และแพลตฟอร์ม Machine Learning แบบ End-to-end Google Brain • 2015	Toolkit และ Runtime สำหรับปรับแต่งและเร่งการทำ Inference พัฒนาโดย Intel	Framework สำหรับ Machine/Deep Learning ที่เน้นการใช้งานผ่าน Python เริ่มโดย FAIR • PyTorch Foundation
ระดับ API	**สูง** Sequential, Functional, fit()	**ระดับคำนวณ** NumPy-style + transformations	**สูงถึงต่ำ** Keras API ถึง TensorFlow Core	**Deployment/Runtime** API สำหรับ compile และ infer	**ยืดหยุ่นหลายระดับ** torch.nn, autograd, compile
การฝึกและการทำนาย	Training + Inference OpenVINO: inference เท่านั้น	รองรับทั้งสองงาน มักใช้ร่วมกับ Keras/Flax	Training + Inference พร้อมเครื่องมือ production	**Inference เท่านั้น** ไม่ใช่ framework สำหรับฝึก	Training + Inference รองรับ research และ production
จุดเด่นสำคัญ	โค้ดกระชับ ใช้งานง่าย สลับ TensorFlow, JAX หรือ PyTorch backend ได้	jit, grad, vmap รองรับ parallelization และงานขนาดใหญ่	Ecosystem ครบ ตั้งแต่ทดลอง ฝึก จนถึงนำไปใช้งานจริง	ลด latency เพิ่ม throughput ลดขนาดโมเดล และใช้ฮาร์ดแวร์ได้มีประสิทธิภาพ	Pythonic และยืดหยุ่น ปรับ training loop ได้ละเอียด มี ecosystem ขนาดใหญ่
การ Debug และปรับแต่ง	เริ่มต้นและอ่านโค้ดง่าย ปรับ train_step ได้เมื่อจำเป็น	ยืดหยุ่น แต่ tracing/JIT ต้องเข้าใจรูปแบบการทำงาน	มี Eager execution และปรับลงระดับ Core ได้	Debug หลังแปลงโมเดล เน้น compatibility/performance	Eager execution เข้าใจง่าย ควบคุมขั้นตอนฝึกได้ละเอียด
ฮาร์ดแวร์และ Deployment	ขึ้นกับ backend ที่เลือก CPU, GPU, TPU หรือ Intel	CPU, GPU และ TPU รองรับ sharding/หลายอุปกรณ์	Server, Web, Mobile, Edge TF Serving, TF.js, Lite และ distributed training	ปรับใช้บน Intel hardware ทั้ง on-device, browser on-premise และ cloud	CPU, GPU และ accelerators Server, Cloud และ Edge
เหมาะสำหรับ	การเรียน การสอน Prototype และ cross-framework	งานวิจัยเชิงคำนวณ และการฝึกขนาดใหญ่	ระบบ ML แบบครบวงจร และ production deployment	เร่งโมเดลที่ฝึกเสร็จแล้ว บน Intel/Edge/AI PC	งานวิจัย Custom training และระบบ production

แหล่งข้อมูลทางการ: [1] keras.io/keras_3 [2] docs.jax.dev [3] tensorflow.org/about [4] intel.com/openvino [5] pytorch.org/foundation

หมายเหตุ: ตารางสรุป “บทบาทและแนวทางใช้งาน” ไม่ใช่ผล benchmark; ความเร็วต้องวัดด้วยโมเดล ข้อมูล และฮาร์ดแวร์เดียวกัน

ch/

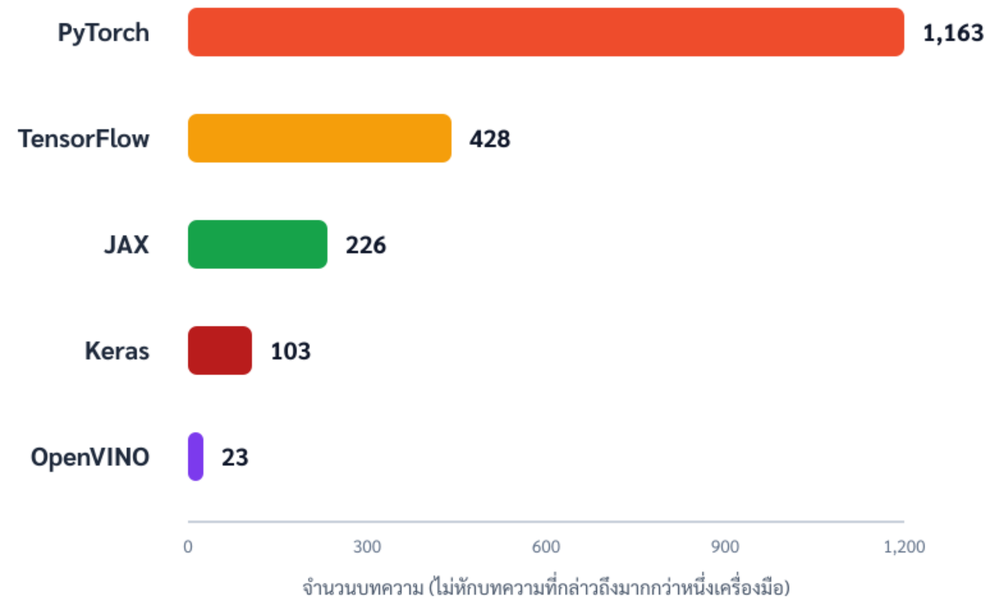
ความนิยมของเครื่องมือ Deep Learning: งานวิจัยและชุมชนนักพัฒนา

ตัวชี้วัดสองแบบสะท้อนคนละมุม และไม่ใช่คะแนนประสิทธิภาพของ Framework

1. การกล่าวถึงในงานวิจัยบน arXiv

จำนวนบทความที่มีชื่อเครื่องมือใน metadata • ปี 2021–2025

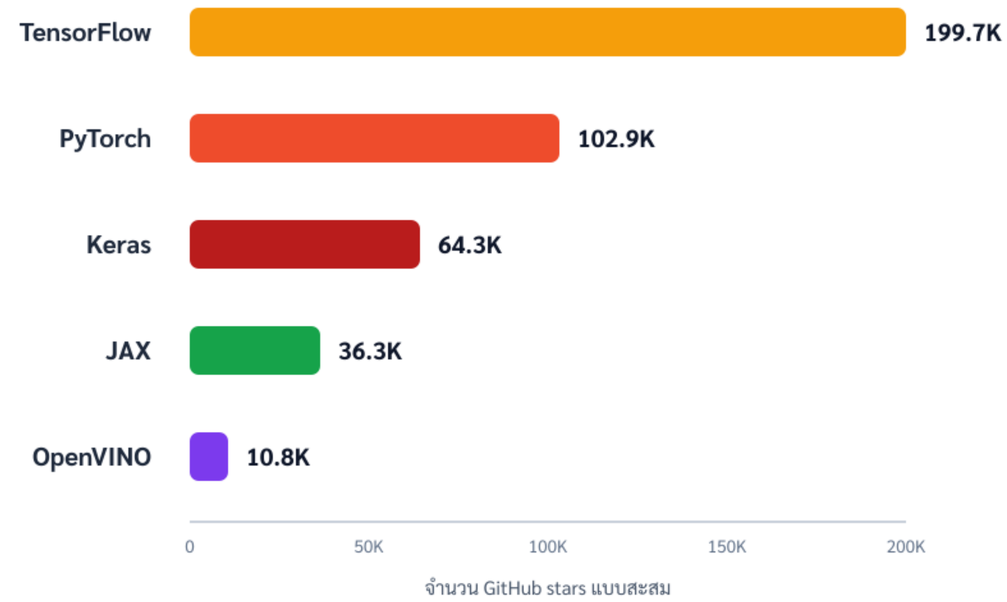
หมวด cs.LG, cs.AI, cs.CV, cs.CL และ stat.ML



2. ขนาดชุมชนบน GitHub

จำนวน stars ของ repository หลัก • Snapshot: 11 กันยายน 2026

Stars สะท้อนความสนใจสะสม ไม่ใช่จำนวนผู้ใช้ปัจจุบันโดยตรง



สรุปอย่างระมัดระวัง

- PyTorch ถูกกล่าวถึงในบทความ arXiv ชุดที่ตรวจสอบมากที่สุด สะท้อนการใช้งานสูงในงานวิจัย AI/ML
- TensorFlow มี GitHub stars สะสมสูงที่สุด ขณะที่ PyTorch อยู่ลำดับที่สอง
- Keras, JAX และ OpenVINO มีบทบาทเฉพาะต่างกัน จึงไม่ควรตีความจำนวนที่น้อยกว่าว่า “ประสิทธิภาพด้อยกว่า”
- การเลือกเครื่องมือควรพิจารณา API, ecosystem, hardware, training และ deployment ร่วมกัน

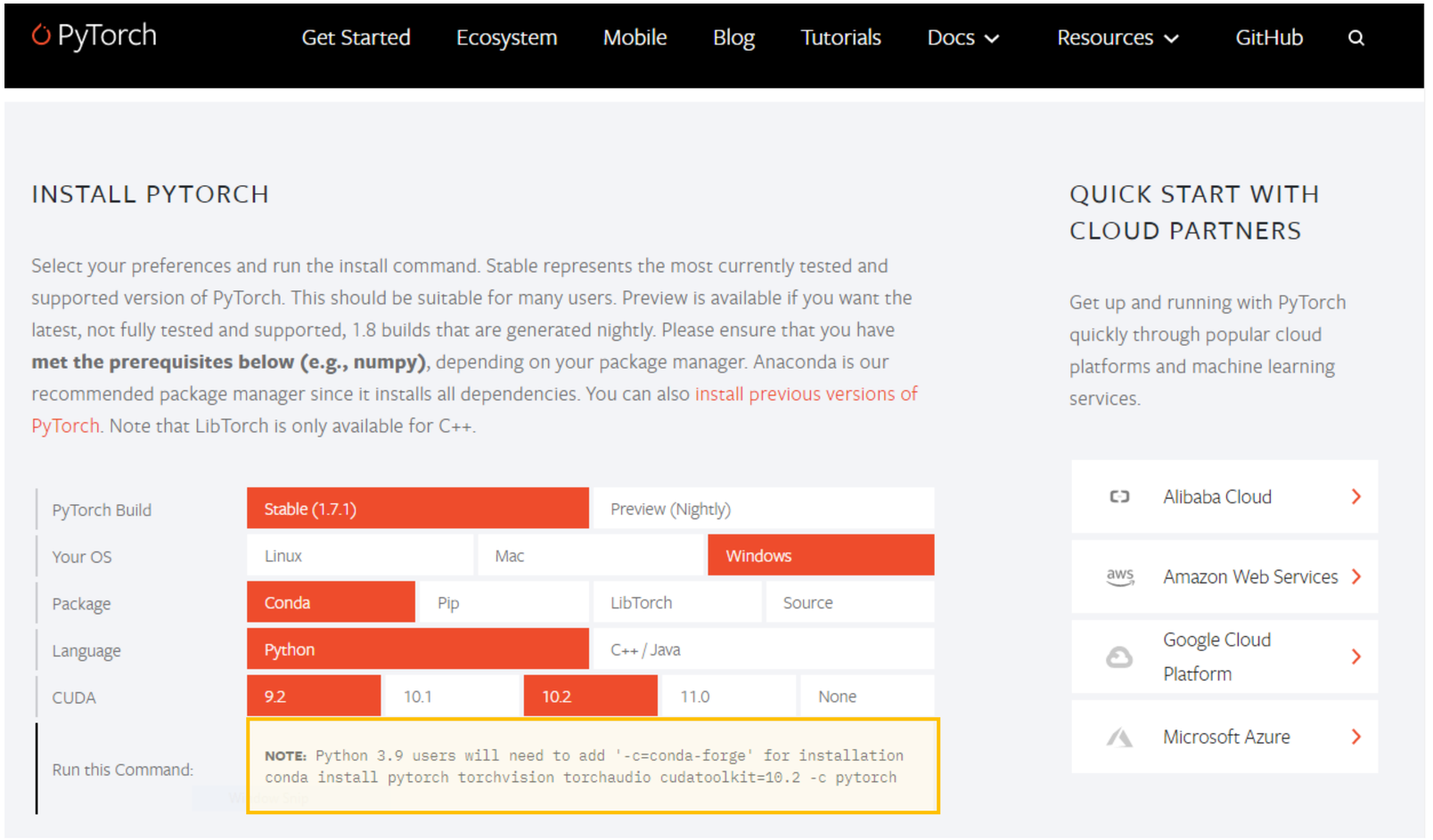
แหล่งข้อมูล: arXiv API (export.arxiv.org/api/query) และ GitHub REST API — repositories: keras-team/keras, jax-ml/jax, tensorflow/tensorflow, openvinotoolkit/openvino, pytorch/pytorch

วิธีนับ arXiv: ค้นคำใน metadata ภายในหมวดที่ระบุและช่วง submittedDate 2021-01-01 ถึง 2025-12-31 • ข้อมูลปี 2026 ไม่รวมเพราะยังไม่ครบปี

Faculty of Information Technology
King Mongkut's Institute of Technology Ladkrabang

การติดตั้งชุดคำสั่ง Pytorch

ให้นักศึกษาเข้าเว็บไซต์ https://pytorch.org/ เพื่อทำการดาวโหลดคำสั่งการติดตั้ง Pytorch ผ่าน Anaconda Command

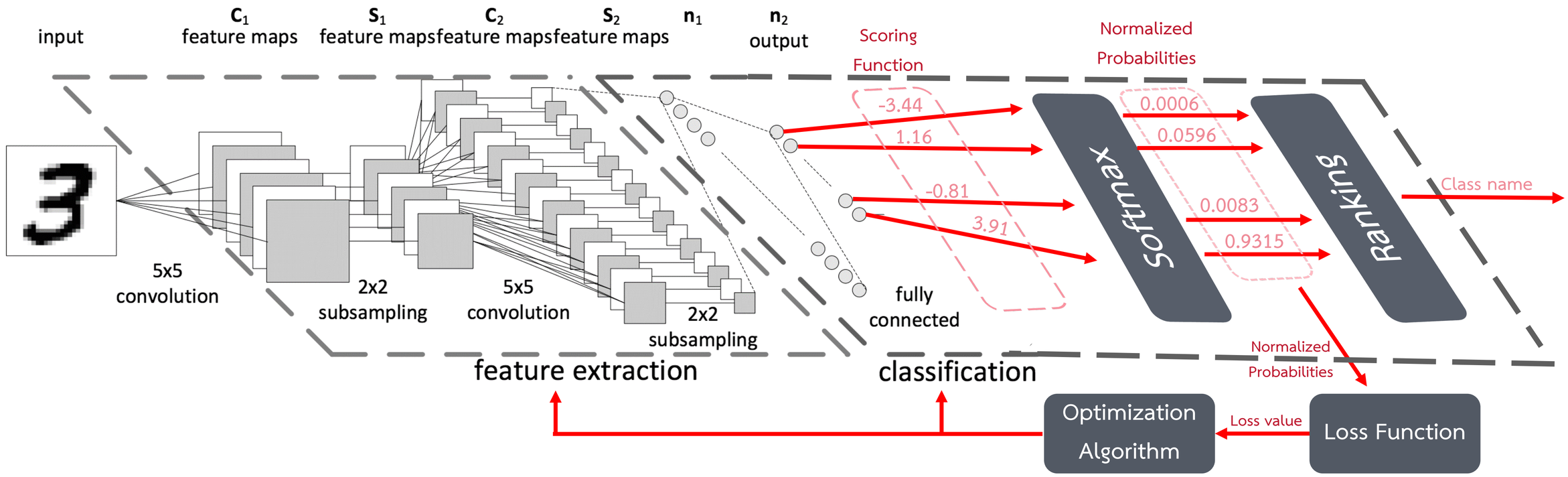


หลักการโครงข่ายประสาทแบบคอนโวลูชัน

Forward Propagation คือ กระบวนการที่ป้อน Input Image เข้าสู่แบบจำลองที่ฝึกสอนเรียบร้อยแล้ว และคืนค่าเป็นเป็น Classname

Backward Propagation คือ กระบวนการที่นำค่า Normalized Probabilities ไปใช้เพื่อปรับปรุง Weight ของแบบจำลองให้เกิดความแม่นยำมากยิ่งขึ้น

Forward Propagation : เพื่อรู้จำรูปภาพ

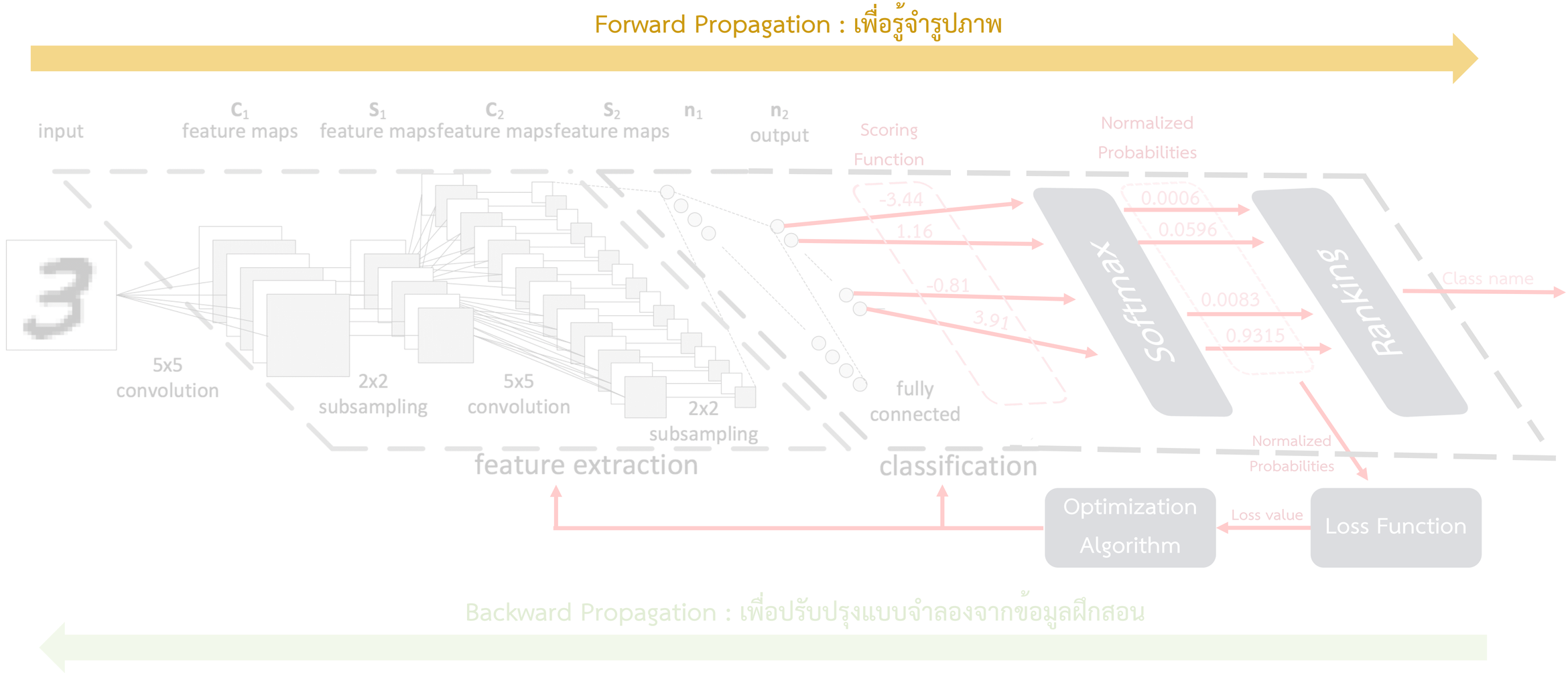


Backward Propagation : เพื่อปรับปรุงแบบจำลองจากข้อมูลฝึกสอน

หลักการโครงข่ายประสาทแบบคอนโวลูชัน

Forward Propagation คือ กระบวนการที่ป้อน Input Image เข้าสู่แบบจำลองที่ฝึกสอนเรียบร้อยแล้ว และคืนค่าเป็นเป็น Classname

Backward Propagation คือ กระบวนการที่นำค่า Normalized Probabilities ไปใช้เพื่อปรับปรุง Weight ของแบบจำลองให้เกิดความแม่นยำมากยิ่งขึ้น



หลักการรู้จำภาพ

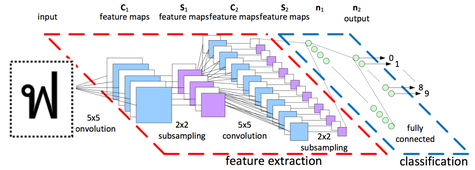
เฉลย
(Label)

ข้อมูลภาพ
(input image)

แบบจำลอง
(Model)

Training Phase
(กระบวนการฝึกสอน)

การสร้างแบบจำลอง



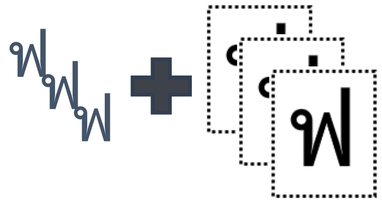
Testing Phase
(กระบวนการทดสอบ)

ข้อมูลภาพทดสอบ

การจัดแบ่งประเภท
Classification

การรู้จำรูปภาพด้วยหลักการโครงข่ายประสาทเทียมแบบคอนโวลูชันจะประกอบไปด้วย 2 ส่วน ได้แก่ (1) ส่วนของกระบวนการฝึกสอน และ (2) กระบวนการทดสอบ โดยที่ ***กระบวนฝึกสอน***เป็นการนำข้อมูลภาพ (input image) และผลเฉลย (Label) ไปฝึกสอนโครงข่ายประสาทเทียมแบบคอนโวลูชันที่ได้กำหนดไว้ร่วมกับวิธีการเรียนรู้ของเครื่องจักร เพื่อให้ได้รับแบบจำลองที่มีประสิทธิภาพและเกิดความแม่นยำสูง ขณะที่***กระบวนการทดสอบ***เป็นการนำภาพทดสอบไปผ่านแบบจำลองที่สร้างขึ้นและจะได้รับคำตอบว่าภาพดังกล่าวคืออะไร

หลักการรู้จำภาพ



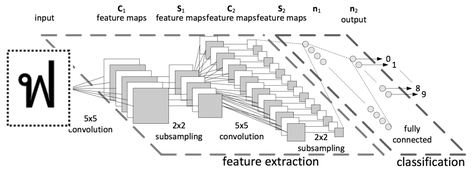


TrainingCNN.py Net.py

การสร้างแบบจำลอง



model.pt



Training Phase

(กระบวนการฝึกสอน)

Testing Phase

(กระบวนการทดสอบ)



การจัดแบ่งประเภท

Classification



TestingCNN.py

ฟ

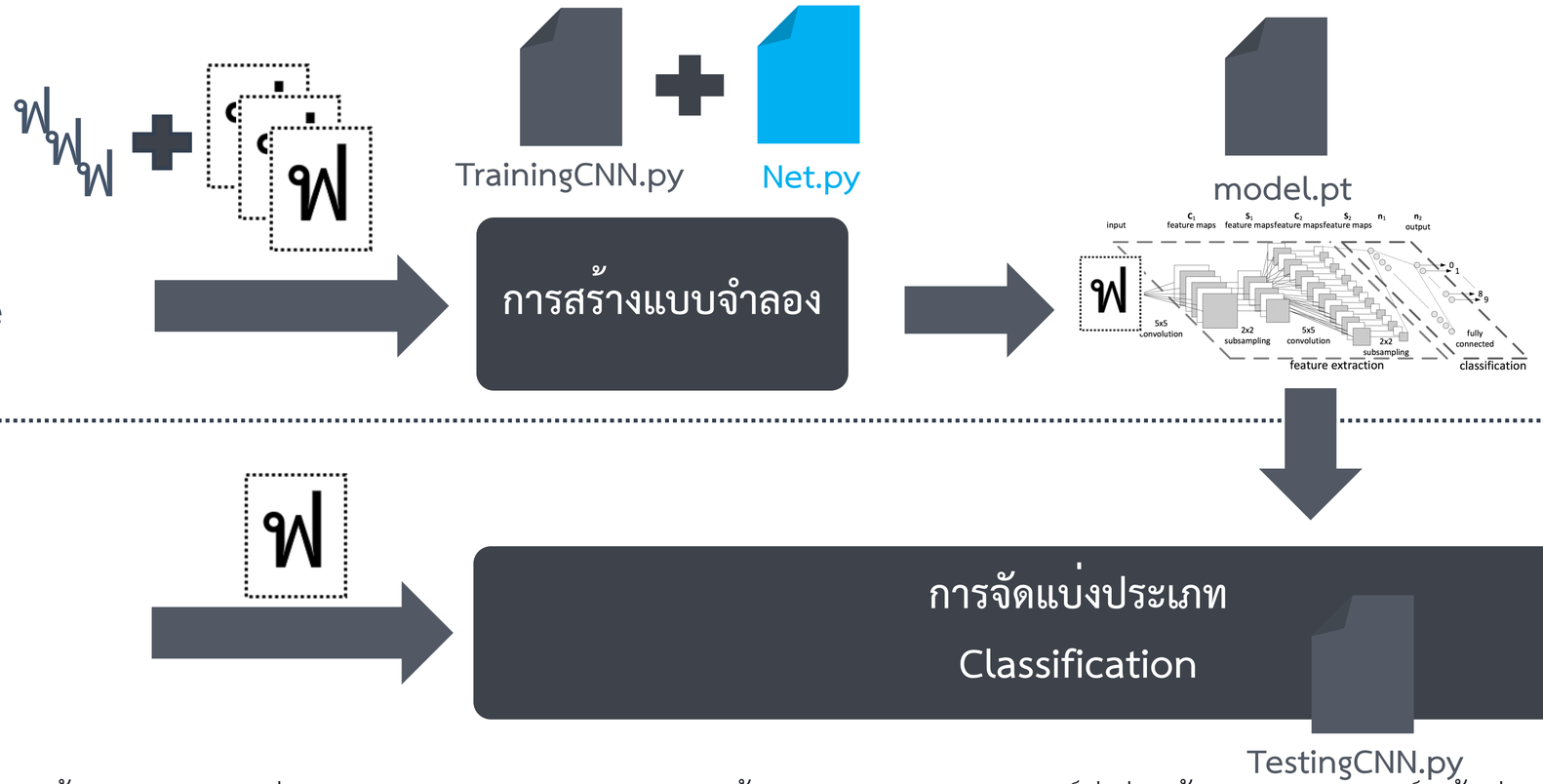
ดังนั้น เมื่อนักศึกษาได้โปรแกรมโครงข่ายประสาทเทียมแบบคอนโวลูชันด้วยภาษาไพทอนจะมีไฟล์ที่เกี่ยวข้องจำนวน 3 ไฟล์ ได้แก่

(1) *Net* เป็นไฟล์ที่ใช้กำหนดโครงสร้างของแบบจำลอง

(2) *TrainingCNN* เป็นไฟล์ที่เรียกใช้งานไฟล์ Net และนำมา***ฝึกสอน***แบบจำลอง

(3) *TestingCNN* เป็นไฟล์ที่เรียกใช้งานไฟล์ Net และนำมา***ทดสอบ***แบบจำลอง

หลักการรู้จำภาพ

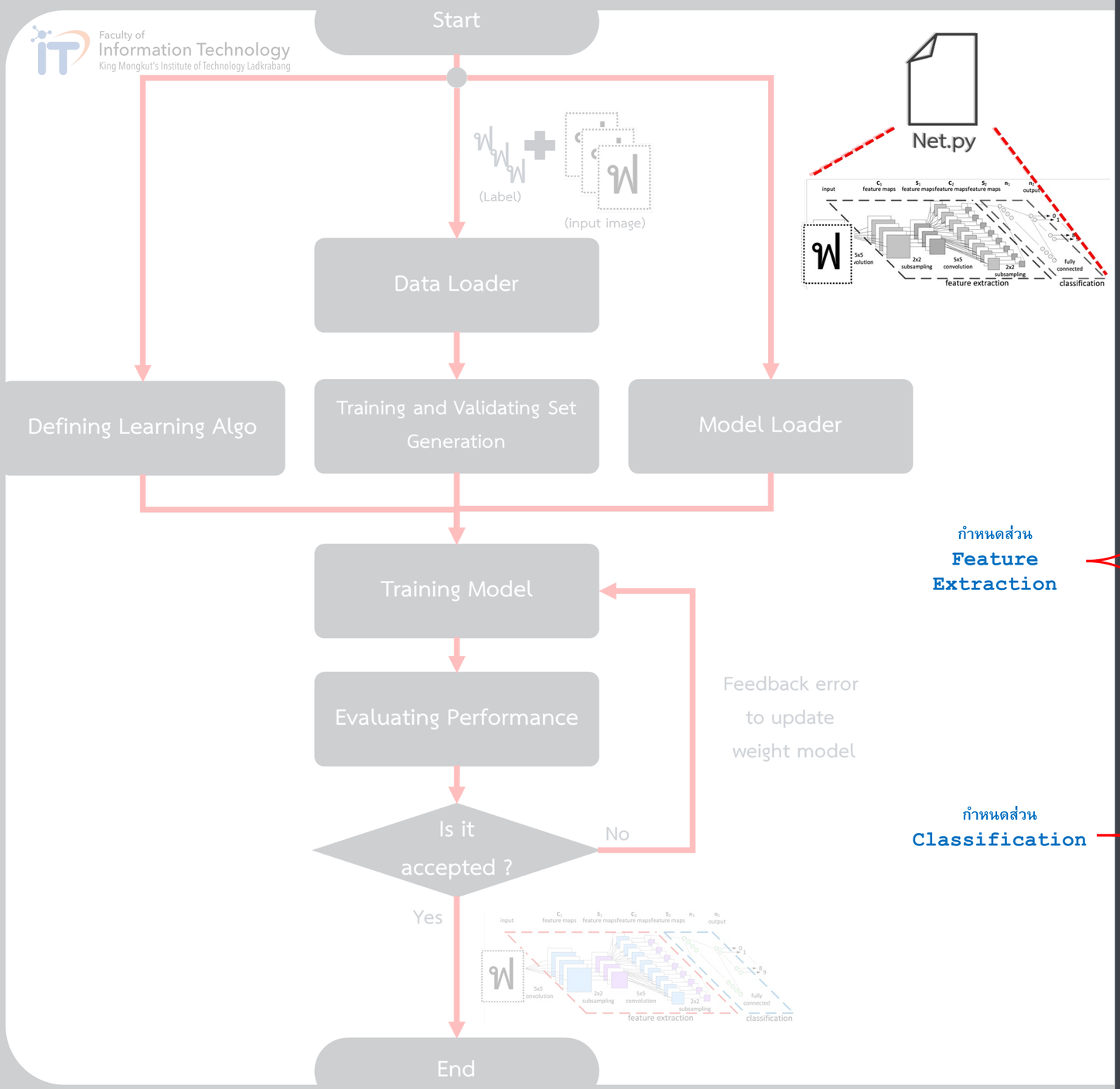


ดังนั้น เมื่อนักศึกษาได้โปรแกรมโครงข่ายประสาทเทียมแบบคอนโวลูชันด้วยภาษาไพทอนจะมีไฟล์ที่เกี่ยวข้องจำนวน 3 ไฟล์ ได้แก่

(1) *Net* เป็นไฟล์ที่ใช้กำหนดโครงสร้างของแบบจำลอง

(2) *TrainingCNN* เป็นไฟล์ที่เรียกใช้งานไฟล์ Net และนำมา***ฝึกสอน***แบบจำลอง

(3) *TestingCNN* เป็นไฟล์ที่เรียกใช้งานไฟล์ Net และนำมา***ทดสอบ***แบบจำลอง



Class Net

เป็นไฟล์ที่ใช้กำหนดโครงสร้างของแบบจำลอง

กำหนดส่วน **Feature Extraction**

กำหนดส่วน **Classification**

```
class Net(Module):
    def __init__(self): # กำหนดโครงสร้างใน Constructor
        super(Net, self).__init__()

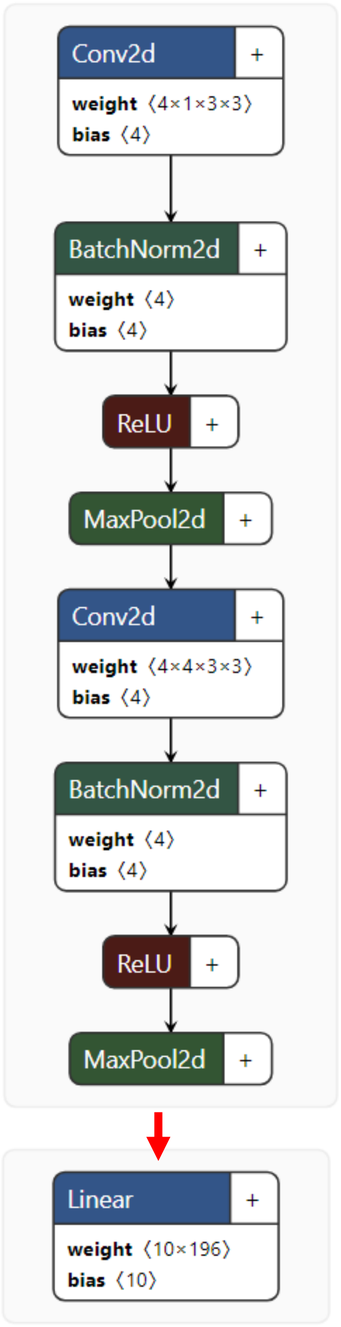
        ตั้งชื่อเองได้
        self.cnn_layers = Sequential(
            # Defining a 2D convolution layer
            Conv2d(1,4,kernel_size=3, stride=1, padding=1),
            BatchNorm2d(4),
            ReLU(inplace=True),
            MaxPool2d(kernel_size=2, stride=2),

            # Defining another 2D convolution layer
            Conv2d(4,4,kernel_size=3, stride=1, padding=1),
            BatchNorm2d(4),
            ReLU(inplace=True),
            MaxPool2d(kernel_size=2, stride=2),
        )

        ตั้งชื่อเองได้
        self.linear_layers = Sequential(
            Linear(4 * 7 * 7, 10)
        )

    # Defining the forward pass
    def forward(self, x):
        x = self.cnn_layers(x)
        x = x.view(x.size(0), -1)
        x = self.linear_layers(x)
        return x
```

Faculty of
Information Technology
King Mongkut's Institute of Technology Ladkrabang



นักศึกษาสามารถนำเสนอแบบจำลองในรูปแบบกราฟฟิกได้ผ่าน https://netron.app/



Class Net

เป็นไฟล์ที่ใช้กำหนดโครงสร้างของแบบจำลอง

กำหนดส่วน **Feature Extraction**

กำหนดส่วน **Classification**

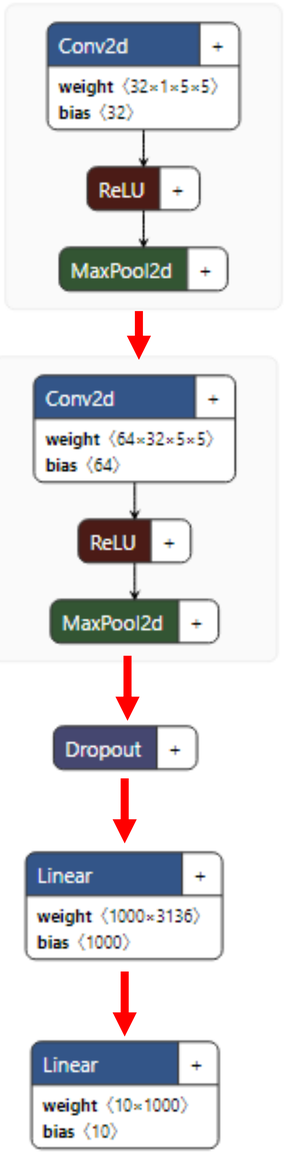
```
class Net(Module):
    def __init__(self): # กำหนดโครงสร้างใน Constructor
        super(Net, self).__init__()

        self.cnn_layers = Sequential(
            # Defining a 2D convolution layer
            Conv2d(1,4,kernel_size=3, stride=1, padding=1),
            BatchNorm2d(4),
            ReLU(inplace=True),
            MaxPool2d(kernel_size=2, stride=2),

            # Defining another 2D convolution layer
            Conv2d(4,4,kernel_size=3, stride=1, padding=1),
            BatchNorm2d(4),
            ReLU(inplace=True),
            MaxPool2d(kernel_size=2, stride=2),
        )

        self.linear_layers = Sequential(
            Linear(4 * 7 * 7, 10)
        )

    # Defining how the data flows through these layers when
      performing the forward pass through the network
    def forward(self, x):
        x = self.cnn_layers(x)
        # convert feature map to vector form
        x = x.view(x.size(0), -1)
        x = self.linear_layers(x)
        return x
```



นักศึกษาสามารถนำเสนอแบบจำลองในรูปแบบกราฟฟิกได้ผ่าน https://netron.app/



Class Net

เป็นไฟล์ที่ใช้กำหนดโครงสร้างของแบบจำลอง

```
class ConvNet(Module):
    def __init__(self):
        super(ConvNet, self).__init__()
        self.layer1 = Sequential(
            Conv2d(1, 32, kernel_size=5, stride=1, padding=2),
            ReLU(),
            MaxPool2d(kernel_size=2, stride=2))

        self.layer2 = Sequential(
            Conv2d(32, 64, kernel_size=5, stride=1, padding=2),
            ReLU(),
            MaxPool2d(kernel_size=2, stride=2))

        self.drop_out = Dropout()

        self.fc1 = Linear(7 * 7 * 64, 1000)

        self.fc2 = Linear(1000, 10)

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = out.reshape(out.size(0), -1)
        out = self.drop_out(out)
        out = self.fc1(out)
        out = self.fc2(out)
        return out

model = ConvNet()
torch.save(model, 'C:/ConvNet.pt')
```



Class Net

เป็นไฟล์ที่ใช้กำหนดโครงสร้างของแบบจำลอง

```
class ConvNet(Module):
    def __init__(self):
        super(ConvNet, self).__init__()
        self.layer1 = Sequential(
            Conv2d(1, 32, kernel_size=5, stride=1, padding=2),
            ReLU(),
            MaxPool2d(kernel_size=2, stride=2))

        self.layer2 = Sequential(
            Conv2d(32, 64, kernel_size=5, stride=1, padding=2),
            ReLU(),
            MaxPool2d(kernel_size=2, stride=2))

        self.drop_out = Dropout()

        self.fc1 = Linear(7 * 7 * 64, 1000)

        self.fc2 = Linear(1000, 10)

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = out.reshape(out.size(0), -1)
        out = self.drop_out(out)
        out = self.fc1(out)
        out = self.fc2(out)
        return out

model = ConvNet()
torch.save(model, 'C:/ConvNet.pt')
```

นักศึกษาสามารถนำเสนอแบบจำลองในรูปแบบกราฟฟิกได้ผ่าน https://netron.app/

เปรียบเทียบการเขียนโค้ดในการสร้างแบบจำลอง

```python
class TinyConv(nn.Module):
    """Tiny convolutional neural network.
    Uses nn.Sequential to improve organization."""
    def __init__(self):
        super(TinyConv, self).__init__()
        self.conv = nn.Sequential(
                            nn.Conv2d(3, 8, kernel_size = (3,3), stride=(3,3), padding=0),
                            nn.ReLU(inplace=True),

                            nn.Conv2d(8, 16, kernel_size = (3,3), stride=(3,3), padding=0),
                            nn.ReLU(inplace=True))
        self.fc = nn.Linear(in_features=16*3*3,out_features=3)

    def forward(self, x):
        x = self.conv(x)
        x = torch.flatten(x,start_dim=1,end_dim=-1)
        x = self.fc(x)
        out = {'out':x}
        return out
```



```python
class TinyConvLessOrganized(nn.Module):
    """Tiny convolutional neural network.
    This model is less organized because it doesn't use nn.Sequential."""
    def __init__(self):
        super(TinyConvLessOrganized, self).__init__()
        self.conv1 = nn.Conv2d(3, 8, kernel_size = (3,3), stride=(3,3), padding=0)
        self.relu = nn.ReLU(inplace=True)
        self.conv2 = nn.Conv2d(8, 16, kernel_size = (3,3), stride=(3,3), padding=0)
        self.fc = nn.Linear(in_features=16*3*3,out_features=3)

    def forward(self, x):
        x = self.conv1(x)
        x = self.relu(x)
        x = self.conv2(x)
        x = self.relu(x)
        x = torch.flatten(x,start_dim=1,end_dim=-1)
        x = self.fc(x)
        out = {'out':x}
        return out
```

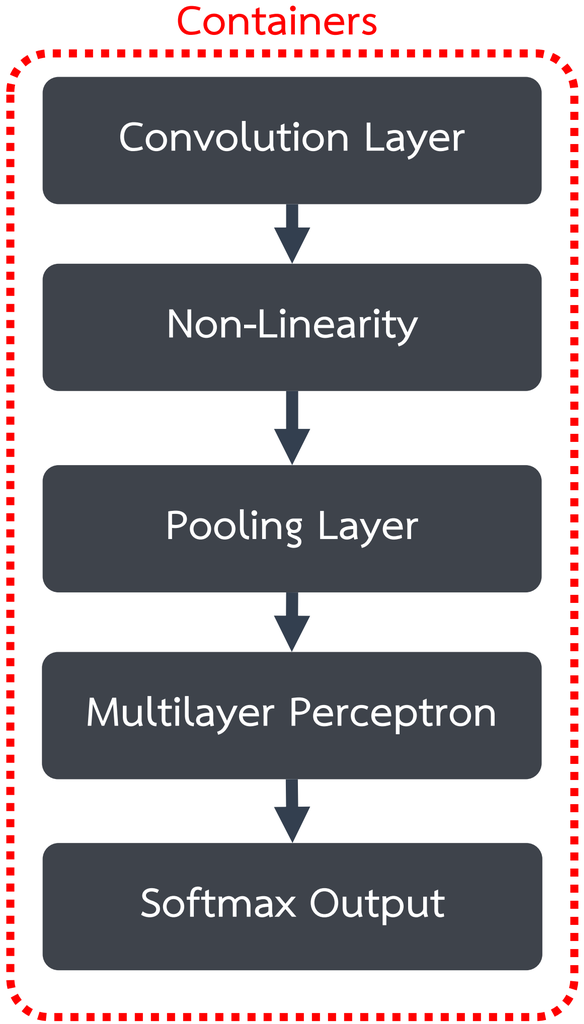


อย่างไรก็ตาม คำสั่ง nn.Sequential ไม่จำเป็นต้องเรียกใช้งาน แต่โดยส่วนใหญ่แล้ว นิยมแนะนำให้เรียกใช้งานคำสั่ง nn.Sequential มากกว่า เนื่องจากช่วยให้โค้ดมีระเบียบมากขึ้นและเป็นการสร้างกลุ่มของ Layer หรือเรียกว่า Block

อ้างอิง https://glassboxmedicine.com/2021/02/06/designing-custom-2d-and-3d-cnns-in-pytorch-tutorial-with-code/

Containers

หมายถึง คลาสที่เป็นนามธรรม (abstract module class) ที่ได้กำหนดว่าโครงข่ายประสาทเทียมต้องมีเมธอดใดบ้าง เพื่อเป็นการบังคับว่าทุกโครงข่ายประสาทเทียมต้องมีเมธอดเหล่านี้ ซึ่งประกอบไปด้วย 3 เมธอด ได้แก่



- **`add(module)`** *เพื่อสำหรับเพิ่มโมดูลลงใน container เป็นลำดับ*
- **`get(index)`** *เพื่อคืนค่าโมดูลที่อยู่ใน container ลำดับที่ index*
- **`size()`** *เพื่อคืนค่าจำนวนโมดูลใน Container*

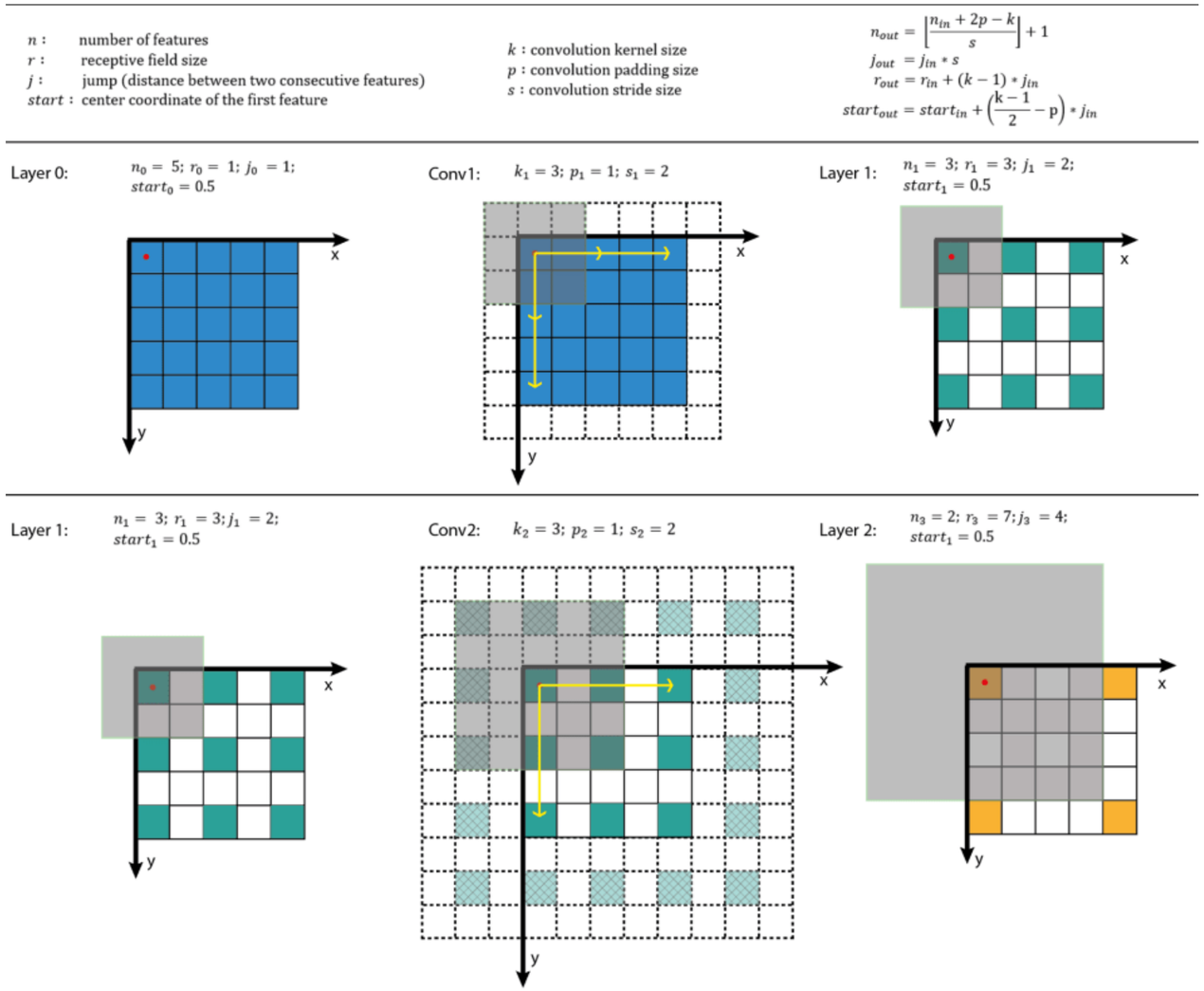
ซึ่งใน PyTorch จะมีคลาสที่เป็นนามธรรมสำหรับโครงข่ายประสาทเทียมแบบซับซ้อน (Complex neural networks) ที่สำคัญดังนี้ Module และ Sequential

ชื่อ	ความหมาย
Module	เป็นพื้นของทุกคลาสที่เกี่ยวข้องกับโมดูลโครงข่ายประสาทเทียม
Sequential	เป็น Container แบบเชื่อมต่อเลเยอร์ในลักษณะ fully แบบ feed-forward

อ้างอิง https://nn.readthedocs.io/en/rtd/containers และ https://pytorch.org/docs/stable/nn.html

2D Convolution Layer

หมายถึง ส่วนที่มีหน้าที่สกัดคุณลักษณะที่สำคัญจากภาพ โดยอาศัยตัวดำเนินการคอนโวลูชั่นรวมกับตัวกรองที่ค่าสัมประสิทธิทุกค่าบนตัวกรองจะมีการปรับเหมาะอัตโนมัติตามค่าความผิดพลาดที่เกิดขึ้นจากแบบจำลอง




```
torch.nn.Conv2d(
    in_channels, out_channels, kernel_size,
    stride=1,  padding=0, dilation=1, groups=1,
    bias=True, padding_mode='zeros'
)
```

หมายเหตุ สีเหลือง คือ บังคับต้องระบุ

คำอธิบายตัวแปร

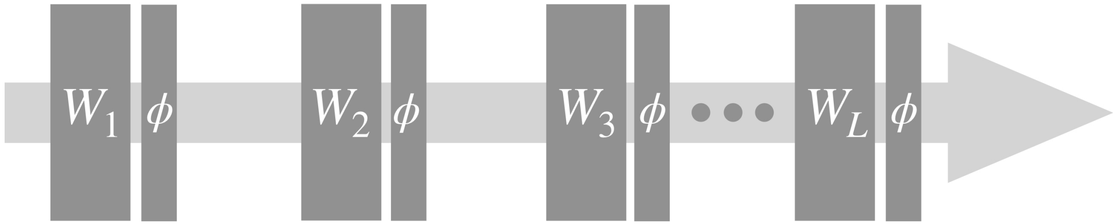
- ***in_channels (int)*** คือ จำนวน channels ของภาพนำเข้า (input image)
  (ภาพสีมี 3 channels ขณะที่ภาพระดับสีเทาและภาพขาวดำมี 1 channel)
- ***out_channels (int)*** คือ จำนวน channels ของผลลัพธ์ที่ผ่านการ conv
- ***kernel_size (int or tuple)*** คือ ขนาดของตัวกรอง conv
- ***stride (int or tuple)*** คือ จำนวนช่องในการขยับตัวกรอง
- ***padding (int or tuple)*** คือ ระบุจำนวนชั้นที่เติมข้อมูล
- ***padding_mode (string)*** คือ วิธีการเติมข้อมูล

อ้างอิง https://syncedreview.com/2017/05/11/a-guide-to-receptive-field-arithmetic-for-convolutional-neural-networks/

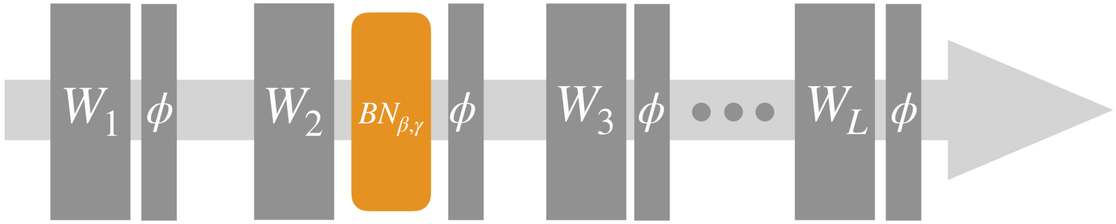
2D Batch Normalization

หมายถึง ส่วนที่ทำหน้าที่ (1) ปรับช่วงค่าข้อมูลให้อยู่ในมาตรฐานเดียวกัน หรือ (2) อยู่ในขอบเขตที่กำหนด หรือ (3) ปรับค่าอยู่ในช่วงเดียวกัน (Scaling Data) ก่อนออกจาก Layer ไปเป็น input ให้ Layer ถัดไป ซึ่งสามารถ (1) ลดอิทธิพลของ feature ไม่ให้ตัวใดตัวหนึ่งมีอิทธิพลมากเกินไป (2) สามารถช่วยเพิ่มความเร็วในการฝึกสอนแบบจำลอง และ (3) ทำให้ค่า Loss ลดลงได้มากกว่า

Standard Network



Adding a BatchNorm layer (between weights and activation function)

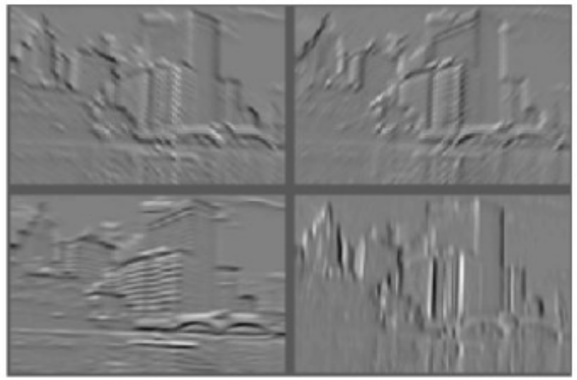



```
torch.nn.BatchNorm2d(
    num_features, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True
)
```

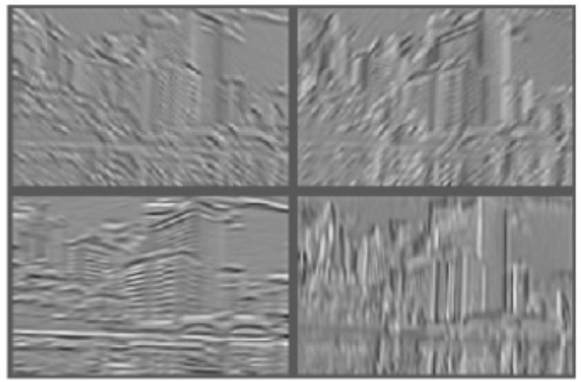
คำอธิบายตัวแปร

หมายเหตุ สีเหลือง คือ บังคับต้องระบุ

- ***num_features*** คือ จำนวน channels ของ input ใน Layer นี้



Feature Maps



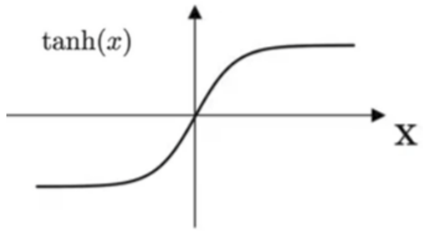
Feature Maps After Contrast Normalization

อ้างอิง https://gradientscience.org/batchnorm/

Non-linear Activation Layer

หมายถึง ส่วนที่ทำให้ผลลัพธ์ของ neuron มีความเป็น Non-linearity ซึ่งสามารถกล่าวได้ว่า Activation function เป็นการแปลงข้อมูลแบบเชิงเส้นให้อยู่ในรูปแบบไม่เชิงเส้นแบบหนึ่ง (Non-linear transformation).

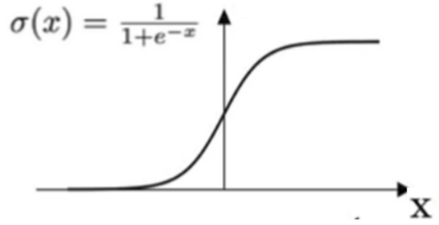
Hyper Tangent Function



Tangent Hyperbolic Function

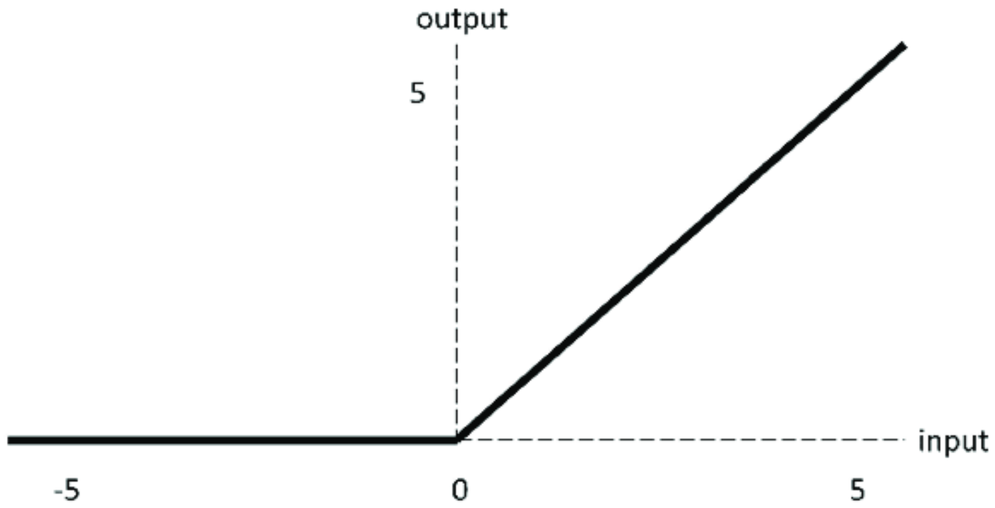
```
torch.nn.Tanh()
```

Sigmoid Function



Sigmoid

```
torch.nn.Sigmoid()
```



Rectified Linear Unit Function

```
torch.nn.ReLU()
```



2D Pooling Layer

หมายถึง กระบวนการ Non-linear และ down-sampling รูปแบบหนึ่งที่พยายามจะแปลงข้อมูลจาก Convolution layer ให้มีความชัดเจนมากขึ้น ซึ่งการทำ pooling สามารถลดเวลาในการประมวลและขนาดของ Feature map ของเลเยอร์ต้น ๆ ได้ดี อีกทั้งยังถือว่าเป็นการสรุปข้อมูลในเบื้องต้นอีกด้วย

2D Max Pooling Layer

```
torch.nn.MaxPool2d(
    kernel_size: (int or Tuple),
    stride: None, padding: 0, dilation: 1, return_indices: False, ceil_mode: False
)
```

หมายเหตุ สีเหลือง คือ บังคับต้องระบุ

โดยที่

- ***kernel_size (int or tuple)*** คือ ขนาดของตัวกรอง conv

2D Average Pooling Layer

```
torch.nn.AvgPool2d(
    kernel_size: (int or Tuple),
    stride: None, padding: 0, ceil_mode: False, count_include_pad: True, divisor_override: None
)
```

หมายเหตุ สีเหลือง คือ บังคับต้องระบุ

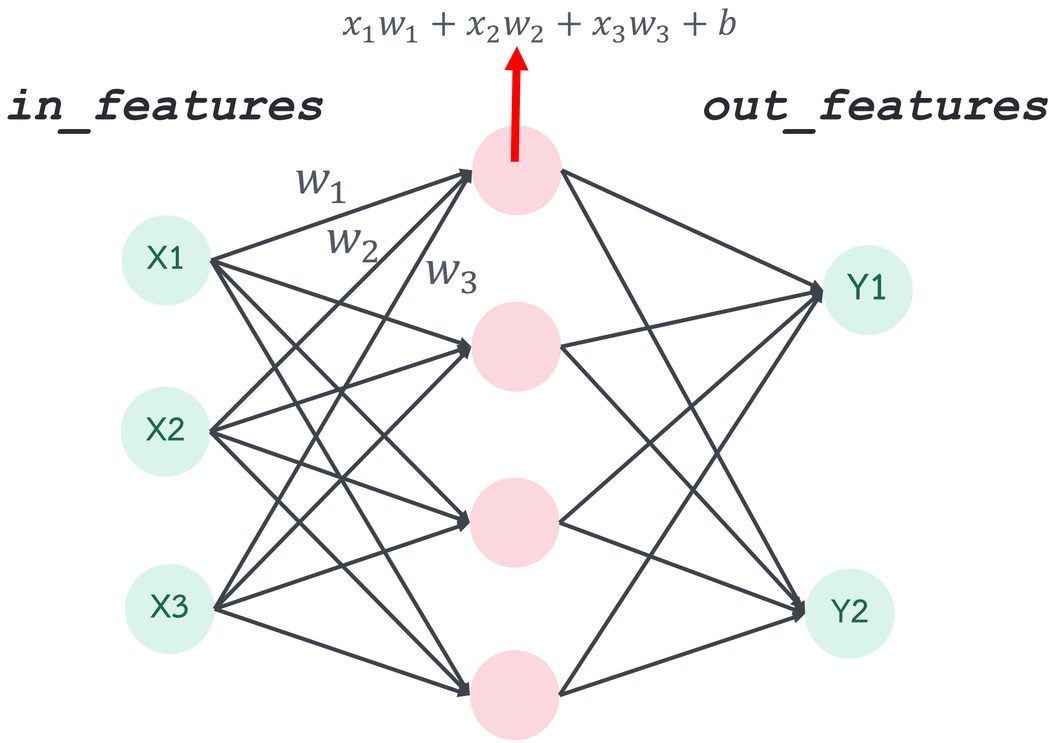
โดยที่

- ***kernel_size (int or tuple)*** คือ ขนาดของตัวกรอง conv

อ้างอิง https://pytorch.org/docs/stable/generated/torch.nn.MaxPool2d.html#torch.nn.MaxPool2d

Fully Connected Layer

คือ การทำงานร่วมกันของ Input Layer, Hidden Layer และ Output Layer ซึ่งมีหน้าที่ในเรียนรู้ให้เครื่องจักรสามารถจำแนกประเภทของวัตถุได้ โดยขนาดของ Output Layer จะมีค่าเท่ากับ (1) จำนวนของคลาสที่ต้องการ หรือ (2) ขนาดของ Input Layer ใน Fully-Connected Layer ถัดไป




```
torch.nn.Linear(
    in_features: int, out_features: int,
    bias: bool = True
)
```

หมายเหตุ สีเหลือง คือ บังคับต้องระบุ

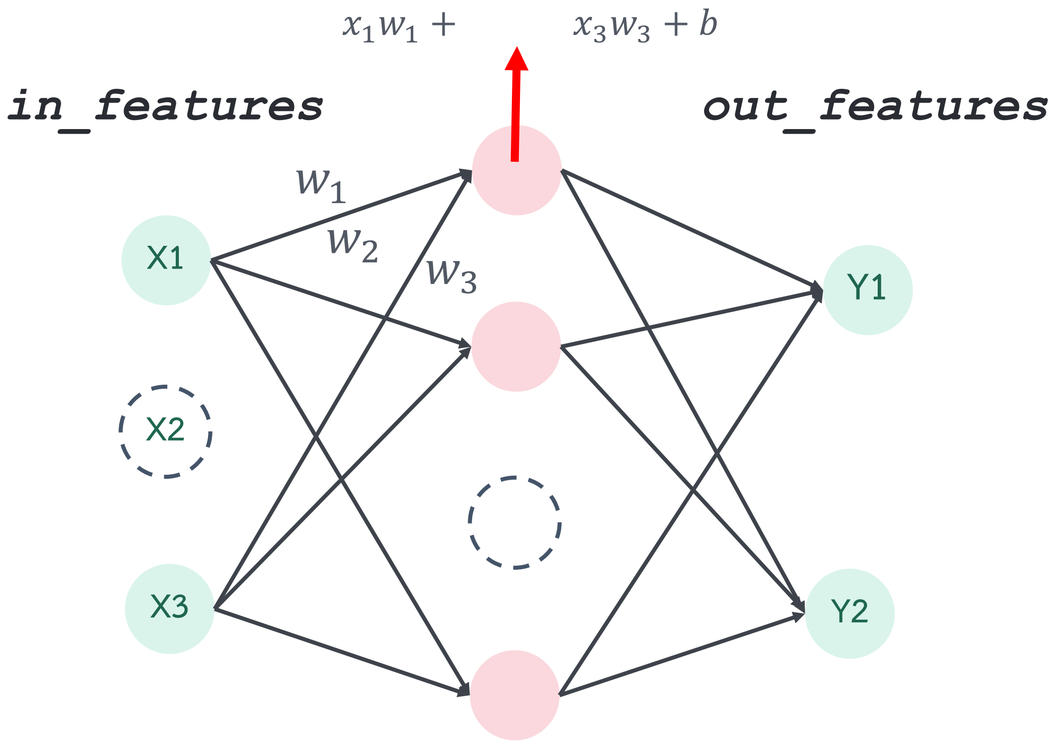
โดยที่

- ***in_features*** คือ ขนาดของ input (ได้แก่ feature หรือ output ของ Layer ก่อนหน้า) ในรูปแบบของเวกเตอร์
- ***out_features*** คือ ขนาดของ output ที่ได้จาก Fully Connected Layer ปัจจุบัน

อ้างอิง https://pytorch.org/docs/stable/generated/torch.nn.Linear.html#torch.nn.Linear

Dropout Layer

คือ ส่วนที่มีหน้าที่สุ่มลบ neuron ลงจากแบบจำลองเพื่อทำให้แบบจำลองมีประสิทธิภาพที่ดีขึ้น เนื่องจากโครงข่ายประสาทเทียมแบบ feedforward ขนาดใหญ่ที่ได้รับการฝึกสอนด้วยชุดข้อมูลขนาดเล็กอาจทำให้ประสิทธิภาพของแบบจำลองแย่เพราะเกิดมาเหตุการณ์ "overfitting” (ช่วงที่แนะนำ 0.2 – 0.5)




```
torch.nn.Dropout(
    p: 0.5, inplace: False
)
```

โดยที่

- ***p*** คือ ค่าความน่าจะเป็นสำหรับการลบ neuron ออกจากแบบจำลอง

การลบในทางปฏิบัติ คือ การกำหนดให้ค่าถ่วงน้ำหนักเท่ากับ 0 เช่น

$$x_1w_1 + x_2w_2 + x_3w_3 + b$$

เมื่อกำหนดให้ w_2 เป็น 0 จะพบว่า

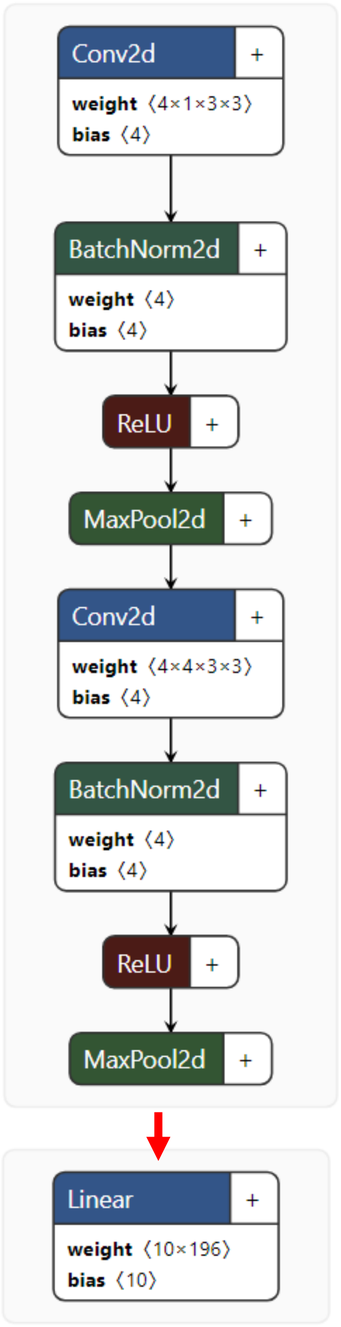
$$x_1w_1 + x_2(0) + x_3w_3 + b$$

หรือสามารถเขียนได้ว่า

$$x_1w_1 + \qquad x_3w_3 + b$$

อ้างอิง https://arxiv.org/abs/1207.0580v1

Faculty of
Information Technology
King Mongkut's Institute of Technology Ladkrabang



นักศึกษาสามารถนำเสนอแบบจำลองในรูปแบบกราฟฟิกได้ผ่าน https://netron.app/



Class Net

เป็นไฟล์ที่ใช้กำหนดโครงสร้างของแบบจำลอง

กำหนดส่วน **Feature Extraction**

กำหนดส่วน **Classification**

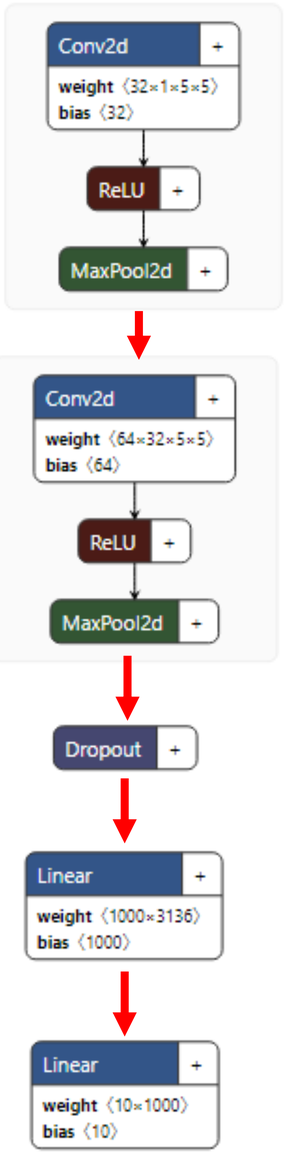
```
class Net(Module):
    def __init__(self): # กำหนดโครงสร้างใน Constructor
        super(Net, self).__init__()

        self.cnn_layers = Sequential(
            # Defining a 2D convolution layer
            Conv2d(1,4,kernel_size=3, stride=1, padding=1),
            BatchNorm2d(4),
            ReLU(inplace=True),
            MaxPool2d(kernel_size=2, stride=2),

            # Defining another 2D convolution layer
            Conv2d(4,4,kernel_size=3, stride=1, padding=1),
            BatchNorm2d(4),
            ReLU(inplace=True),
            MaxPool2d(kernel_size=2, stride=2),
        )

        self.linear_layers = Sequential(
            Linear(4 * 7 * 7, 10)
        )

    # Defining how the data flows through these layers when
      performing the forward pass through the network
    def forward(self, x):
        x = self.cnn_layers(x)
        # convert feature map to vector form
        x = x.view(x.size(0), -1)
        x = self.linear_layers(x)
        return x
```



นักศึกษาสามารถนำเสนอแบบจำลองในรูปแบบกราฟฟิกได้ผ่าน https://netron.app/



Class Net

เป็นไฟล์ที่ใช้กำหนดโครงสร้างของแบบจำลอง

```
class ConvNet(Module):
    def __init__(self):
        super(ConvNet, self).__init__()
        self.layer1 = Sequential(
            Conv2d(1, 32, kernel_size=5, stride=1, padding=2),
            ReLU(),
            MaxPool2d(kernel_size=2, stride=2))

        self.layer2 = Sequential(
            Conv2d(32, 64, kernel_size=5, stride=1, padding=2),
            ReLU(),
            MaxPool2d(kernel_size=2, stride=2))

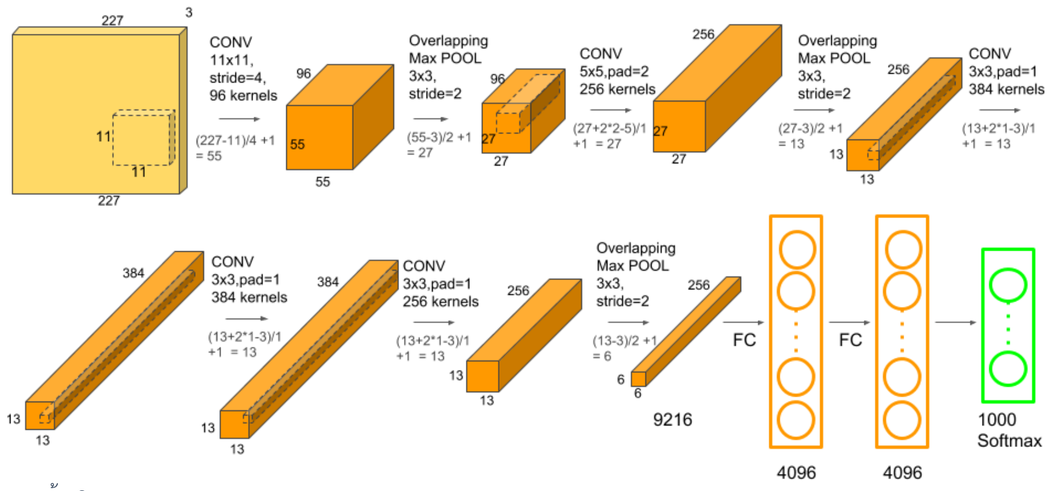
        self.drop_out = Dropout()

        self.fc1 = Linear(7 * 7 * 64, 1000)

        self.fc2 = Linear(1000, 10)

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = out.reshape(out.size(0), -1)
        out = self.drop_out(out)
        out = self.fc1(out)
        out = self.fc2(out)
        return out

model = ConvNet()
torch.save(model, 'C:/ConvNet.pt')
```



อ้างอิง https://www.linkedin.com/pulse/alexnet-architecture-changed-cnns-nedhir-ben-hammouda/

- inplace=True ใน PyTorch ใช้เพื่อกำหนดว่าต้องการให้การดำเนินการ (operation) นั้นเกิดขึ้น **ในตัวมันเอง** โดยไม่ต้องสร้าง Tensor ใหม่ ผลลัพธ์ของการดำเนินการจะถูกเก็บทับบนตัวแปร Tensor เดิมแทนที่จะคืนค่าผลลัพธ์เป็น Tensor ใหม่ ซึ่งช่วยลดการใช้หน่วยความจำในการคำนวณ

- คำสั่ง nn.AdaptiveAvgPool2d((6, 6)) จะปรับเพียง Width และ Height เท่านั้น จะไม่ยุ่งกับ Batch Size และ Channel ตัวอย่างเช่น กำหนดให้ Tensor Shape (Batch Size, Channel, Height, Width)
  - ข้อมูลนำเข้า (1, 3, 10, 14)
  - ข้อมูลส่งออก (1, 3, 6, 6)

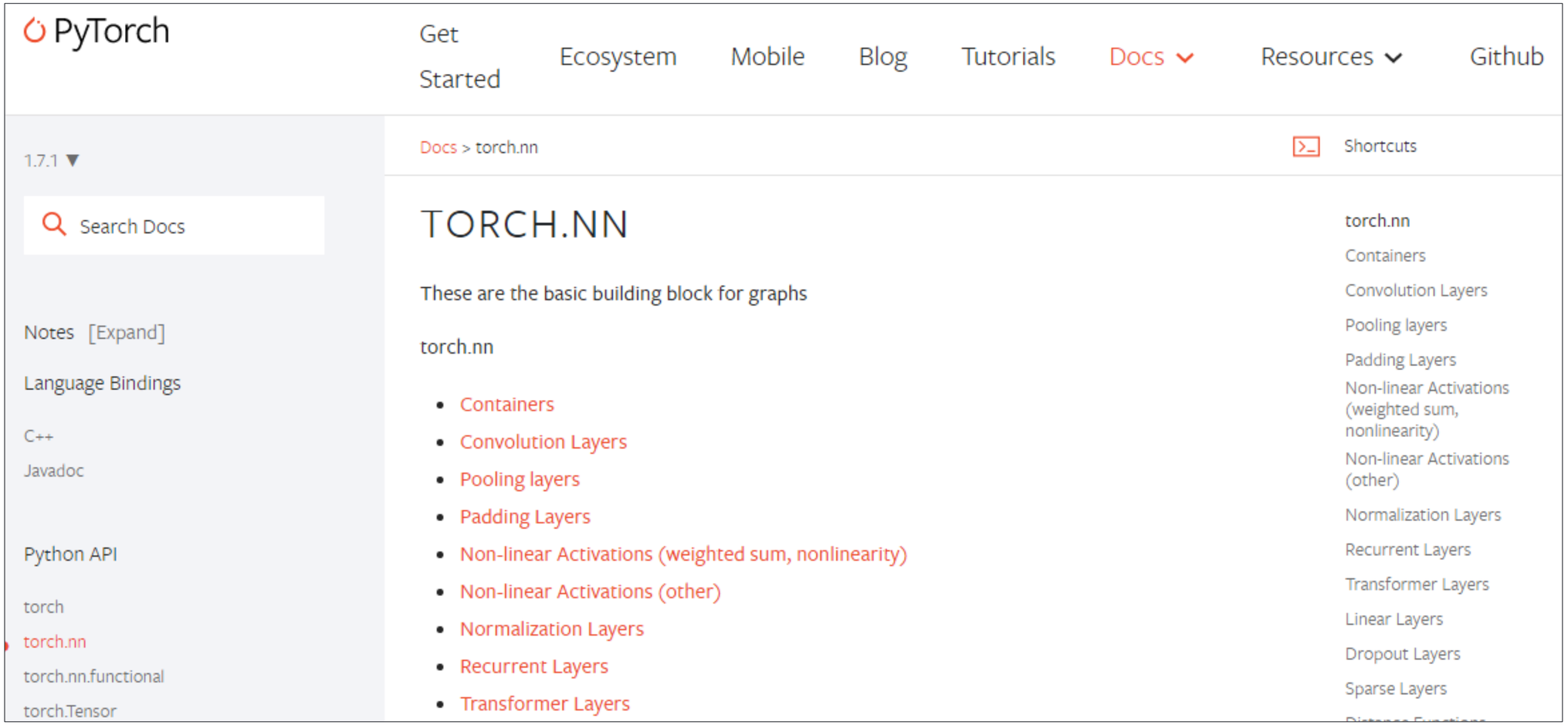
```python
class AlexNet(nn.Module):
    def __init__(self, num_classes: int = 1000, dropout: float = 0.5) -> None:
        super().__init__()
        _log_api_usage_once(self)
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 192, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(192, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )
        self.avgpool = nn.AdaptiveAvgPool2d((6, 6))
        self.classifier = nn.Sequential(
            nn.Dropout(p=dropout),
            nn.Linear(256 * 6 * 6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(p=dropout),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Linear(4096, num_classes),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x
```

อ้างอิง https://github.com/pytorch/vision/blob/main/torchvision/models/alexnet.py

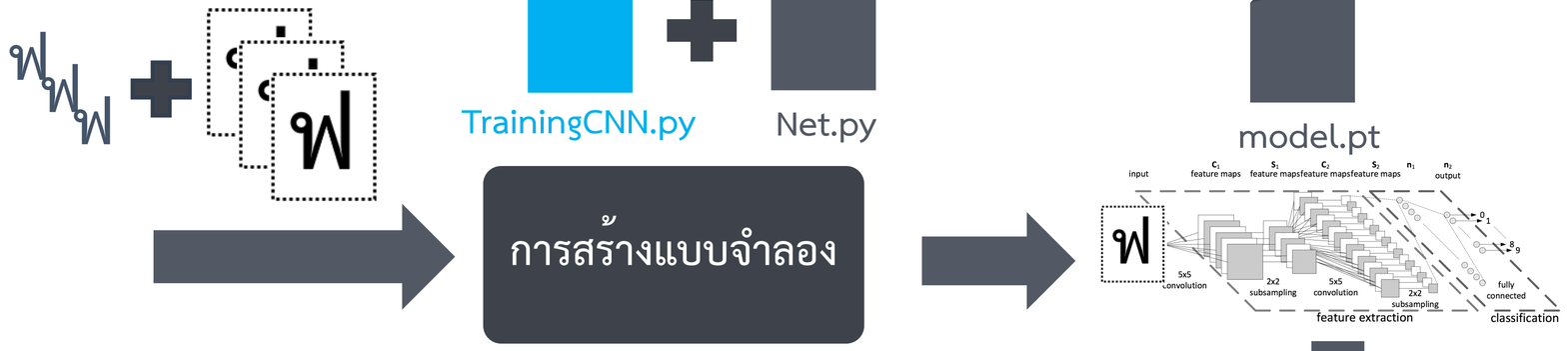
รายละเอียดเพิ่มเติม

สิ่งที่ปรากฏในสไลน์เป็นเพียงแค่ข้อมูลบางส่วนซึ่งมีการพูดถึงหรือการใช้งานอย่างกว้างขวางและพบเจอมาก ซึ่งนักศึกษาสามารถหาข้อมูลหรือทางเลือกอื่น ๆ ในการกำหนดโครงสร้างได้จากเว็บไซต์ต่อไปนี้



https://pytorch.org/docs/stable/nn.html

หลักการรู้จำภาพ



Training Phase

(กระบวนการฝึกสอน)

Testing Phase

(กระบวนการทดสอบ)



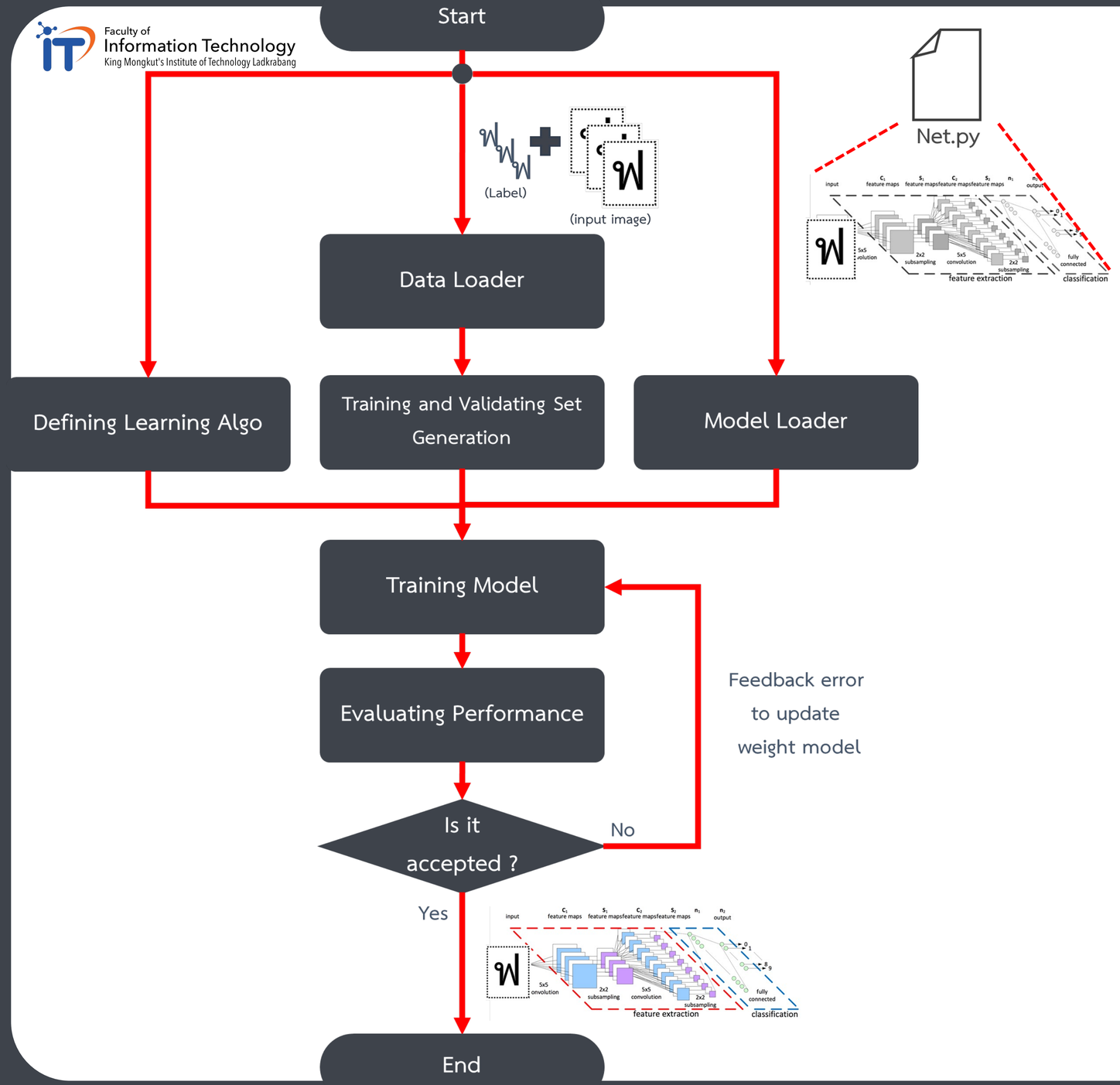
ดังนั้น เมื่อนักศึกษาได้โปรแกรมโครงข่ายประสาทเทียมแบบคอนโวลูชันด้วยภาษาไพทอนจะมีไฟล์ที่เกี่ยวข้องจำนวน 3 ไฟล์ ได้แก่

(1) *Net* เป็นไฟล์ที่ใช้กำหนดโครงสร้างของแบบจำลอง

(2) *TrainingCNN* เป็นไฟล์ที่เรียกใช้งานไฟล์ Net และนำมา***ฝึกสอน***แบบจำลอง

(3) *TestingCNN* เป็นไฟล์ที่เรียกใช้งานไฟล์ Net และนำมา***ทดสอบ***แบบจำลอง

Faculty of
Information Technology
King Mongkut's Institute of Technology Ladkrabang

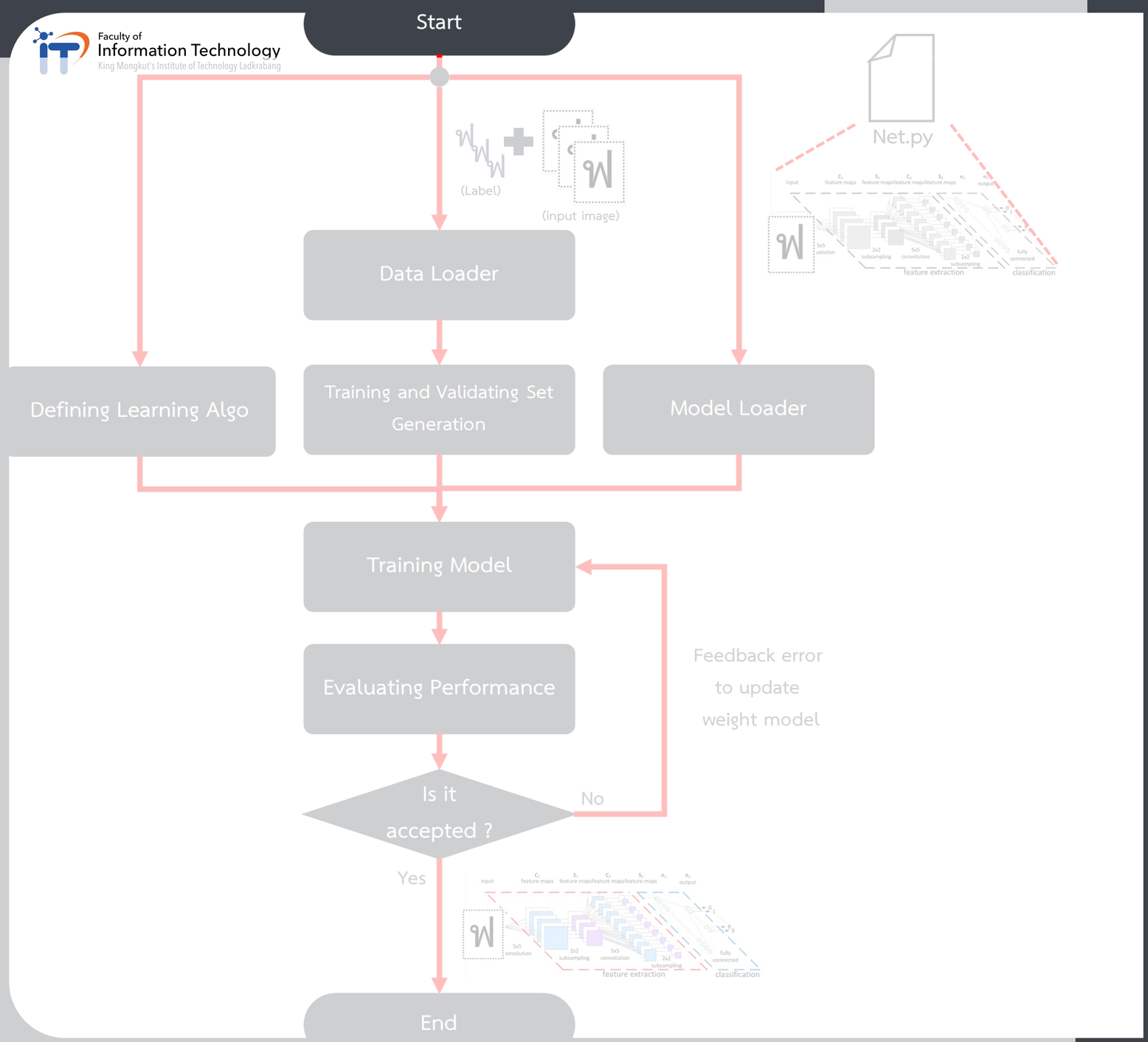


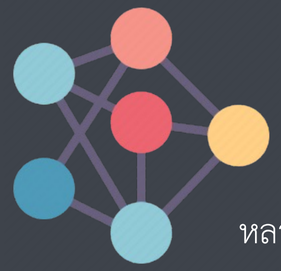
ขั้นตอนการฝึกสอนเครื่องจักร

จะเขียนโปรแกรมลงในไฟล์ TrainingCNN.py ซึ่งจะประกอบ ด้วย 7 ขั้นตอนหลัก ได้แก่

- **Data Loader**
  การโหลดข้อมูลรูปภาพและเฉลยเข้าสู่โปรแกรม
- **Training and Validating Set Generation**
  แบ่งข้อมูลออกเป็น 2 ชุด ได้แก่ ชุดฝึกสอนและชุดทดสอบ
- **Model Loader**
  การโหลดโครงสร้างของสถาปัตยกรรมที่จะใช้งานเข้าสู่โปรแกรม
- **Defining Learning Algorithm**
  การกำหนดวิธีการเรียนรู้และพารามิเตอร์ต่าง ๆ
- **Training Model**
  ฝึกสอนแบบจำลอง
- **Evaluating Performance**
  คำนวณค่าความผิดพลาดและค่าความแม่นยำ
- **Is it accepted ?**
  ประเมินผลว่าแบบจำลองมีประสิทธิภาพหรือไม่

Faculty of
Information Technology
King Mongkut's Institute of Technology Ladkrabang





Import ชุดคำสั่งที่จำเป็น

มีความจำเป็นต้องใช้คำสั่งต่าง ๆ มากมายที่มาจากหลากหลายชุดคำสั่ง เพื่อให้สามารถฝึกสอนเครื่องจักรได้

```
import pandas as pd
import numpy as np
from Net import Net

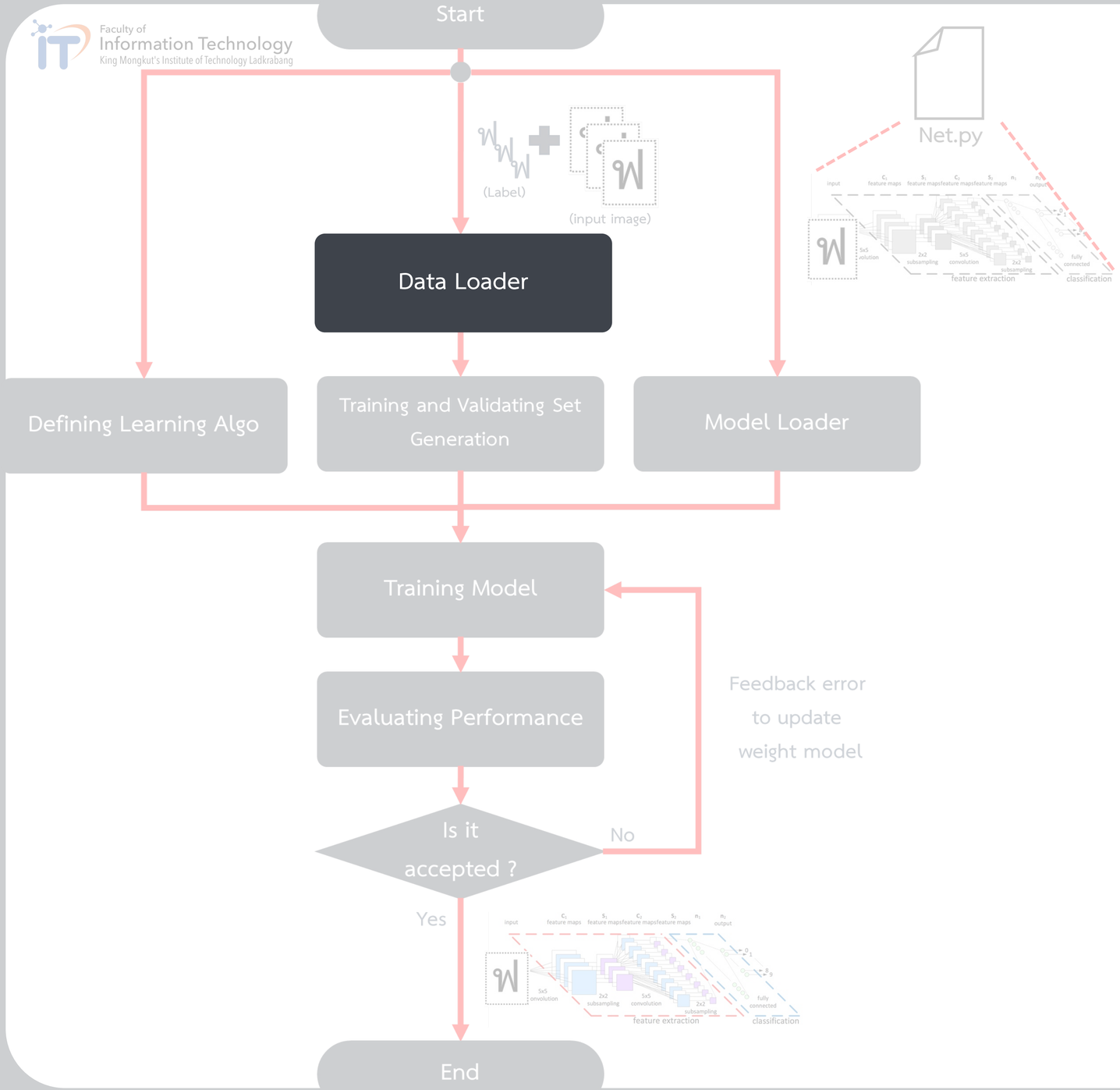
# for reading and displaying images
from skimage.io import imread
import matplotlib.pyplot as plt

# for creating validation set
from sklearn.model_selection import train_tes
t_split

# for evaluating the model
from sklearn.metrics import accuracy_score
from tqdm import tqdm

# PyTorch libraries and modules
import torch
from torch.autograd import Variable
from torch.nn import Linear, ReLU, CrossEntro
pyLoss, Sequential, Conv2d, MaxPool2d, Module
, Softmax, BatchNorm2d, Dropout
from torch.optim import Adam, SGD
```

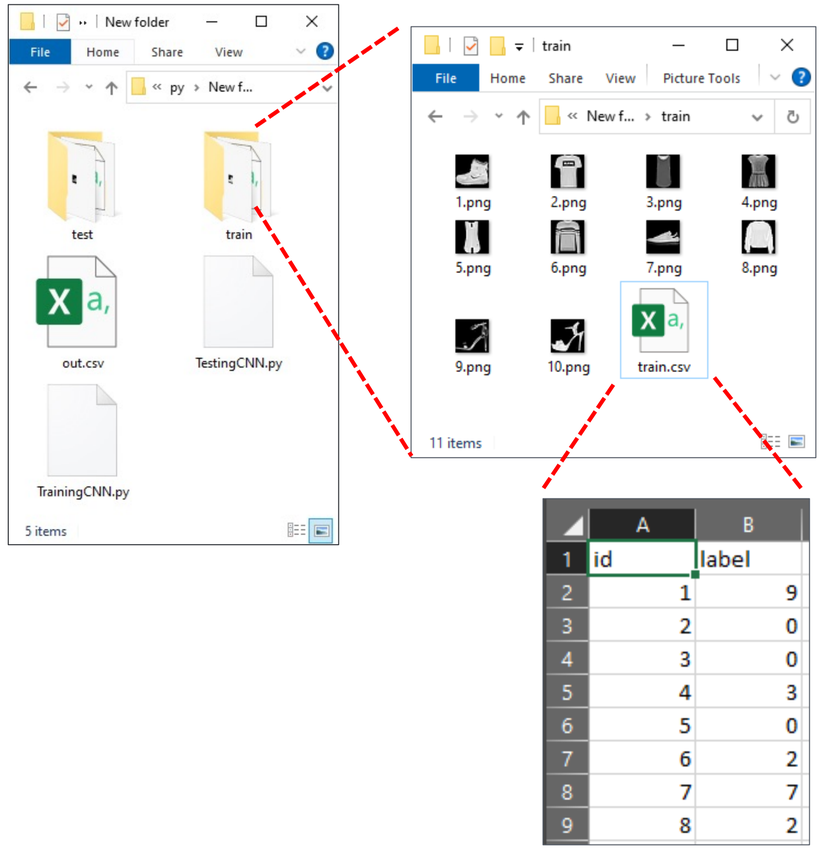
Faculty of Information Technology
King Mongkut's Institute of Technology Ladkrabang



Data Loader

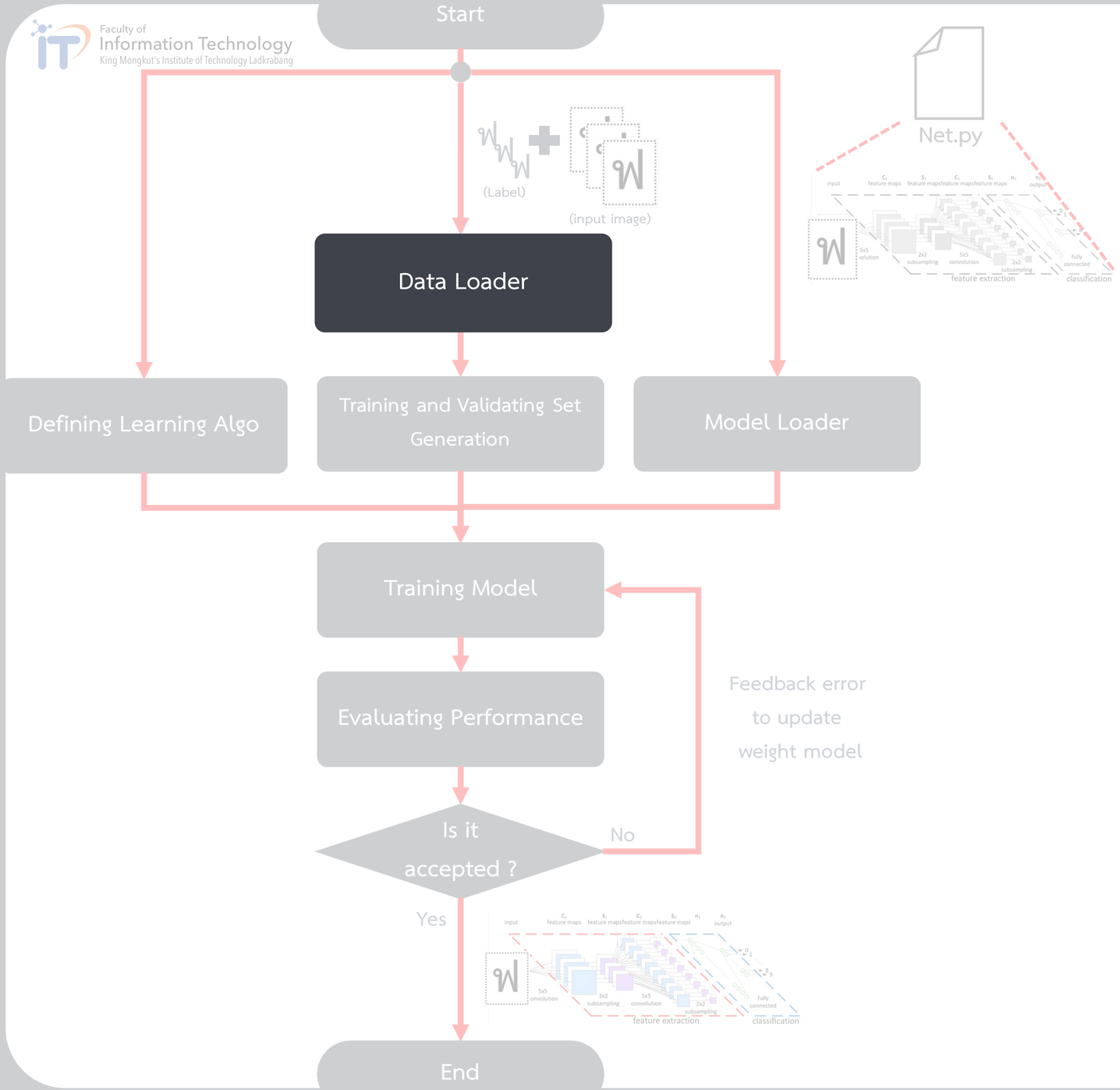
การโหลดข้อมูลรูปภาพและเฉลยเข้าสู่โปรแกรม

```
train = pd.read_csv('C:/train/train.csv')
train.head()
```



	A	B
1	id	label
2	1	9
3	2	0
4	3	0
5	4	3
6	5	0
7	6	2
8	7	7
9	8	2

Faculty of
Information Technology
King Mongkut's Institute of Technology Ladkrabang



Data Loader

การโหลดข้อมูลรูปภาพและเฉลยเข้าสู่โปรแกรม

```
train = pd.read_csv('C:/train/train.csv')
train.head()

train_img = []
for img_name in tqdm(train['id']):
    # defining the image path
    impath = 'C:/train/'+str(img_name)+'.png'

    # reading the image
    img = imread(impath, as_gray=True)

    # normalizing the pixel values
    img /= 255.0

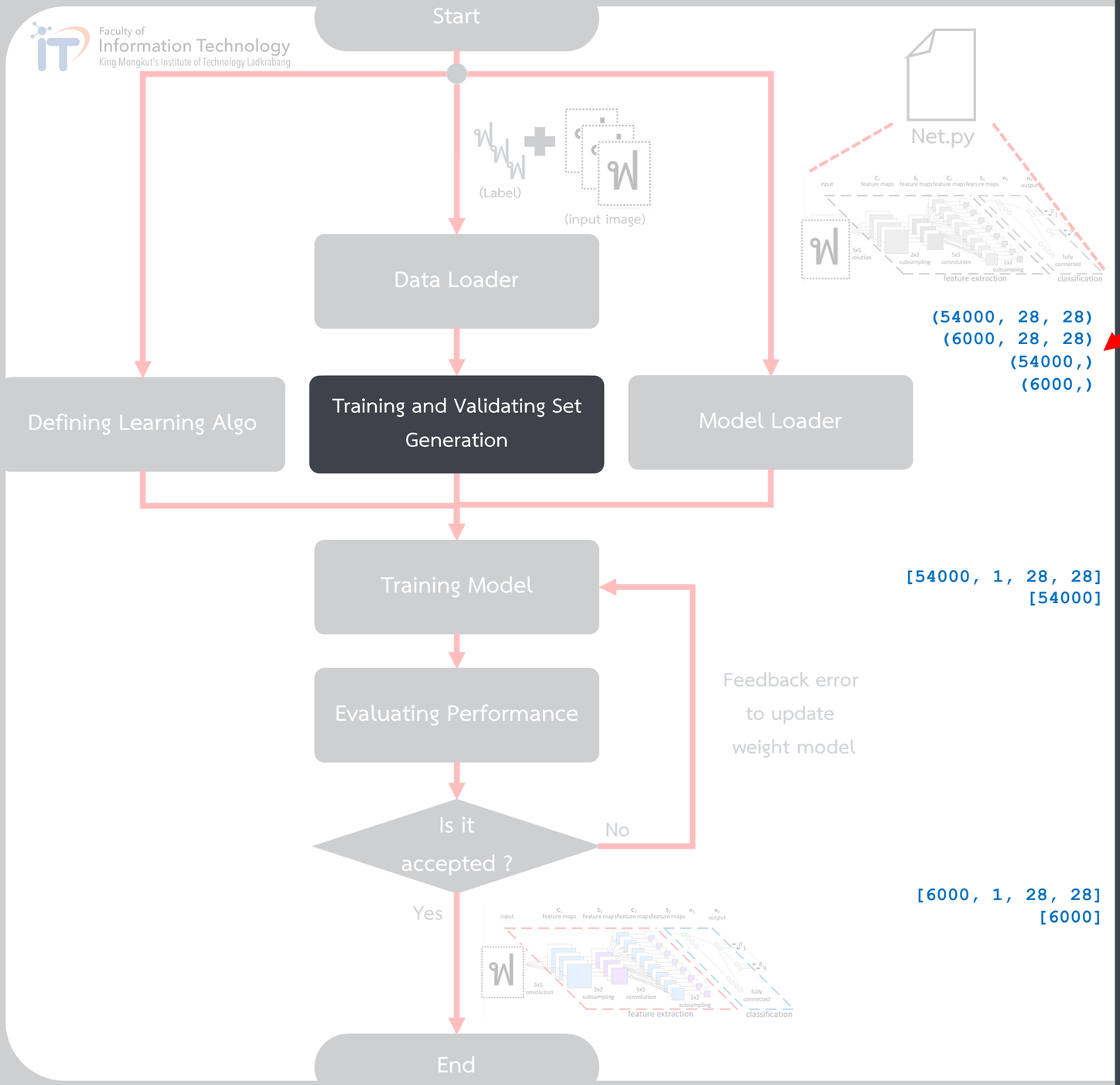
    # converting the type of pixel to float
    img = img.astype('float32')

    # appending the image into the list
    train_img.append(img)

# converting the list to numpy array
train_x = np.array(train_img)

# defining the target
train_y = train['label'].values

print(train_x.shape)
# print >>> (60000, 28, 28)
```



(54000, 28, 28)
(6000, 28, 28)
(54000,)
(6000,)

[54000, 1, 28, 28]
[54000]

[6000, 1, 28, 28]
[6000]



Training & Validating Set Generation

แบ่งข้อมูลออกเป็น 2 ชุด ได้แก่ ชุดฝึกสอนและชุดทดสอบ

```
train_x, val_x, train_y, val_y =
train_test_split(train_x, train_y, test_size = 0.1)

# converting training images into torch format
train_x = train_x.reshape(54000, 1, 28, 28)
train_x  = torch.from_numpy(train_x).to(torch.float32)

# converting the target into torch format
train_y = train_y.astype(int);
train_y = torch.from_numpy(train_y).to(torch.float32)

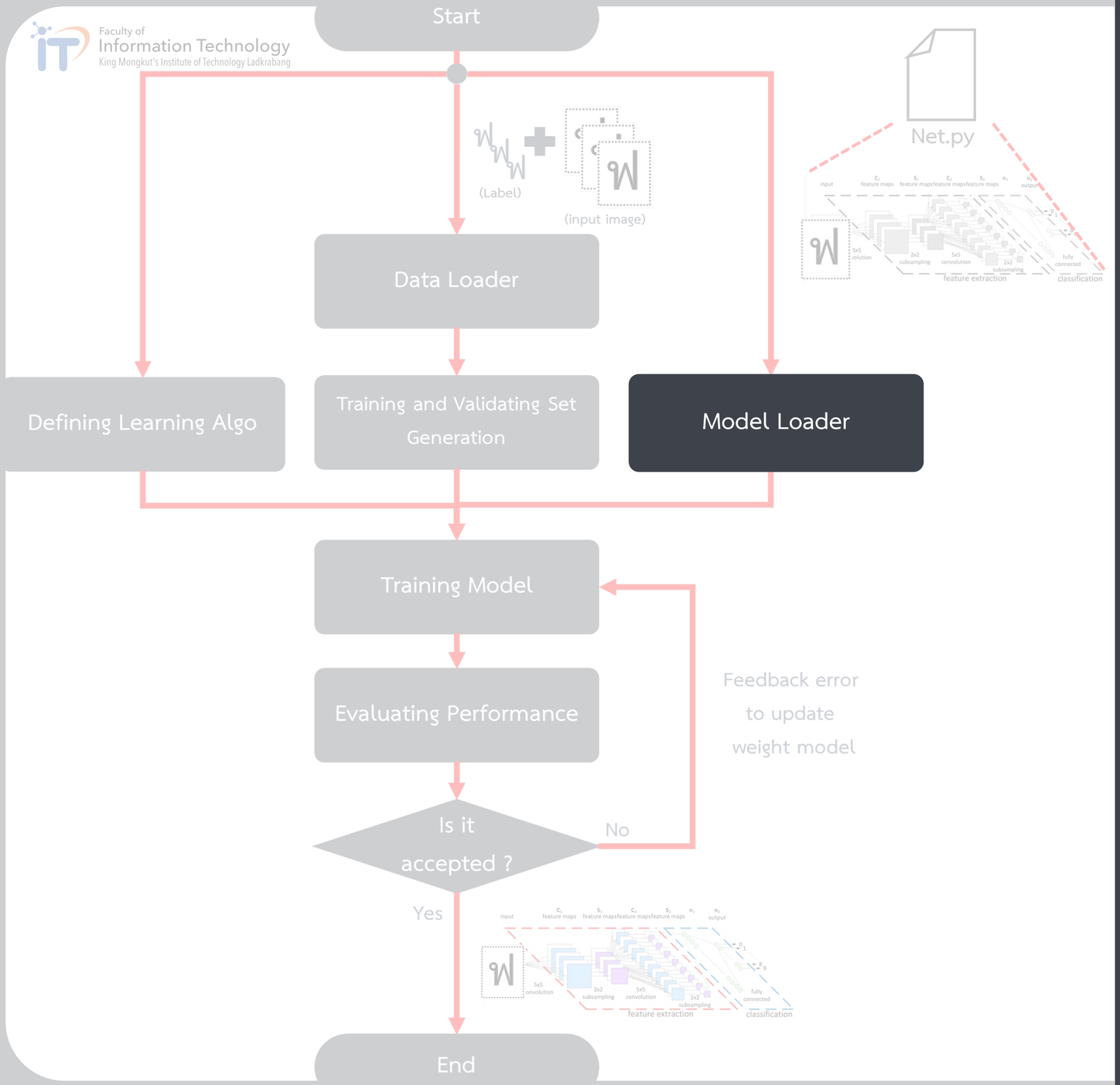
# shape of training data
print(train_x.shape, train_y.shape)

# converting validation images into torch format
val_x = val_x.reshape(6000, 1, 28, 28)
val_x  = torch.from_numpy(val_x).to(torch.float32)

# converting the target into torch format
val_y = val_y.astype(int);
val_y = torch.from_numpy(val_y).to(torch.float32)

# shape of validation data
print(val_x.shape, val_y.shape)
```

Faculty of
Information Technology
King Mongkut's Institute of Technology Ladkrabang

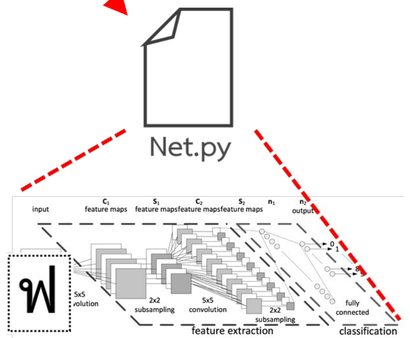


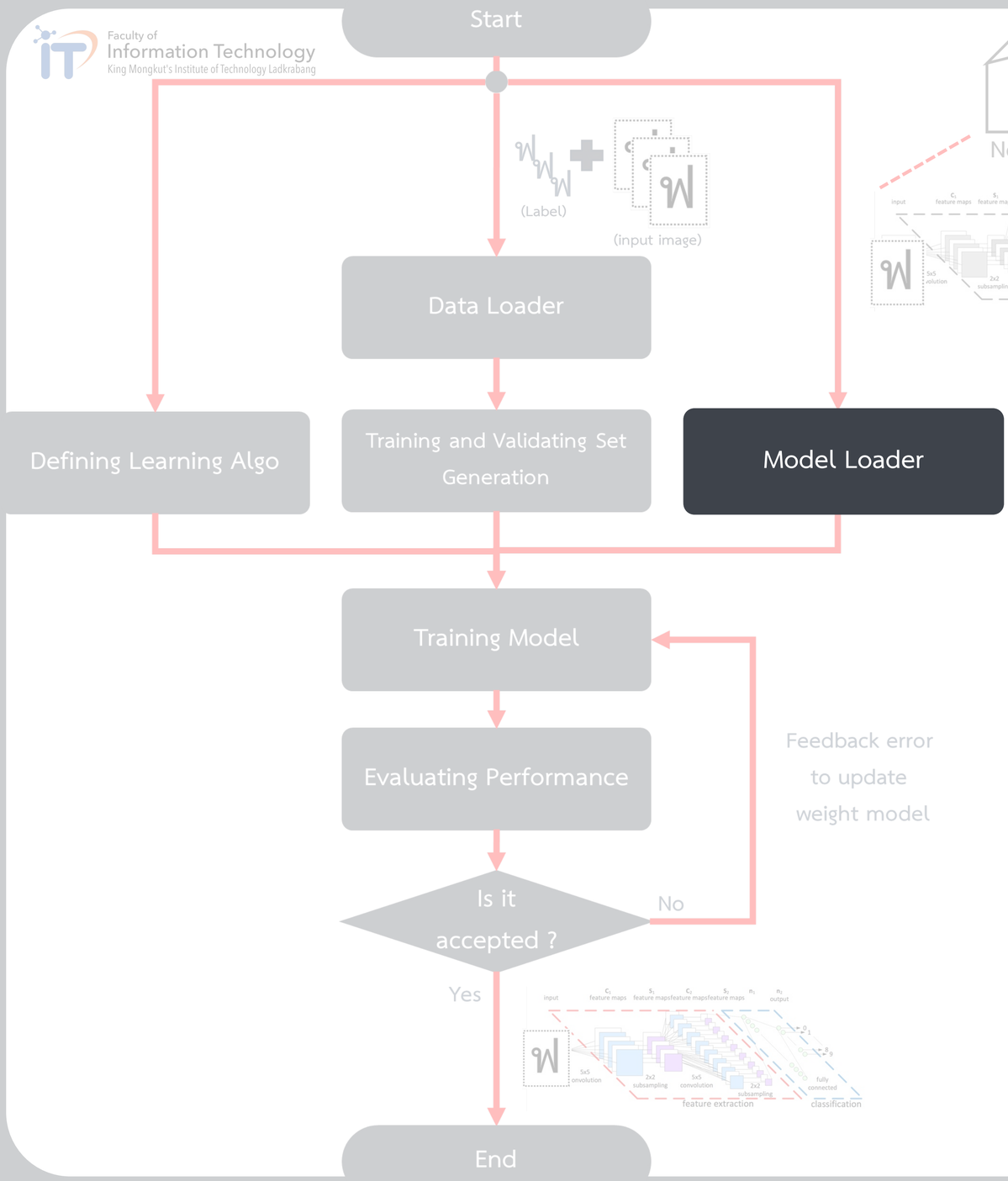


Model Loader

การโหลดโครงสร้างของสถาปัตยกรรมที่จะใช้งานเข้าสู่โปรแกรม

```
from Net import Net #ซึ่งอยู่ในโค้ดของ Import ชุดคำสั่งที่จำเป็น
```



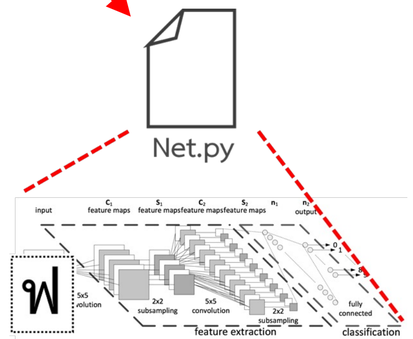




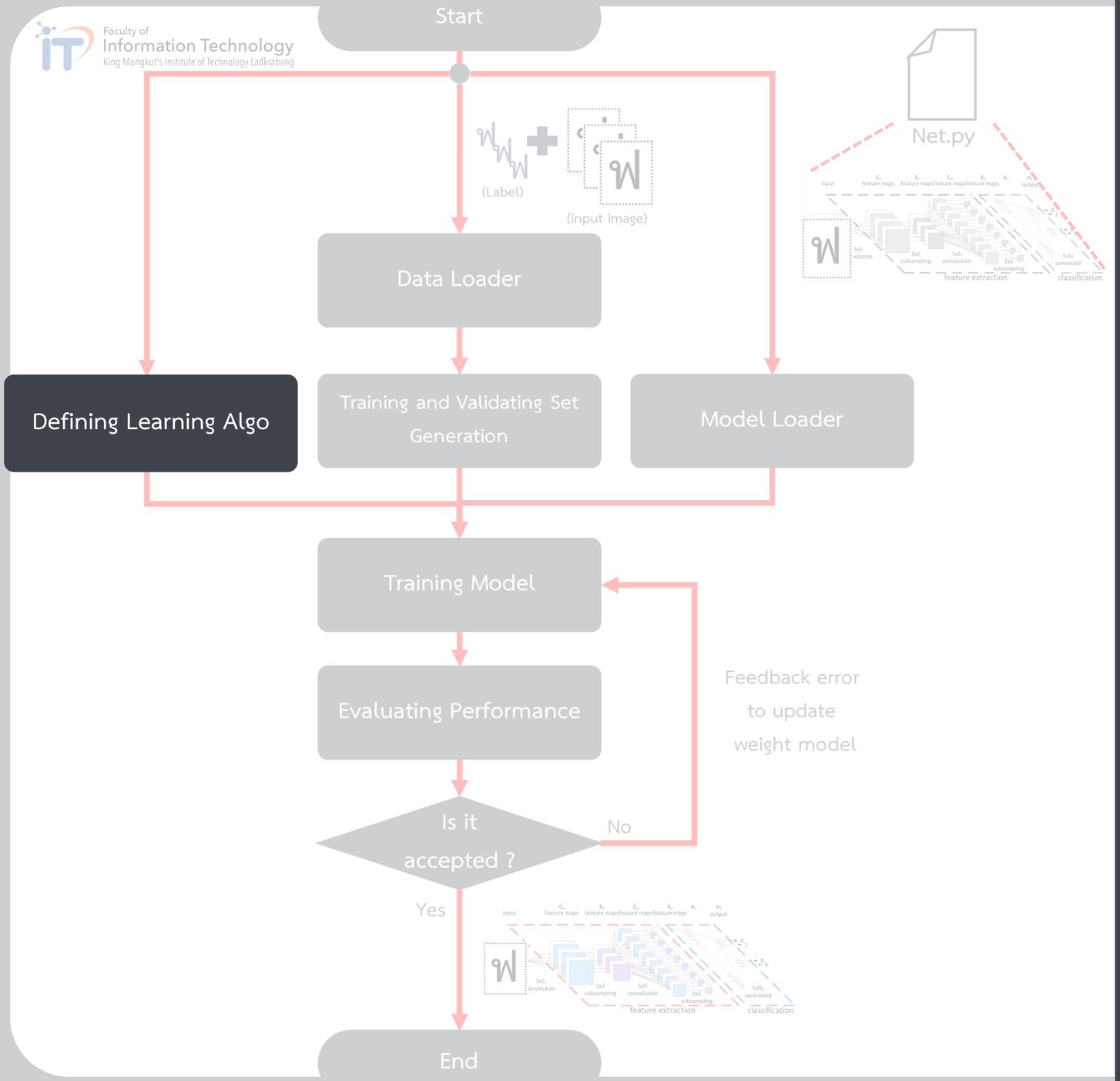
Model Loader

การโหลดโครงสร้างของสถาปัตยกรรมที่จะใช้งานเข้าสู่โปรแกรม

```
from Net import Net #ซึ่งอยู่ในโค้ดของ Import ชุดคำสั่งที่จำเป็น
```



นักศึกษาต้องกำหนดโครงสร้างของ CNN (Bone Model) ของนักศึกษาก่อนว่าจะใช้รูปแบบใด อาทิเช่น AlexNet, ResNet, VGG-19, GoogLeNet, MobiliNet, Inception, หรือ DenseNet นอกจากนี้ นักศึกษายังสามารถกำหนดโครงสร้าง CNN ของนักศึกษาได้เอง ซึ่งนักศึกษาสามารถสร้างและเก็บไว้ในรูปแบบไพธอนคลาสได้



Defining Learning Algorithm

การกำหนดวิธีการเรียนรู้และพารามิเตอร์ต่าง ๆ

```
# defining the model
model = Net()

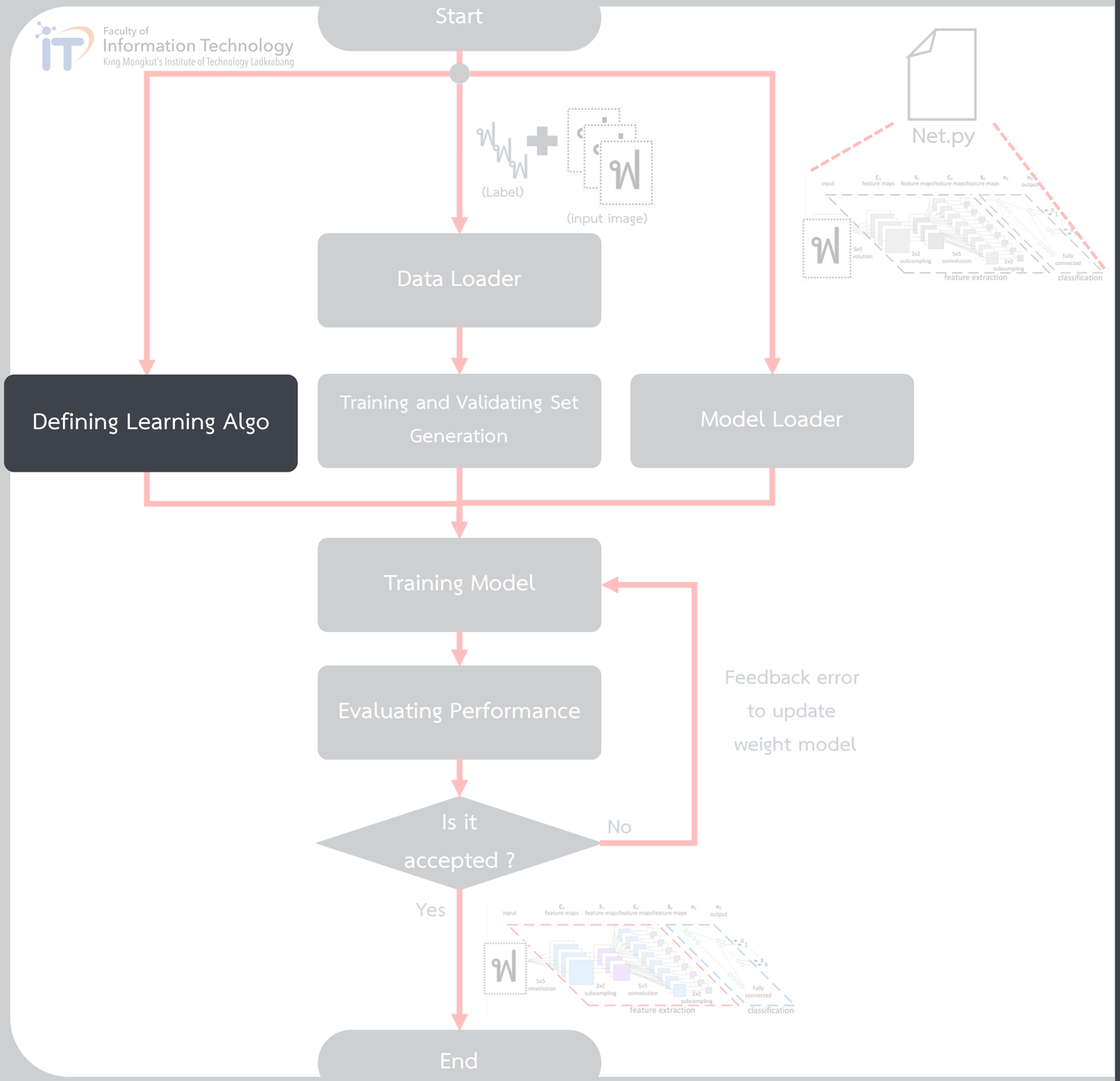
# defining the optimizer
# optimizer = Adam(model.parameters(), lr=0.07)
optimizer = SGD(model.parameters(), lr=0.07, momentum=0.9)

# defining the loss function
criterion = CrossEntropyLoss()

# checking if GPU is available
if torch.cuda.is_available():
    model = model.cuda()
    criterion = criterion.cuda()

print(model)
```

```
Net(
  (cnn_layers): Sequential(
    (0): Conv2d(1, 4, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(4, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace)
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(4, 4, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): BatchNorm2d(4, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU(inplace)
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (linear_layers): Sequential(
    (0): Linear(in_features=196, out_features=10, bias=True)
  )
)
```





Defining Learning Algorithm

การกำหนดวิธีการเรียนรู้และพารามิเตอร์ต่าง ๆ

```
# defining the model
model = Net()

# defining the optimizer
# optimizer = Adam(model.parameters(), lr=0.07)
optimizer = SGD(model.parameters(), lr=0.07, momentum=0.9)

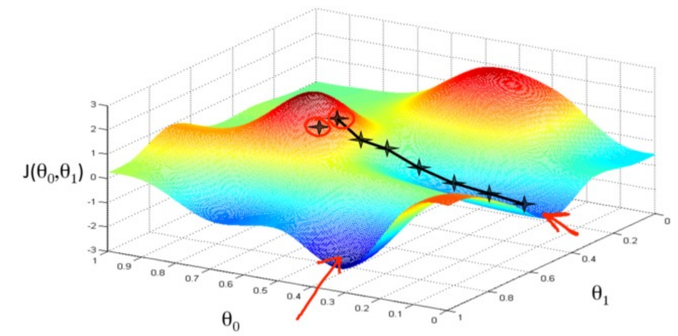
# defining the loss function
criterion = CrossEntropyLoss()

# checking if GPU is available
if torch.cuda.is_available():
    model = model.cuda()
    criterion = criterion.cuda()

print(model)
```

TORCH.OPTIM

คือ แพ็คเกจที่รวบรวมคำสั่งที่เกี่ยวข้องกับอัลกอริทึมปรับปรุงแบบจำลอง (Optimization Algorithm) อยู่



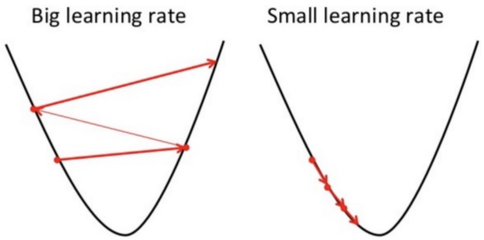
$$\theta^{(\text{next step})} = \theta - \eta \nabla_\theta \text{MSE}(\theta)$$

θ เริ่มต้นเป็นค่า random

learning rate
จะก้าวใหญ่แค่ไหน

gradient ของ MSE(θ) w.r.t. θ
เป็นตัวบอกทิศทางลงภูเขา

$$\nabla_\theta \text{MSE}(\theta) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \text{MSE}(\theta) \\ \frac{\partial}{\partial \theta_1} \text{MSE}(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{MSE}(\theta) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T \cdot (\mathbf{X} \cdot \theta - \mathbf{y})$$



อ้างอิง https://medium.com/mmp-li/gradient-descent-คีย์เวิร์ดที่-data-science-ต้องรู้จัก-machine-learning-101-b5c78c1787a0

วิธีการใช้งาน optimizer จากแพ็คเกจ TORCH.OPTIM

- สร้าง optimizer object สำหรับเก็บข้อมูลสถานะและค่าพารามิเตอร์ต่าง ๆ ของแบบจำลอง
- กำหนดค่าพารามิเตอร์ต่าง ๆ ที่จำเป็น อาทิเช่น learning rate, optimizer-specific เป็นต้น
- อัพเดตค่าต่าง ๆ ของแบบจำลองด้วย gradients

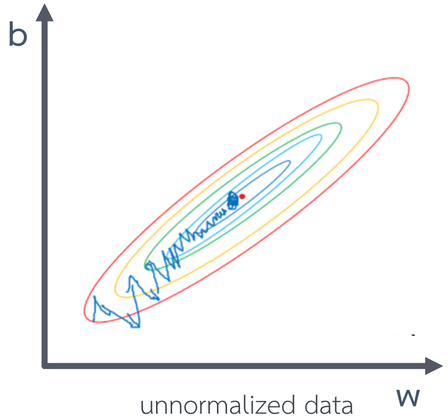
ตัวอย่างเช่น

```
# Stochastic Gradient Descent Algorithm
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

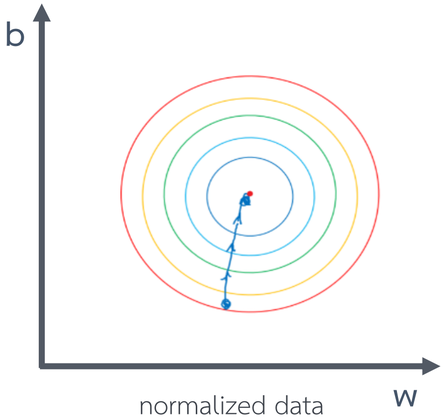
# Adam algorithm.
optimizer = optim.Adam([var1, var2], lr=0.0001)
```

Normalization

คือ การปรับช่วงของข้อมูลให้อยู่ในช่วงที่กำหนด



unnormalized data



normalized data

ข้อควรระวัง

นักศึกษาควรต้องตรวจสอบช่วงของค่าข้อมูลก่อนว่ามีความแตกต่างกันมากหรือไม่ เนื่องจากอาจส่งผลให้ใช้เวลาในการประมวลผลที่นานกว่าจะเข้าใกล้ค่าต่ำสุดสำหรับ Gradient Descent ตัวอย่างเช่น แอททริบิว A อยู่ในช่วง 0 - 1 ขณะที่แอททริบิว B อยู่ในช่วง 0 - 255 ซึ่งจะพบว่าทั้ง 2 แอททริบิวมีช่วงที่แตกต่างกันอย่างชัดเจน เพื่อที่จะหลีกเลี่ยงปัญหาข้างต้น ดังนั้น การปรับรูปแบบบรรทัดฐาน (Normalization) จึงถูกนำมาใช้งาน โดยอาศัยหลักการทางสถิติดังแสดงในสมการต่อไปนี้

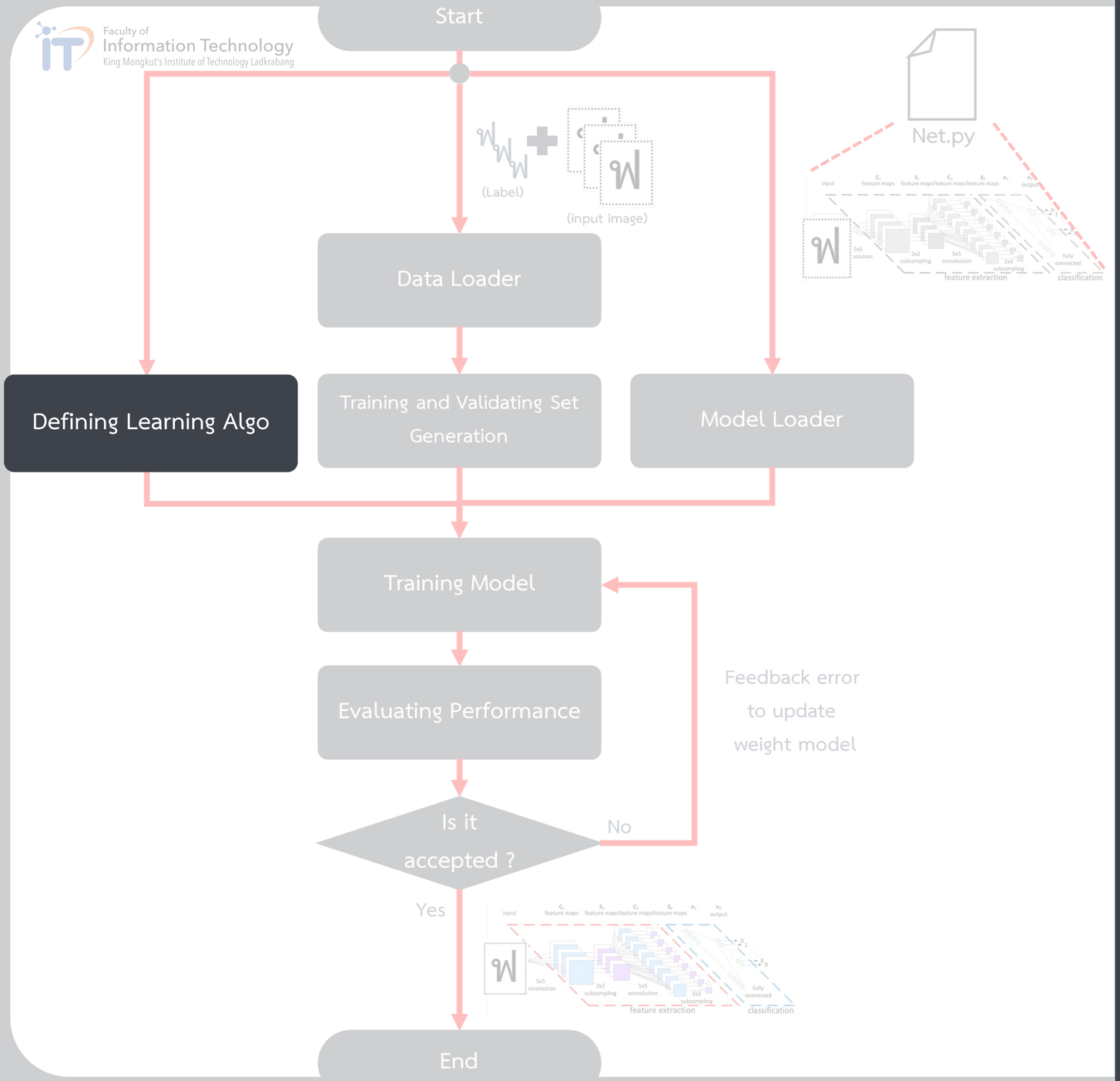
mean

$$x'_i = \frac{x_i - \mu}{\sigma}$$

S.D.

หลังจากการปรับรูปแบบบรรทัดฐานแล้ว x' ควรปรับให้ค่าเฉลี่ย (μ) เท่ากับ 0 และส่วนเบี่ยงเบนมาตรฐาน (σ) เท่ากับ 1

อ้างอิง https://medium.com/mmp-li/gradient-descent-คีย์เวิร์ดที่-data-science-ต้องรู้จัก-machine-learning-101-b5c78c1787a0 และ https://towardsdatascience.com/gradient-descent-algorithm-and-its-variants-10f652806a3





Defining Learning Algorithm

การกำหนดวิธีการเรียนรู้และพารามิเตอร์ต่าง ๆ

```
# defining the model
model = Net()

# defining the optimizer
# optimizer = Adam(model.parameters(), lr=0.07)
optimizer = SGD(model.parameters(), lr=0.07, momentum=0.9)

# defining the loss function
criterion = CrossEntropyLoss()

# checking if GPU is available
if torch.cuda.is_available():
    model = model.cuda()
    criterion = criterion.cuda()

print(model)
```

Loss Functions (Cost Function) for Classification Problem

คือ สมการผลรวมของค่าความแตกต่าง (Difference) หรือค่าความผิดพลาด (Error) ระหว่างค่าที่ได้จากแบบจำลองกับค่าจากข้อมูลจริง ซึ่งนิยมนำมาใช้เพื่อวัดประสิทธิภาพของแบบจำลองว่ามีความเหมาะสมกับข้อมูลหรือไม่ โดยทั่วไปแล้วค่าของสมการยิ่งน้อยยิ่งดี

Loss Function, Cost Function หรือ Error Function

คือ สมการทางคณิตศาสตร์ที่คำนวณค่า Event จาก Input หลาย ๆ ตัว ออกมาเป็นตัวเลขจำนวนจริง 1 ตัว เพื่อใช้แทน Cost ของ Event นั้น

Optimization

คือ การหาวิธีทำให้ค่า Output ของ Loss Function ออกมาน้อยที่สุด (Minimize)



KENG SURAPONG

PROJECT MANAGER

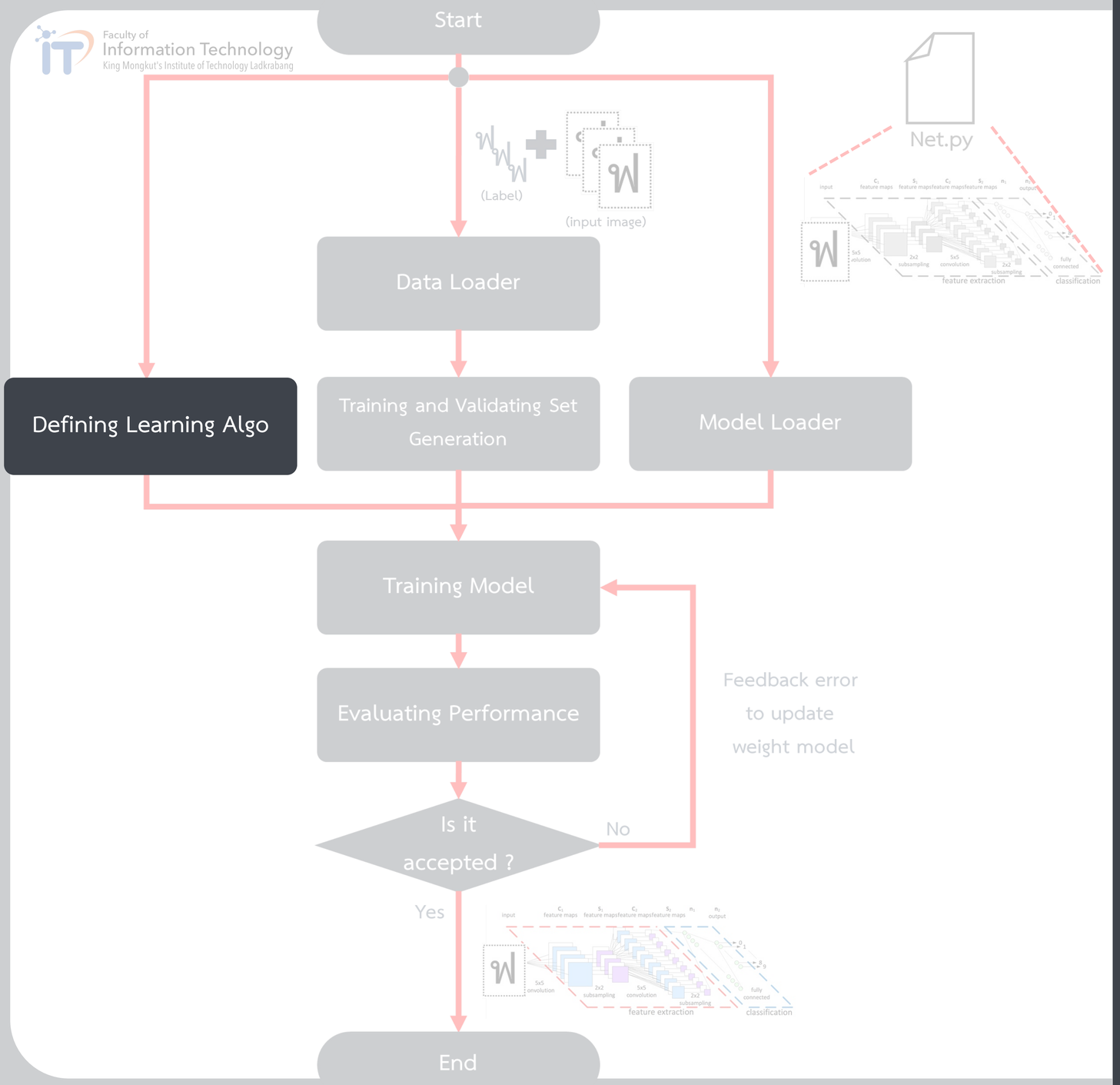
@BUA LABS

Loss Functions จะสะท้อนถึงประสิทธิภาพของแบบจำลอง หรือกล่าวได้ว่า “*พารามิเตอร์แต่ละตัวของแบบจำลองมีความเหมาะสมกับข้อมูลหรือไม่*” ซึ่ง Loss Functions จะรับค่า Probability จาก Softmax มาคำนวณเพื่อดูว่า (1) ***ผลทำนาย***ถูกต้องหรือไม่ และ (2) ***ค่าความมั่นใจ***ของแบบจำลองเป็นอย่างไร อาทิเช่น

- **กรณีที่ 1 :** *ทำนายถูก และมีความมั่นใจมาก ซึ่งจะสามารถเรียกได้ว่า ดีและจะให้ค่า Loss ที่น้อย*
- **กรณีที่ 2 :** *ทำนายถูก และมีความมั่นใจน้อย ซึ่งจะสามารถเรียกได้ว่า ไม่ดีและจะให้ค่า Loss ที่มาก*
- **กรณีที่ 3 :** *ทำนายไม่ถูก และมีความมั่นใจมาก ซึ่งจะสามารถเรียกได้ว่า ไม่ดีและจะให้ค่า Loss ที่มาก*

ซึ่งในงานประเภท Classification Problem จะมี Loss Functions ที่น่าสนใจอยู่ด้วยกัน 2 สมการ ได้แก่ (1) Hinge Loss และ (2) Cross Entropy Loss

อ้างอิง https://pytorch.org/docs/stable/nn.html และ https://towardsdatascience.com/common-loss-functions-in-machine-learning-46af0ffc4d23





Defining Learning Algorithm

การกำหนดวิธีการเรียนรู้และพารามิเตอร์ต่าง ๆ

```
# defining the model
model = Net()

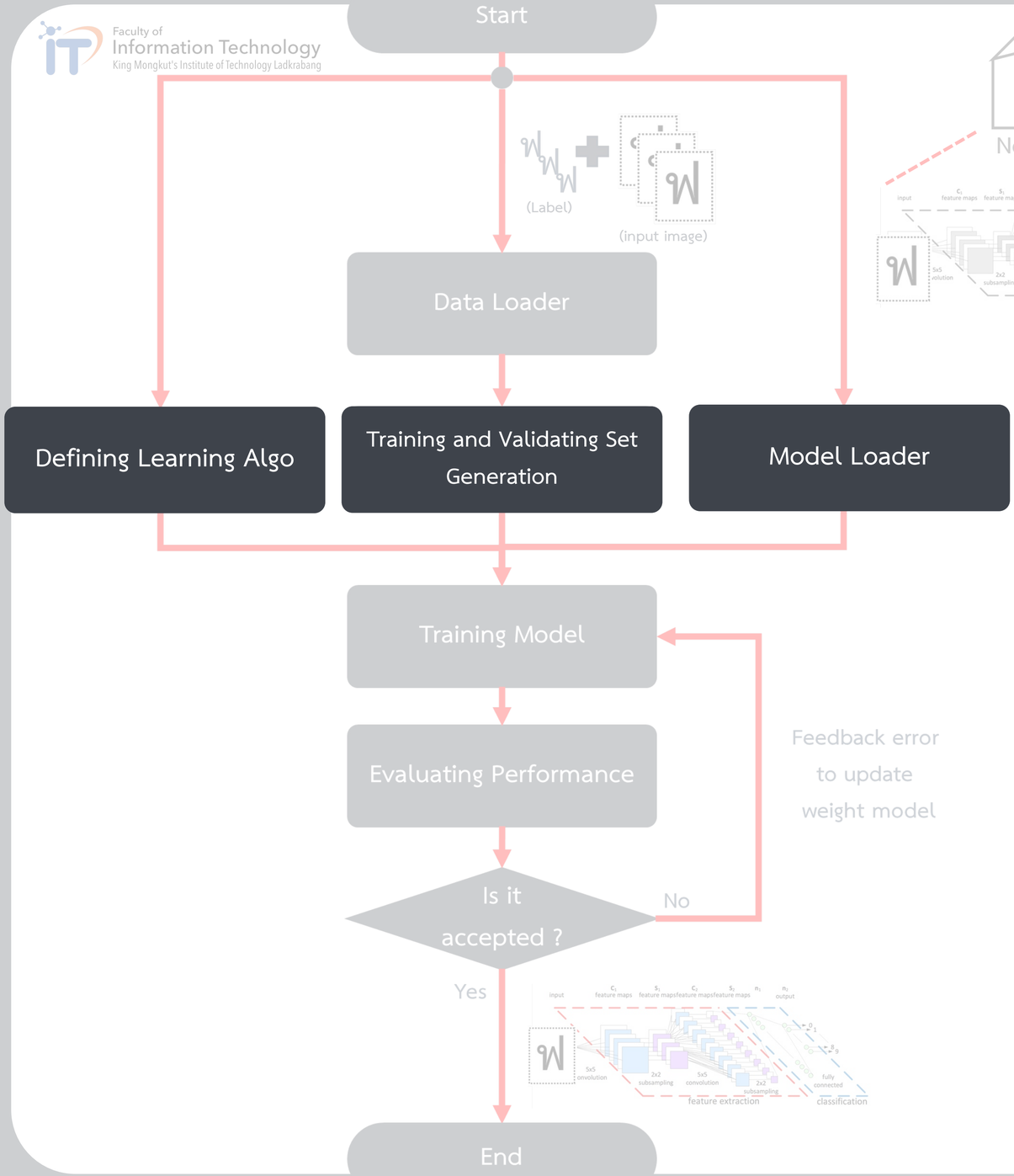
# defining the optimizer
# optimizer = Adam(model.parameters(), lr=0.07)
optimizer = SGD(model.parameters(), lr=0.07, momentum=0.9)

# defining the loss function
criterion = CrossEntropyLoss()

# checking if GPU is available
if torch.cuda.is_available():
    model = model.cuda()
    criterion = criterion.cuda()

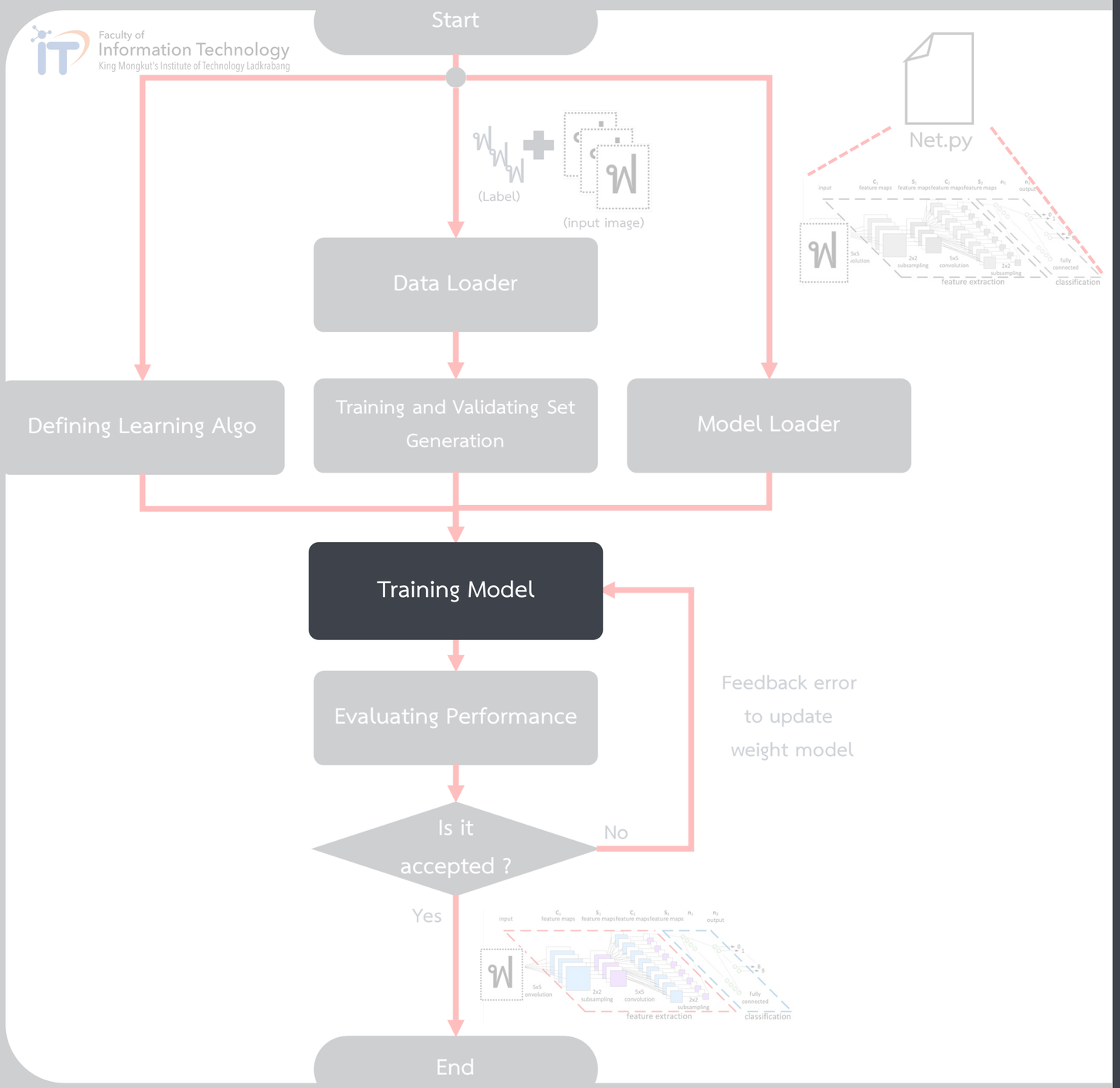
print(model)
```

Faculty of
Information Technology
King Mongkut's Institute of Technology Ladkrabang



Defining Learning Algorithm

การกำหนดวิธีการเรียนรู้และพารามิเตอร์ต่าง ๆ





Training Model

ฝึกสอนแบบจำลอง

```
# empty list to store training losses
train_losses = []

# empty list to store validation losses
val_losses = []

# defining the number of epochs
n_epochs = 25

# training the model
for epoch in tqdm(range(n_epochs)):

    model.train()
    tr_loss = 0

    # getting the training set
    x_train, y_train = Variable(train_x), Variable(train_y)

    # getting the validation set
    x_val, y_val = Variable(val_x), Variable(val_y)

    # clearing the Gradients of the model parameters
    optimizer.zero_grad()

    # prediction for training and validation set
    output_train = model(x_train)
    output_val   = model(x_val)
```

ฟังก์ชัน model.train() ใน PyTorch

model.train() ใน PyTorch เป็นคำสั่งที่ใช้เพื่อกำหนดว่าโมเดลกำลังอยู่ในโหมด "training" (การฝึกฝน) แทนที่จะเป็นโหมด "evaluation" (การทดสอบ) การสลับโหมดระหว่างการฝึกฝนและการทดสอบเป็นสิ่งสำคัญ เนื่องจากพฤติกรรมของบางเลเยอร์ในโมเดล เช่น Dropout หรือ Batch Normalization จะเปลี่ยนไปตามโหมดที่โมเดลอยู่

- **Dropout**

ในโหมดการฝึก (train()), Dropout จะเปิดใช้งาน โดยจะสุ่มปิดบางหน่วย (neurons) ระหว่างการฝึก เพื่อช่วยป้องกันการ overfitting แต่ในโหมดการทดสอบ (eval()), Dropout จะถูกปิด (หรือถูกข้ามไป) และไม่มีการปิดหน่วยใดๆ

- **Batch Normalization**

ในโหมดการฝึก (train()), Batch Normalization จะคำนวณค่า mean และ variance จาก mini-batch ในแต่ละครั้งที่ทำ forward pass แต่ในโหมดการทดสอบ (eval()), Batch Normalization จะใช้ค่า mean และ variance ที่คำนวณไว้จากการฝึกเพื่อให้โมเดลทำงานคงที่ในการทดสอบ

อย่างไรก็ตาม คำสั่ง model.train() ไม่ได้ใช้สำหรับการฝึกโมเดลโดยตรง แต่มันเป็นการบอก PyTorch ว่าโมเดลอยู่ในโหมดการฝึก และเมื่อต้องการทดสอบโมเดลหลังจากการฝึก เราควรสลับเป็นโหมด eval() โดยใช้ model.eval() เพื่อให้แน่ใจว่าเลเยอร์ต่าง ๆ เช่น Dropout และ Batch Normalization ทำงานในโหมดการทดสอบอย่างถูกต้อง นอกจากนี้ หากไม่กำหนดค่า model.train() ใน PyTorch ค่าเริ่มต้น (default) ของโมเดลจะอยู่ใน โหมดการฝึก (training mode) โดยอัตโนมัติ ซึ่งหมายความว่า

ฟังก์ชัน Variable(...)ใน PyTorch

Variable(...) เป็นคำสั่งที่ใช้ใน PyTorch เวอร์ชันเก่า (ก่อนเวอร์ชัน 0.4) เพื่อห่อหุ้ม Tensor ให้กลายเป็น Variable ซึ่งเป็นคลาสที่ใช้ในการเก็บข้อมูล (Tensor) และคำนวณค่าการไล่ระดับ (gradients) ระหว่างการทำ backpropagation

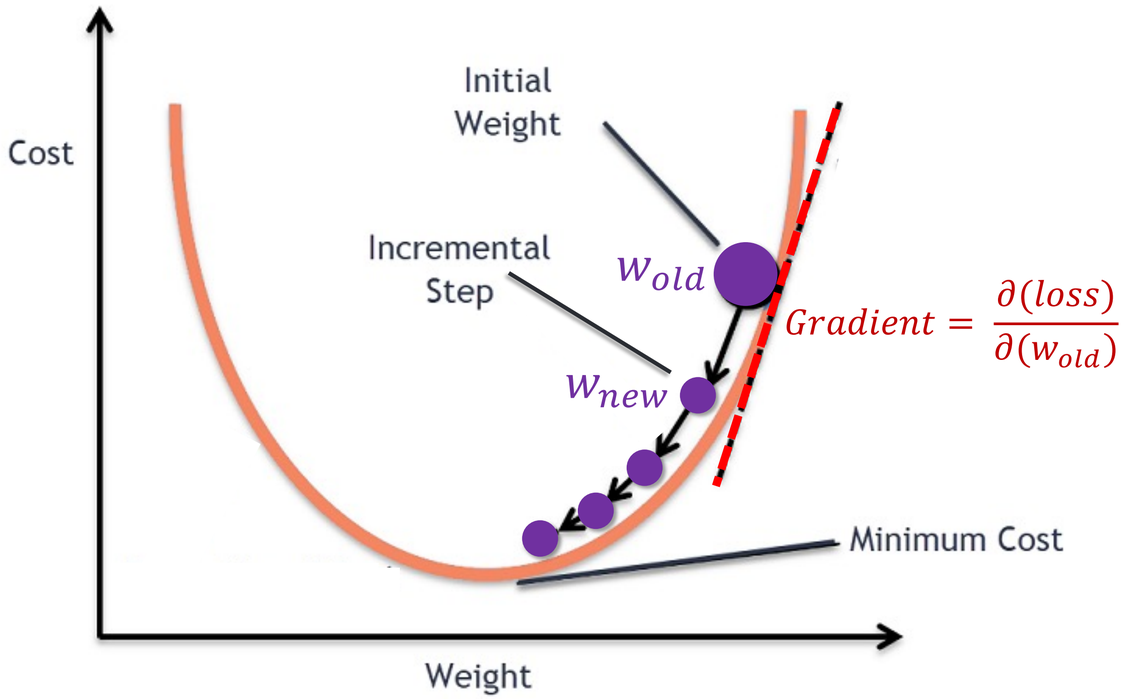
- **ใน PyTorch เวอร์ชันเก่า (ก่อนเวอร์ชัน 0.4)**

Variable ใน PyTorch ใช้เพื่อเก็บข้อมูล (Tensor) และทำให้สามารถคำนวณค่าการไล่ระดับของค่าใน Tensor นั้นได้ ตัวแปร Tensor ที่นำมาใส่ใน Variable เพื่อให้มันสามารถเก็บค่าและทำงานร่วมกับกราฟการคำนวณเชิงสัญลักษณ์ (computational graph) ของ PyTorch ฟังก์ชันนี้จะถูกใช้เมื่อเราต้องการให้ PyTorch คำนวณ gradient โดยอัตโนมัติเมื่อทำการฝึกฝนโมเดล

- **ใน PyTorch เวอร์ชันใหม่ (หลังเวอร์ชัน 0.4)**

ไม่จำเป็นต้องใช้ Variable อีกต่อไป เนื่องจาก PyTorch ได้รวมฟังก์ชันของ Variable เข้ากับ Tensor โดยตรง ดังนั้น เราสามารถทำงานกับ Tensor ได้เหมือนเดิมโดยไม่ต้องใช้ Variable

ค่าการไล่ระดับ (Gradients)



อ้างอิง https://www.kaggle.com/code/aryaupadhyay95/gradient-descent

Gradient ในบริบทของการเรียนรู้เชิงลึก (Deep Learning) หมายถึง เวกเตอร์ของอนุพันธ์ย่อย (Partial Derivatives) ของฟังก์ชันเป้าหมาย (เช่น ฟังก์ชันความสูญเสีย) ซึ่งแสดงทิศทางและอัตราการเปลี่ยนแปลงของฟังก์ชันนั้นเมื่อเทียบกับพารามิเตอร์ต่าง ๆ ของโมเดล

ในทางคณิตศาสตร์ Gradient ของฟังก์ชันเป็นการวัด อัตราการเปลี่ยนแปลงของฟังก์ชันเมื่อเปลี่ยนแปลงพารามิเตอร์ ซึ่ง Gradient จะชี้ไปในทิศทางของการเพิ่มขึ้นที่เร็วที่สุดของฟังก์ชัน ดังนั้น ถ้าต้องการลดค่าของฟังก์ชัน จึงจำเป็นต้องปรับพารามิเตอร์ไปในทิศทางตรงกันข้ามของ Gradient ในอัลกอริทึม Gradient Descent ซึ่งเป็นหนึ่งในวิธีที่ใช้ปรับพารามิเตอร์ของโมเดล จะคำนวณ Gradient ของฟังก์ชันความสูญเสีย (Loss Function) เพื่อบอกว่าควรปรับค่าพารามิเตอร์ไปในทิศทางใดเพื่อลดค่าฟังก์ชันความสูญเสีย

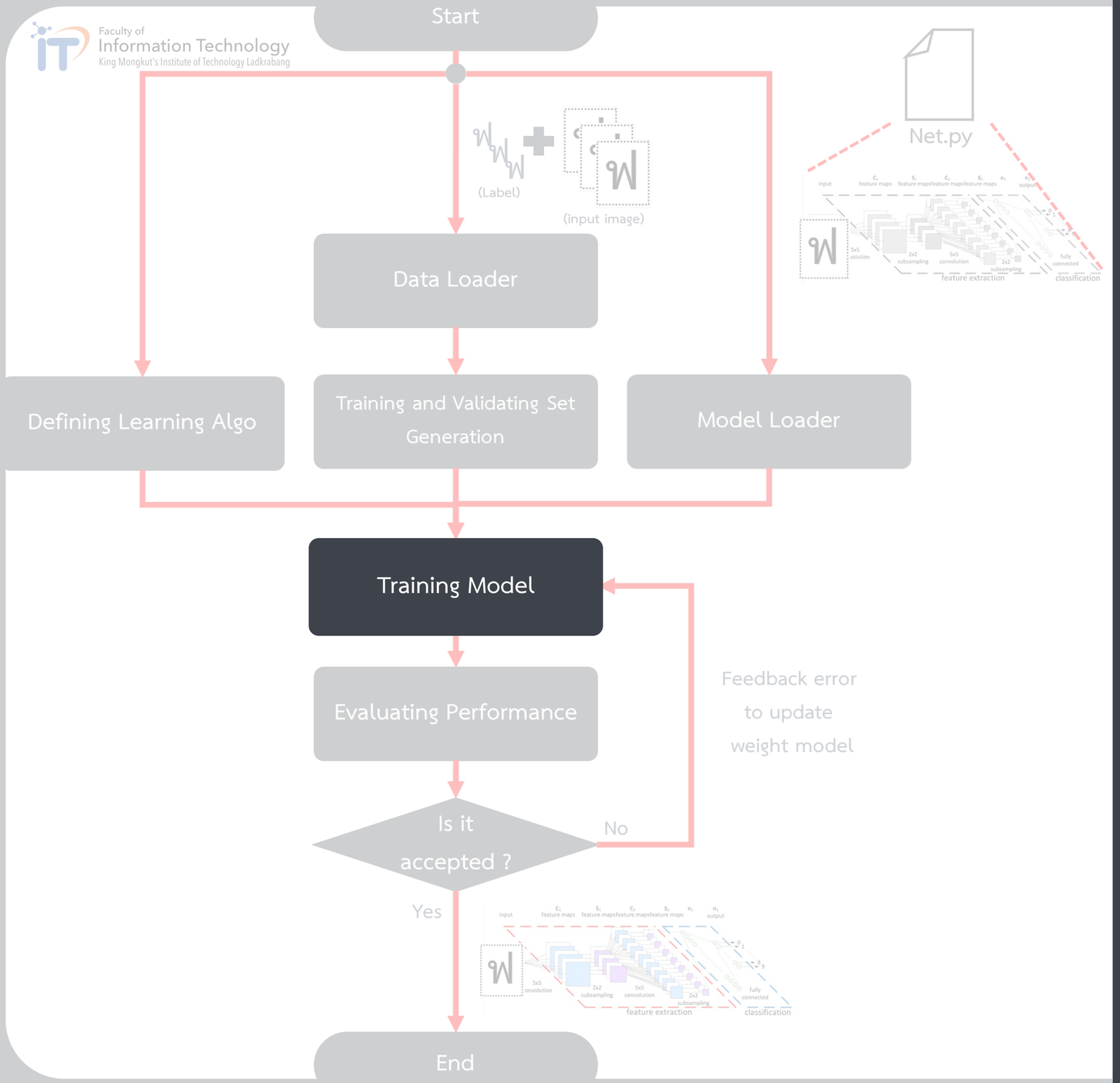
$$w_{new} = w_{old} - \alpha \frac{\partial(loss)}{\partial(w_{old})}$$

ฟังก์ชัน optimizer.zero_grad() ใน PyTorch

optimizer.zero_grad() ใน PyTorch เป็นคำสั่งที่ใช้ รีเซ็ต (reset) ค่า gradients ของพารามิเตอร์ทั้งหมดที่ถูกจัดการโดย optimizer ให้เป็นศูนย์ก่อนที่จะทำการคำนวณ gradients ใหม่ในการฝึกแต่ละรอบ (forward และ backward pass)

สาเหตุที่ต้องใช้ optimizer.zero_grad()

ใน PyTorch ค่า gradients ที่คำนวณได้จากการทำ backpropagation (ผ่าน loss.backward()) จะถูกสะสม (accumulated) ในพารามิเตอร์ต่างๆ ของโมเดล หากไม่ได้รีเซ็ตค่า gradients เป็นศูนย์ในแต่ละรอบ ค่า gradients จากรอบก่อนหน้าจะถูกบวกเข้าไปกับค่า gradients ของรอบปัจจุบัน ทำให้ค่าที่ได้ไม่ถูกต้อง ดังนั้น การใช้ optimizer.zero_grad() ก่อนการคำนวณ backpropagation ในแต่ละรอบการฝึกฝนจึงเป็นสิ่งจำเป็นเพื่อให้แน่ใจว่าค่า gradients ใหม่จะถูกคำนวณจากข้อมูลใหม่เท่านั้น





Training Model

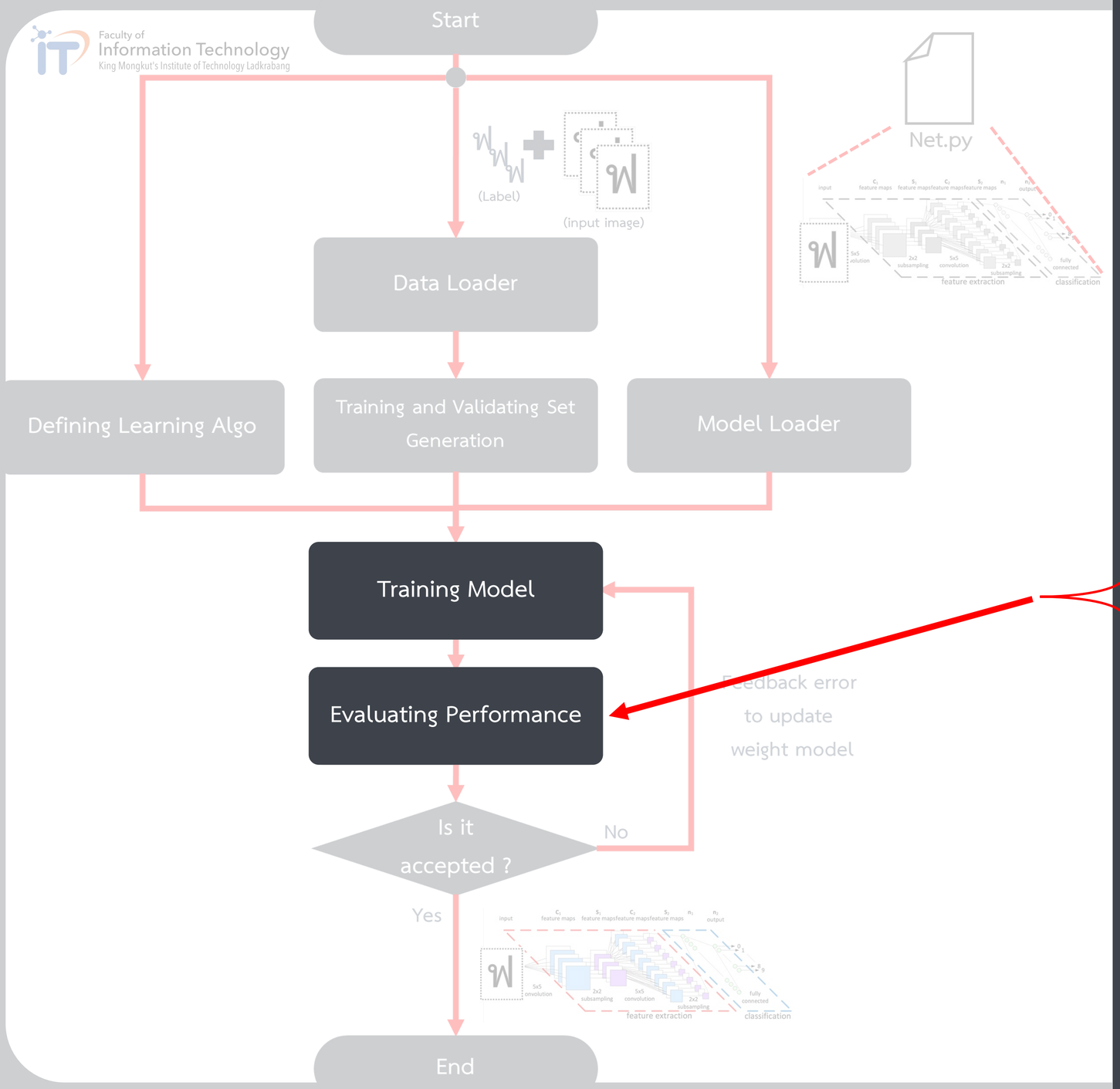
ฝึกสอนแบบจำลอง

```
# computing the training and validation loss
# we convert the results because they aren't in
the good format

y_train = y_train.long()
y_train = y_train.squeeze_()
y_val = y_val.long()
y_val = y_val.squeeze_()

loss_train = criterion(output_train, y_train)
loss_val = criterion(output_val, y_val)
train_losses.append(loss_train)
val_losses.append(loss_val)

# computing the updated weights of all the model parameters
loss_train.backward()
optimizer.step()
tr_loss = loss_train.item()
```





Training Model

ฝึกสอนแบบจำลอง

```
# computing the training and validation loss
# we convert the results because they aren't in
the good format

y_train = y_train.long()
y_train = y_train.squeeze_()
y_val = y_val.long()
y_val = y_val.squeeze_()

loss_train = criterion(output_train, y_train)
loss_val = criterion(output_val, y_val)
train_losses.append(loss_train)
val_losses.append(loss_val)

# computing the updated weights of all the model parameters
loss_train.backward()
optimizer.step()
tr_loss = loss_train.item()
```

ฟังก์ชัน squeeze() ใน PyTorch

python Copy code

```python
import torch
y_train = torch.zeros(2, 1, 2, 1, 2)  # สร้าง tensor ขนาด [2, 1, 2, 1, 2]
print(y_train.shape)          # Output: torch.Size([2, 1, 2, 1, 2])

a = y_train.squeeze()         # ลบมิติที่มีขนาดเป็น 1
print(a.shape)                # Output: torch.Size([2, 2, 2])
```

ฟังก์ชัน squeeze() ใน PyTorch ใช้เพื่อลดขนาดของ Tensor โดยลบมิติที่มีขนาดเท่ากับ 1 ออก เช่น หาก Tensor มีขนาดเป็น (10, 1, 28, 28) และใช้ squeeze() จะได้ Tensor ที่มีขนาดเป็น (10, 28, 28) โดยมิติที่ขนาดเท่ากับ 1 จะถูกลบออก

python Copy code

```python
import torch
y_train = torch.zeros(2, 1, 2, 1, 2)  # สร้าง tensor ขนาด [2, 1, 2, 1, 2]
print(y_train.shape)          # Output: torch.Size([2, 1, 2, 1, 2])

y_train.squeeze_()            # ลบมิติที่มีขนาดเป็น 1
print(y_train.shape)          # Output: torch.Size([2, 2, 2])
```

นอกจากนี้ เครื่องหมายขีดล่าง _ ใน squeeze_() บ่งบอกว่าเป็น in-place operation นั่นหมายความว่าการเปลี่ยนแปลงจะเกิดขึ้นกับตัวแปรนั้นๆ โดยตรง ไม่ได้สร้างตัวแปรใหม่

ฟังก์ชัน self.long() และ self.to(torch.int64)

```python
import torch
x = torch.tensor([1.2, 3.4, 5.6])
y = x.long()  # แปลงเป็น torch.int64
print(y.dtype)  # Output: torch.int64
```

```python
import torch
x = torch.tensor([1.2, 3.4, 5.6])
y = x.to(torch.int64)  # แปลงเป็น torch.int64
print(y.dtype)  # Output: torch.int64
```

ใน PyTorch, self.long() และ self.to(torch.int64) ทำงานคล้ายกันคือการเปลี่ยนประเภทของข้อมูล (data type) ของ Tensor ให้เป็น 64-bit integer (torch.int64) หรือที่เรียกว่า long tensor ใน PyTorch แต่ทั้งสองวิธีมีความแตกต่างกันเล็กน้อยในด้านการใช้งานและความยืดหยุ่น

- **self.long()**

ฟังก์ชัน long() เป็นวิธีที่ง่ายและตรงไปตรงมาในการแปลง Tensor ให้เป็นประเภท long หรือ torch.int64 ใช้เมื่อเรารู้อยู่แล้วว่าเราต้องการแปลง Tensor เป็น torch.int64

- **self.to(torch.int64)**

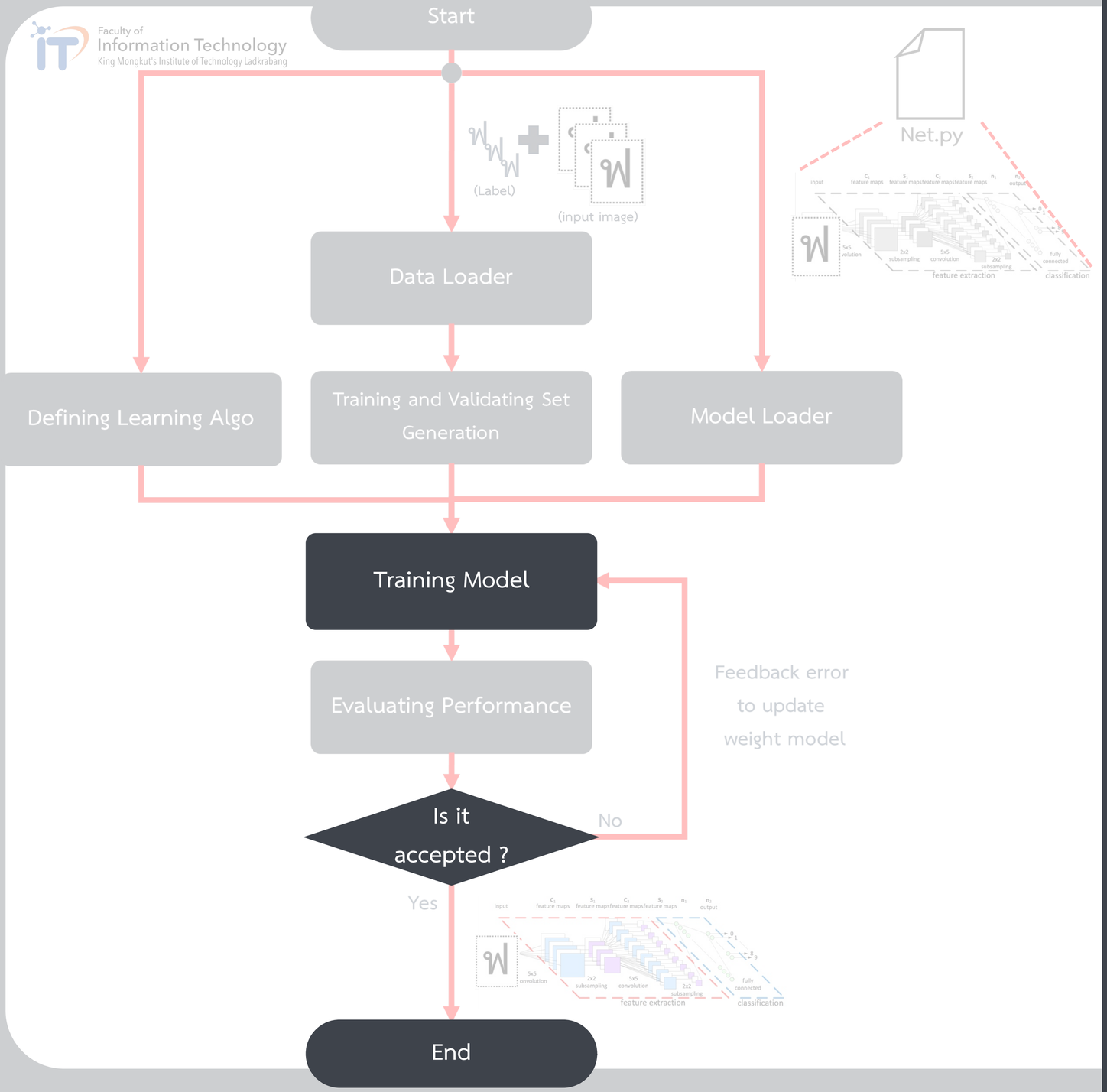
ฟังก์ชัน to() เป็นวิธีที่ยืดหยุ่นกว่า เพราะสามารถแปลง Tensor เป็นชนิดข้อมูลอื่นๆ ได้ อาทิเช่น torch.float32, torch.int32, torch.cuda, ฯลฯ ไม่ใช่แค่ torch.int64 เพียงอย่างเดียว เหมาะสมเมื่อคุณต้องการความยืดหยุ่นในการระบุชนิดของข้อมูล (dtype) ที่ชัดเจน

ฟังก์ชัน criterion(...), backward(...) และ step(...)

- คำสั่ง **loss_train = criterion(output_train, y_train)** ทำหน้าที่คำนวณ ค่าความสูญเสีย (loss) ระหว่างค่าผลลัพธ์ที่โมเดลทำนาย (output_train) กับค่าจริง (y_train) โดยใช้ฟังก์ชันความสูญเสียที่กำหนดโดย criterion ซึ่งเป็นขั้นตอนสำคัญในการฝึกโมเดล Machine Learning เพราะค่าความสูญเสียนี้จะถูกใช้ในการคำนวณ gradient เพื่อนำไปปรับปรุงพารามิเตอร์ของโมเดล
- คำสั่ง **loss_train.backward()** ใช้เพื่อคำนวณ gradient ของค่าความสูญเสีย (loss) ย้อนกลับไปยังพารามิเตอร์ของโมเดล ผ่านการทำงานของ backpropagation กระบวนการนี้สำคัญสำหรับการฝึกโมเดลเชิงลึก (deep learning) เพราะ gradients เหล่านี้จะถูกใช้ในการอัปเดตพารามิเตอร์ของโมเดล (เช่น weights และ biases) ด้วยการปรับค่าเพื่อให้โมเดลเรียนรู้จากข้อมูล
- คำสั่ง **optimizer.step()** ใช้สำหรับ อัปเดตพารามิเตอร์ของโมเดล โดยใช้ค่า gradients ที่ถูกคำนวณไว้ด้วย loss.backward()

กระบวนการทำงานระห่าง criterion(...), backward(...) และstep(...)

- **ขั้นตอน Forward Pass**
  1. โมเดลคำนวณค่าผลลัพธ์ (output_train) จากข้อมูลอินพุต (x_train) โดยผ่านเลเยอร์ต่างๆ ของโมเดล
  2. ค่าความสูญเสีย (loss_train) ถูกคำนวณโดยใช้ฟังก์ชันความสูญเสีย (loss function) เช่น MSELoss หรือ CrossEntropyLoss เพื่อวัดว่าผลลัพธ์ของโมเดลแตกต่างจากค่าจริง (target) มากแค่ไหน
- **ขั้นตอน Backward Pass**
  1. เมื่อเรียก loss_train.backward() PyTorch จะคำนวณ gradients ของค่าความสูญเสียย้อนกลับไปยังพารามิเตอร์ของโมเดลทุกตัวที่กำหนดให้มี requires_grad=True
  2. ค่ากราดิเอนต์เหล่านี้จะบอกว่าแต่ละพารามิเตอร์ (เช่น weights, biases) ควรปรับอย่างไรเพื่อให้ค่าความสูญเสียลดลง
- **ขั้นตอน การอัปเดตพารามิเตอร์**
  1. เมื่อทำ backward pass ผ่านการใช้ loss.backward() แล้ว ค่า gradients ของพารามิเตอร์แต่ละตัวในโมเดลจะถูกคำนวณและเก็บไว้ในตัวแปรที่เกี่ยวข้องกับพารามิเตอร์นั้นๆ
  2. คำสั่ง optimizer.step() จะใช้ค่า gradients เหล่านั้นในการปรับค่าพารามิเตอร์ (เช่น weights, biases) เพื่อให้โมเดลเรียนรู้จากข้อมูลและลดค่าความสูญเสียในรอบถัดไป





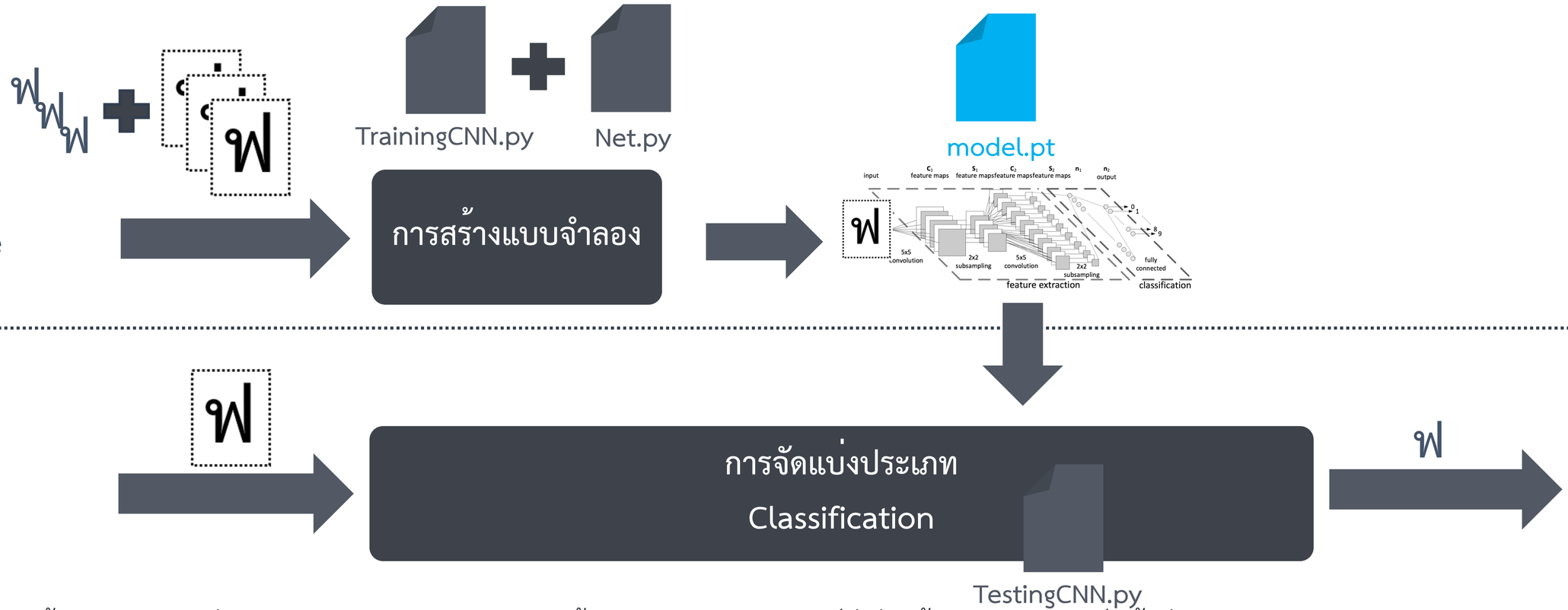
Training Model

ฝึกสอนแบบจำลอง

```
# Saving Entire Model
# A common PyTorch convention is to save models using either a .pt or .pth file extension.

torch.save(model, 'C:/model04.pt')
```

หลักการรู้จำภาพ



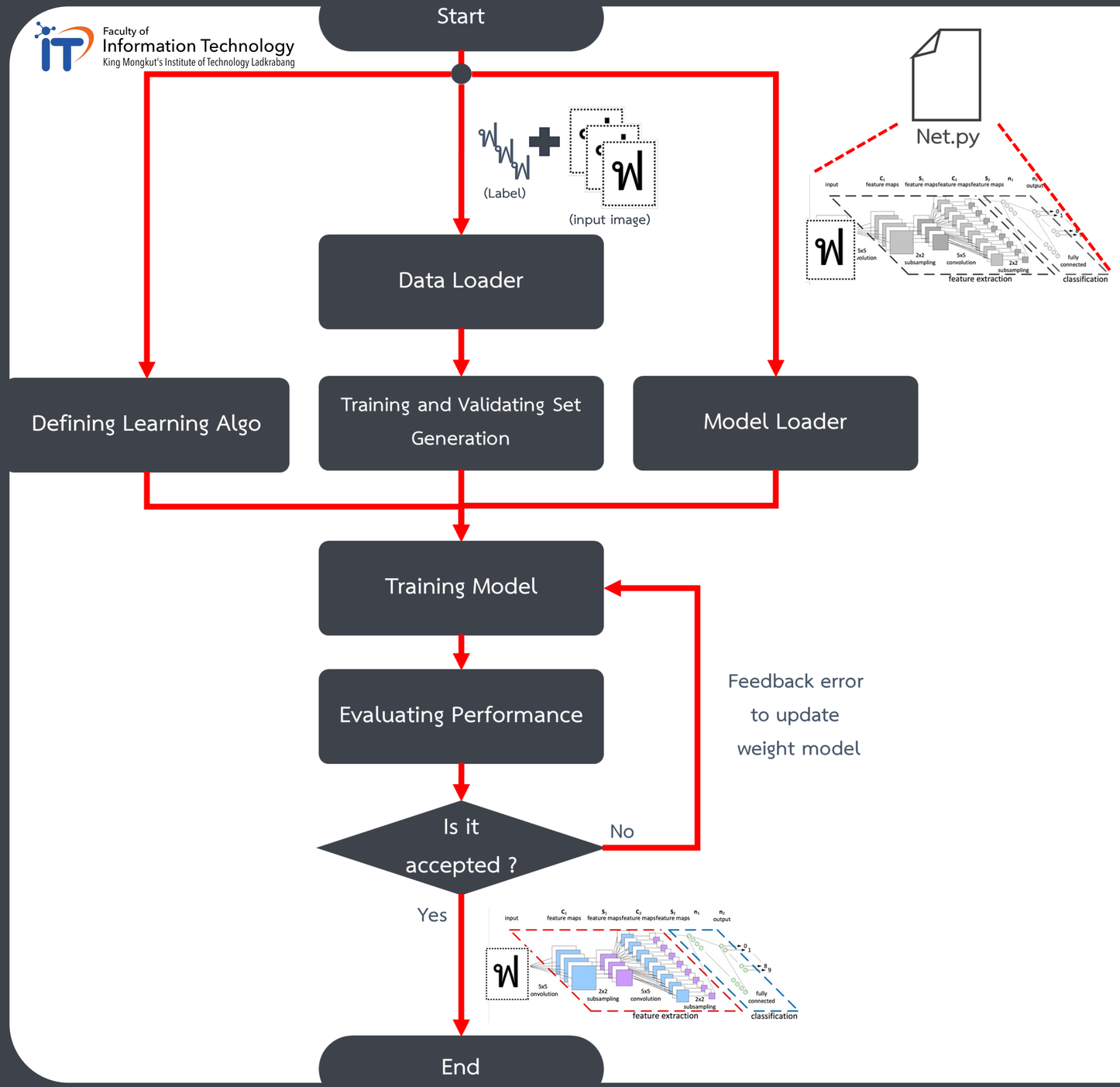
ดังนั้น เมื่อนักศึกษาได้โปรแกรมโครงข่ายประสาทเทียมแบบคอนโวลูชันด้วยภาษาไพทอนจะมีไฟล์ที่เกี่ยวข้องจำนวน 3 ไฟล์ ได้แก่

(1) *Net* เป็นไฟล์ที่ใช้กำหนดโครงสร้างของแบบจำลอง

(2) *TrainingCNN* เป็นไฟล์ที่เรียกใช้งานไฟล์ Net และนำมา***ฝึกสอน***แบบจำลอง

(3) *TestingCNN* เป็นไฟล์ที่เรียกใช้งานไฟล์ Net และนำมา***ทดสอบ***แบบจำลอง

Faculty of
Information Technology
King Mongkut's Institute of Technology Ladkrabang



ทดลองใช้แบบจำลอง (option)

นำข้อมูลจากชุด Validate มาทดสอบกับแบบจำลองที่ฝึกสอนไว้แล้ว

```
# prediction for validation set
# don't need gradients in test step since the parameter updates has been done in training step. Using 'torch.no_grad()' in the test and validation phase yields the faster inference(speed up computation) and reduced memory usage(which allows us to use larger size of batch).

with torch.no_grad():
    #output = model(val_x.cuda())
    output = model(val_x)

softmax = torch.exp(output).cpu()
prob = list(softmax.numpy())
predictions = np.argmax(prob, axis=1)

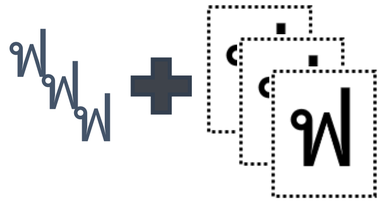
# accuracy on validation set
accuracy_score(val_y, predictions)
```

ฟังก์ชัน optimizer.zero_grad() ใน PyTorch

การใช้ torch.no_grad() ร่วมกับ output_train = model(x_train) เป็นวิธีการปิดการคำนวณ gradients ในระหว่างการทำ forward pass ใน PyTorch ซึ่งมีประโยชน์ในหลายสถานการณ์ เช่น การทดสอบโมเดล (inference) ที่เราไม่ต้องการคำนวณ gradients เพื่อลดการใช้หน่วยความจำและเพิ่มความเร็วในการประมวลผล

โดยที่ การทำงานของ torch.no_grad(): ถือว่าเป็นการปิดการคำนวณค่า gradients สำหรับ tensors ทั้งหมดที่อยู่ภายในบล็อกของคำสั่ง with torch.no_grad() ซึ่งมีประโยชน์ในช่วงการทดสอบ (inference) เมื่อเราต้องการคำนวณ output โดยไม่จำเป็นต้องอัปเดตพารามิเตอร์ (weights) ทำให้ลดการใช้หน่วยความจำ และเพิ่มความเร็ว เนื่องจากไม่ต้องคำนวณค่า gradients การทำ forward pass จะเร็วขึ้น

หลักการรู้จำภาพ



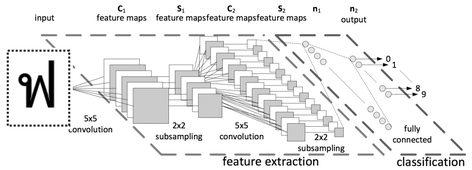




Training Phase

(กระบวนการฝึกสอน)

การสร้างแบบจำลอง



Testing Phase

(กระบวนการทดสอบ)



การจัดแบ่งประเภท

Classification



TestingCNN.py

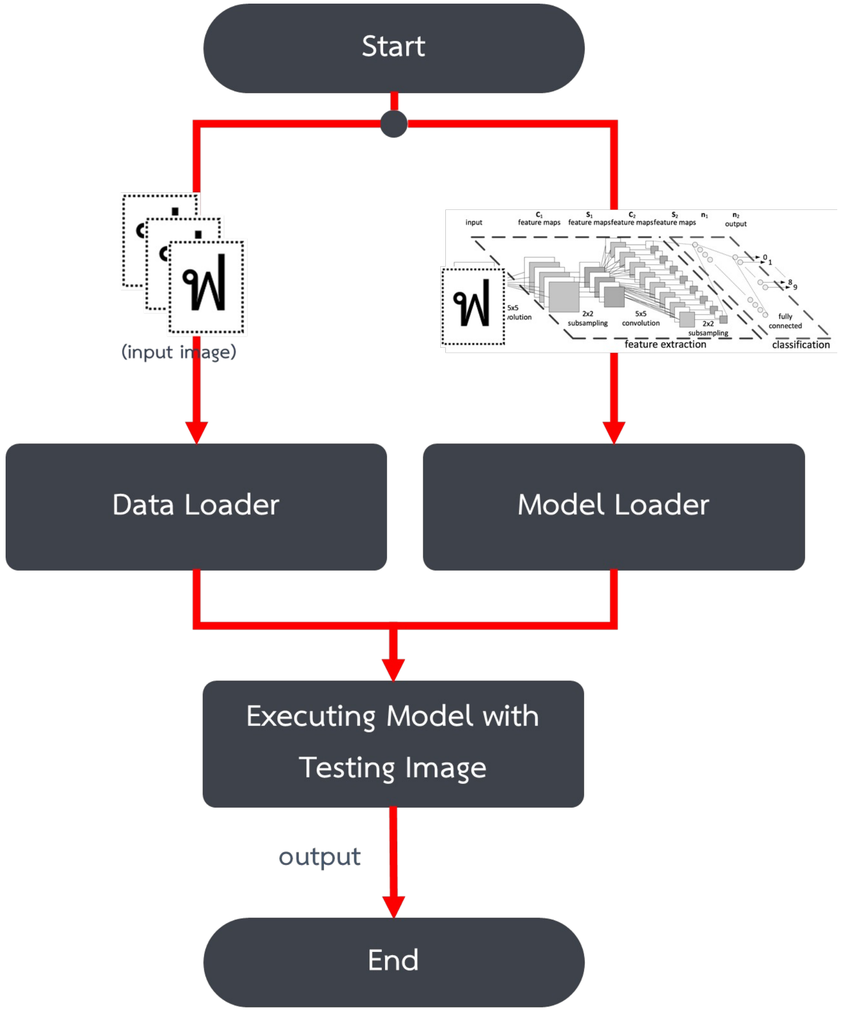
ฟ

ดังนั้น เมื่อนักศึกษาได้โปรแกรมโครงข่ายประสาทเทียมแบบคอนโวลูชันด้วยภาษาไพทอนจะมีไฟล์ที่เกี่ยวข้องจำนวน 3 ไฟล์ ได้แก่

(1) *Net* เป็นไฟล์ที่ใช้กำหนดโครงสร้างของแบบจำลอง

(2) *TrainingCNN* เป็นไฟล์ที่เรียกใช้งานไฟล์ Net และนำมา***ฝึกสอน***แบบจำลอง

(3) *TestingCNN* เป็นไฟล์ที่เรียกใช้งานไฟล์ Net และนำมา***ทดสอบ***แบบจำลอง





Import ชุดคำสั่งที่จำเป็น

มีความจำเป็นต้องใช้คำสั่งต่าง ๆ มากมายที่มาจาก
หลากหลายชุดคำสั่ง เพื่อให้สามารถฝึกสอนเครื่องจักรได้

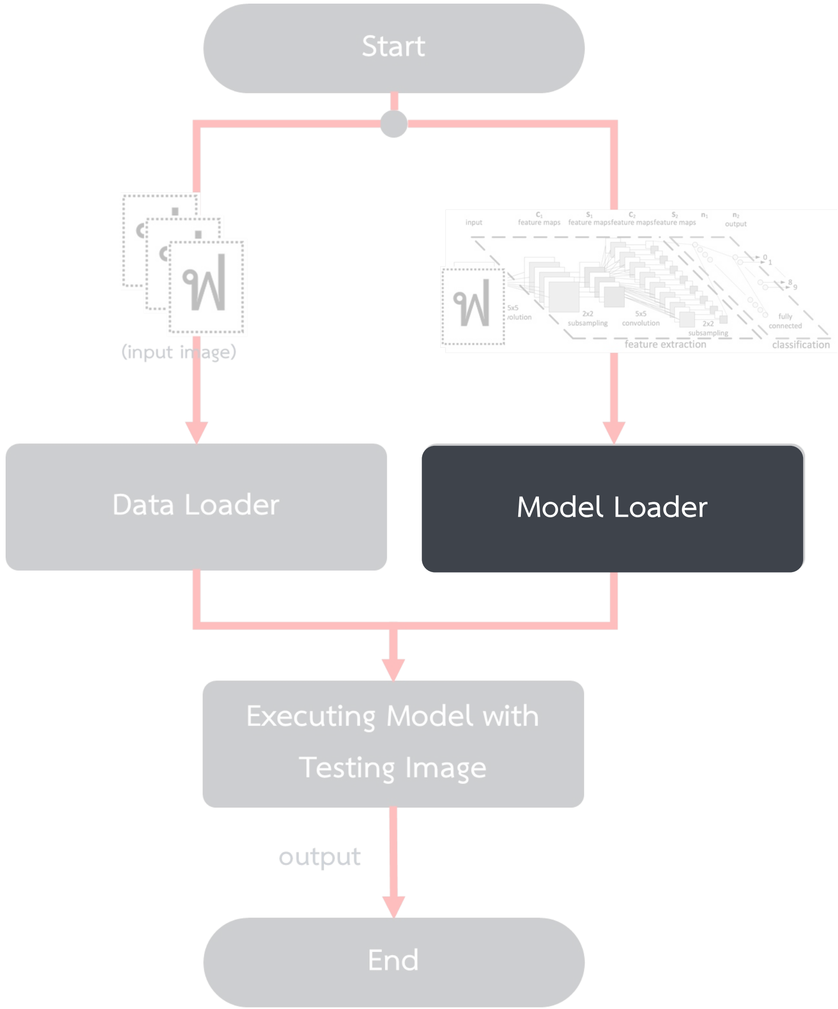
```
import pandas as pd
import numpy as np
from Net import Net

# for reading and displaying images
from skimage.io import imread
import matplotlib.pyplot as plt

# for creating validation set
from sklearn.model_selection import train_test_split

# for evaluating the model
from sklearn.metrics import accuracy_score
from tqdm import tqdm

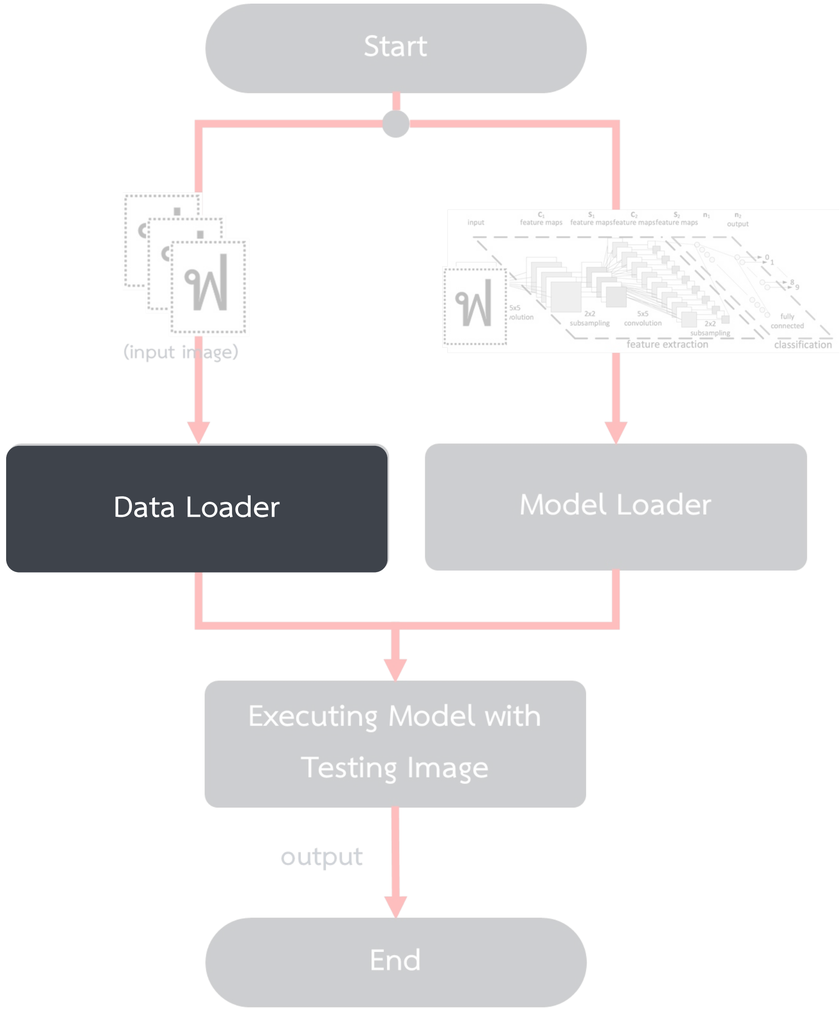
# PyTorch libraries and modules
import torch
from torch.autograd import Variable
from torch.nn import Linear, ReLU, CrossEntropyLoss
, Sequential, Conv2d, MaxPool2d, Module, Softmax, B
atchNorm2d, Dropout
from torch.optim import Adam, SGD
```



Model Loader

การโหลดโครงสร้างของสถาปัตยกรรมที่จะใช้งาน
เข้าสู่โปรแกรม

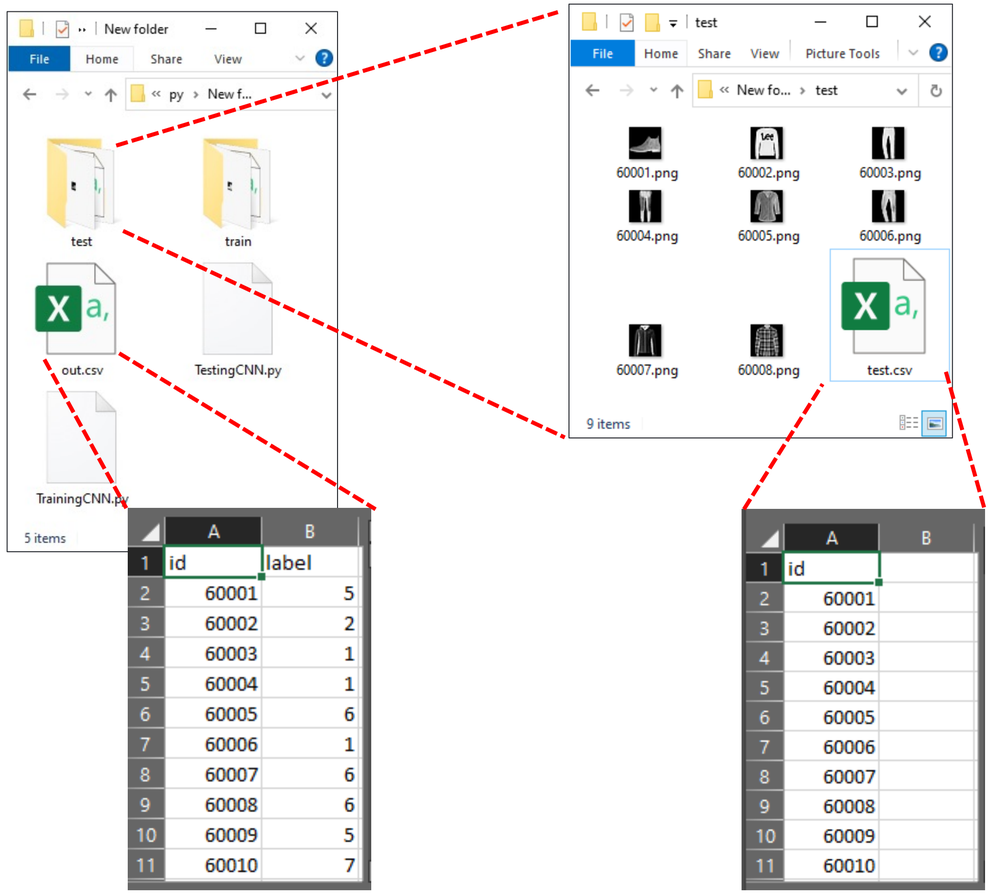
```
# Load the Trained Model from model04.pt
model = torch.load('C:/model04.pt')
```



Data Loader

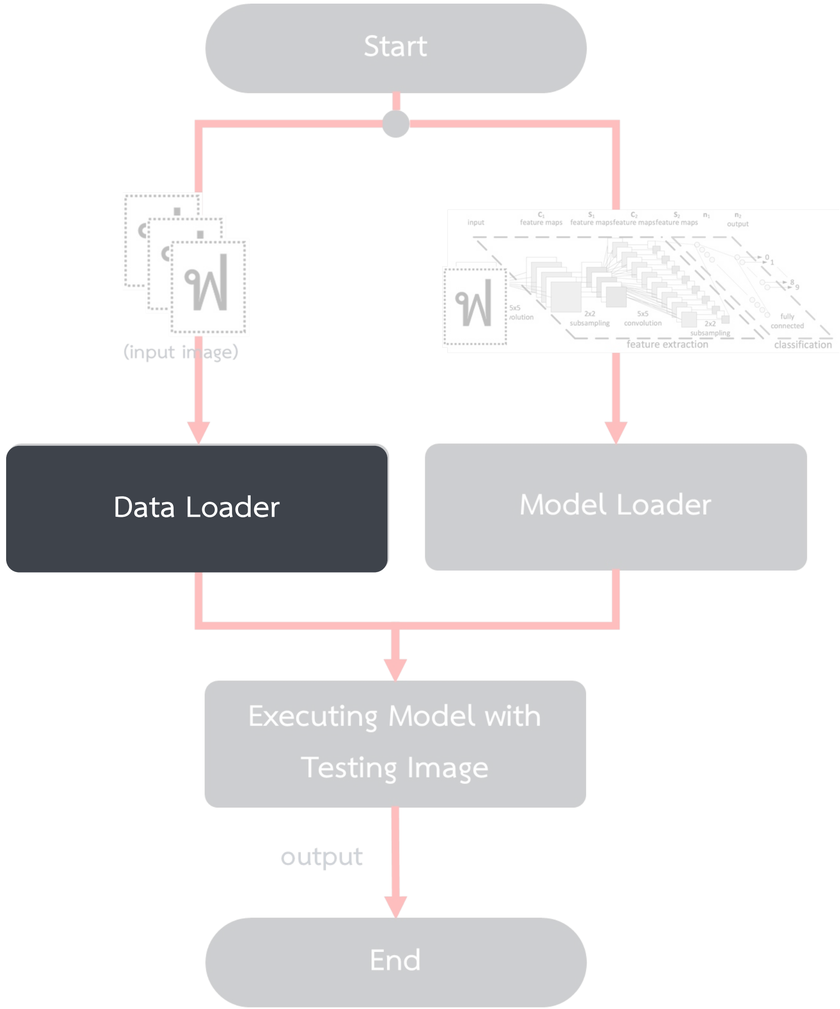
การโหลดข้อมูลรูปภาพเข้าสู่โปรแกรม (ไม่มีเฉลย)

```
# Load Data File
test = pd.read_csv('C:/test/test.csv')
outXls = pd.read_csv('C:/out.csv')
```



	A	B
1	id	label
2	60001	5
3	60002	2
4	60003	1
5	60004	1
6	60005	6
7	60006	1
8	60007	6
9	60008	6
10	60009	5
11	60010	7

	A	B
1	id	
2	60001	
3	60002	
4	60003	
5	60004	
6	60005	
7	60006	
8	60007	
9	60008	
10	60009	
11	60010	



Data Loader

การโหลดข้อมูลรูปภาพเข้าสู่โปรแกรม (ไม่มีเฉลย)

```
# Load Data File
test = pd.read_csv('C:/test/test.csv')
outXls = pd.read_csv('C:/out.csv')

# loading test images
test_img = []
for img_name in tqdm(test['id']):
    # defining the image path
    image_path = '/test/' + str(img_name) + '.png'

    # reading the image
    img = imread(image_path, as_gray=True)

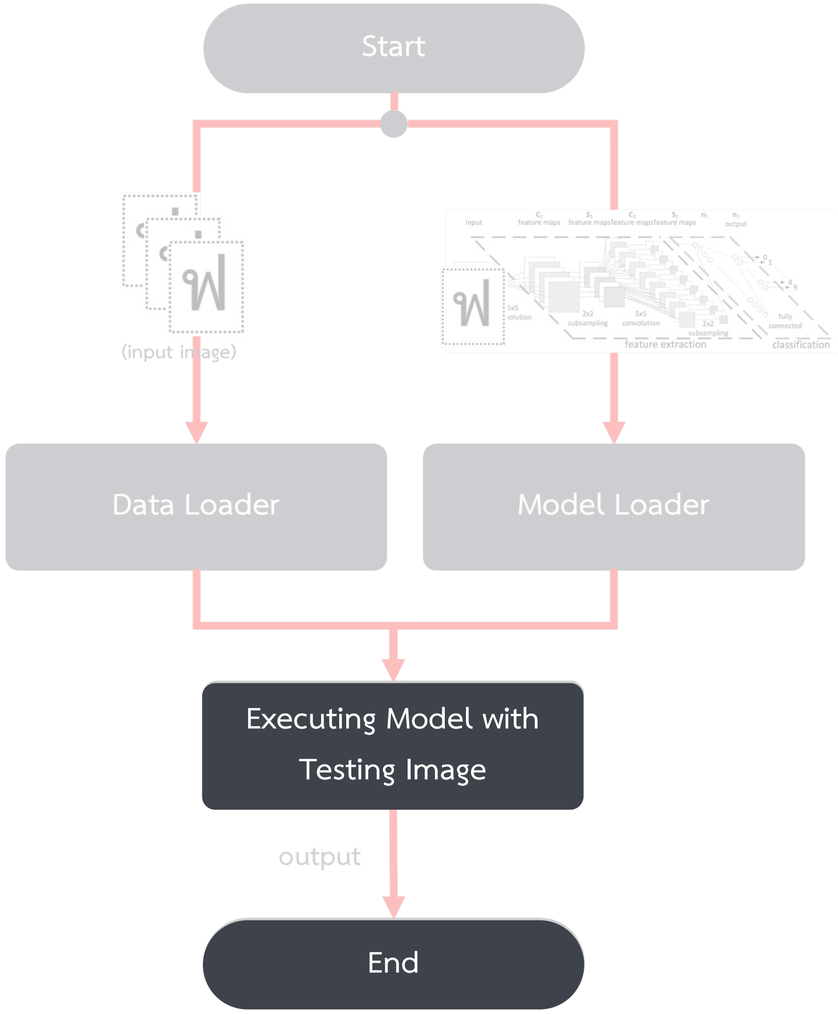
    # normalizing the pixel values
    img /= 255.0

    # converting the type of pixel to float 32
    img = img.astype('float32')

    # appending the image into the list
    test_img.append(img)

# converting the list to numpy array
test_x = np.array(test_img)

# converting training images into torch format
test_x = test_x.reshape(10000, 1, 28, 28)
test_x = torch.from_numpy(test_x).to(torch.float32)
```



Executing Model with Testing Image

การนำรูปภาพมาทดสอบกับแบบจำลองที่ฝึกสอนแล้ว

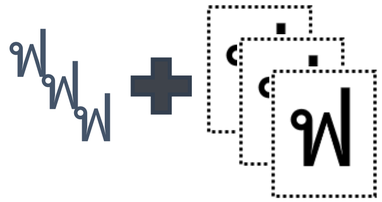
```
# generating predictions for test set
with torch.no_grad():
    #output = model(test_x.cuda())
    output = model(test_x)

softmax = torch.exp(output).cpu()
prob = list(softmax.numpy())
predictions = np.argmax(prob, axis=1)

# replacing the label with prediction
outXls['label'] = predictions
outXls.head()

# saving the file
outXls.to_csv('C:/out.csv', index=False)
```

หลักการรู้จำภาพ



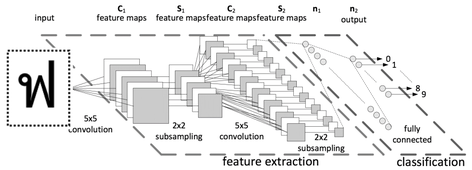


TrainingCNN.py Net.py

การสร้างแบบจำลอง



model.pt



Training Phase

(กระบวนการฝึกสอน)

Testing Phase

(กระบวนการทดสอบ)



การจัดแบ่งประเภท

Classification



TestingCNN.py

ฟ

ดังนั้น เมื่อนักศึกษาได้โปรแกรมโครงข่ายประสาทเทียมแบบคอนโวลูชันด้วยภาษาไพทอนจะมีไฟล์ที่เกี่ยวข้องจำนวน 3 ไฟล์ ได้แก่

(1) *Net* เป็นไฟล์ที่ใช้กำหนดโครงสร้างของแบบจำลอง

(2) *TrainingCNN* เป็นไฟล์ที่เรียกใช้งานไฟล์ Net และนำมา***ฝึกสอน***แบบจำลอง

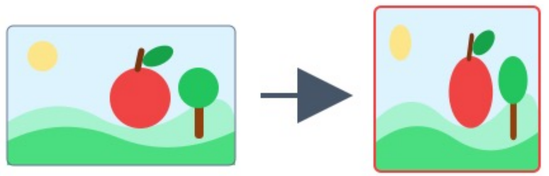
(3) *TestingCNN* เป็นไฟล์ที่เรียกใช้งานไฟล์ Net และนำมา***ทดสอบ***แบบจำลอง

การปรับภาพก่อนนำเข้าสู่ CNN

ตัวอย่างการแปลงภาพอัตราส่วน 16:10 ให้เป็นอินพุตสี่เหลี่ยมจัตุรัส เช่น 224 × 224 × 3

ขนาดที่ต้องใช้จริงขึ้นอยู่กับ Architecture และ Pretrained Weights

1. Direct Resize (Stretch)



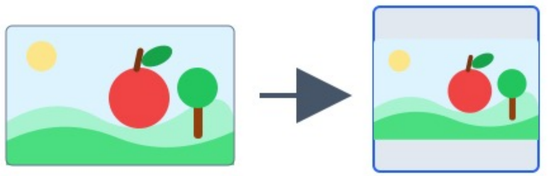
บังคับ H × W ให้ตรงเป้าหมายโดยไม่รักษาสัดส่วน

เหมาะกับ: ภาพที่มีสัดส่วนใกล้เคียงกัน

ระวัง: วัตถุอาจยืด บีบ หรือรูปร่างผิดเพี้ยน

รวดเร็วและง่าย แต่ไม่ควรใช้เมื่อรูปร่างเป็นคุณลักษณะสำคัญ

2. Resize + Padding (Letterbox)



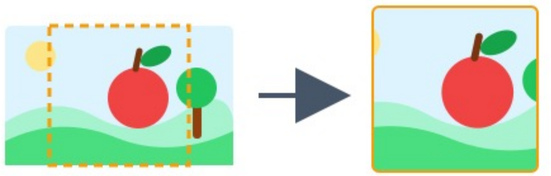
รักษาอัตราส่วน แล้วเติมพื้นที่ว่างให้ครบขนาด

เหมาะกับ: Detection, Segmentation, OCR

ระวัง: Padding อาจกลายเป็นลักษณะที่โมเดลจดจำ

เหมาะเมื่อภาพมีหลายอัตราส่วนและต้องรักษารูปร่างวัตถุ

3. Center Crop + Resize



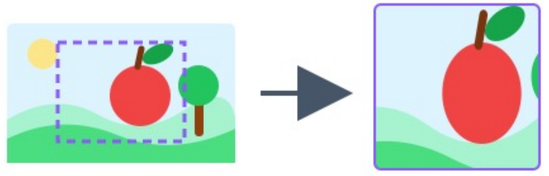
ตัดบริเวณกึ่งกลาง แล้วปรับเป็นขนาดเป้าหมาย

เหมาะกับ: สินค้า ใบหน้า หรือวัตถุที่อยู่กลางภาพ

ระวัง: วัตถุบริเวณขอบภาพอาจถูกตัดหาย

ไม่มี padding และไม่บิดสัดส่วน แต่ต้องมั่นใจในตำแหน่งวัตถุ

4. Random Resized Crop



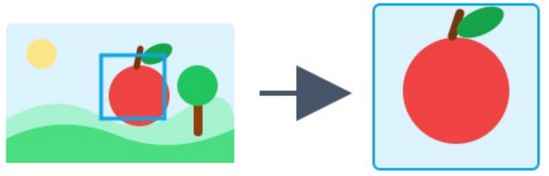
สุ่มตำแหน่งและขนาดของ crop ระหว่างการฝึก

เหมาะกับ: Data augmentation สำหรับ Classification

ระวัง: อาจตัดคุณลักษณะสำคัญหรือเปลี่ยนความหมาย

ใช้แบบสุ่มเฉพาะ Training; Validation/Test ใช้แบบคงที่

5. ROI / Object-Centric Crop



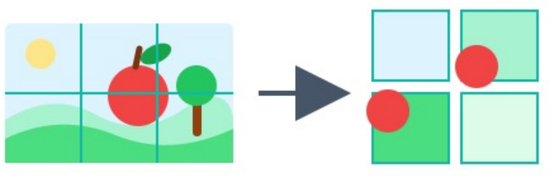
ตัดเฉพาะกรอบวัตถุหรือบริเวณที่สนใจ

เหมาะกับ: ภาพแพทย์ ใบไม้ รอยโรค และ Tiny Object

ระวัง: ต้องมี Bounding Box, Mask หรือตัวตรวจหา ROI

ช่วยลดพื้นหลังและทำให้วัตถุขนาดเล็กเห็นชัดขึ้น

6. Tiling / Sliding Window



แบ่งภาพความละเอียดสูงเป็น Patch ขนาดเล็ก

เหมาะกับ: Histology, Satellite และ Segmentation

ระวัง: ใช้เวลามากขึ้นและต้องรวมผลของแต่ละ Patch

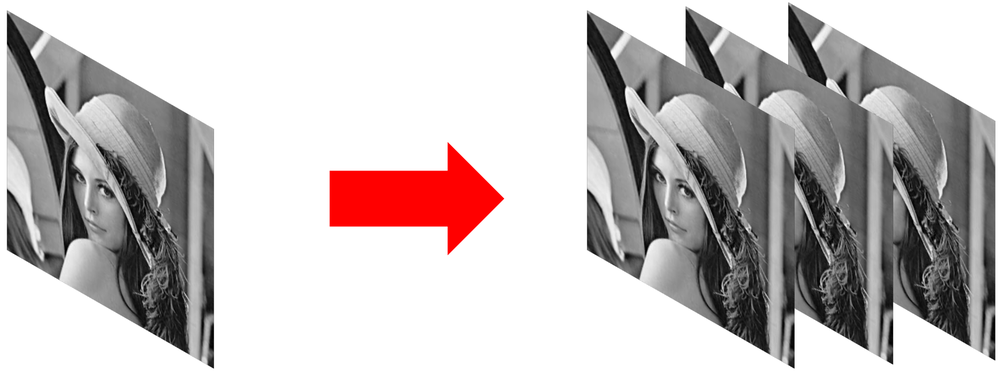
ใช้ Overlap เพื่อลดปัญหาวัตถุถูกตัดตรงขอบ Patch

ข้อควรจำ: ภาพทั่วไปใช้ Bilinear/Bicubic • Segmentation Mask ใช้ Nearest-neighbor • Image และ Label ต้องถูกแปลงด้วยพารามิเตอร์เดียวกัน

ควรทำ Color conversion และ Normalization ให้ตรงกับโมเดล pretrained หลังจากจัดการขนาดและพื้นที่ภาพแล้ว

ข้อควรระวัง

ข้อควรระวัง : input image ขนาด 227 × 227 × 3 แสดงว่า ข้อมูลต้องมี 3 แพลน อาทิเช่น ภาพสี แต่ถ้าข้อมูลเป็นภาพขาวดำหรือระดับสีเทาต้องทำอย่างไร





เสริมท้ายบท

- **Regularization** คือ วิธีการที่ใช้แก้หรือบรรเทาปัญหา overfitting หรือ underfitting ของแบบจำลอง ซึ่งมี 3 วิธีการที่กำลังได้รับความนิยมอยู่ในปัจจุบัน ได้แก่ (1) Data Augmentation, (2) Batch Normalization และ (3) Dropout

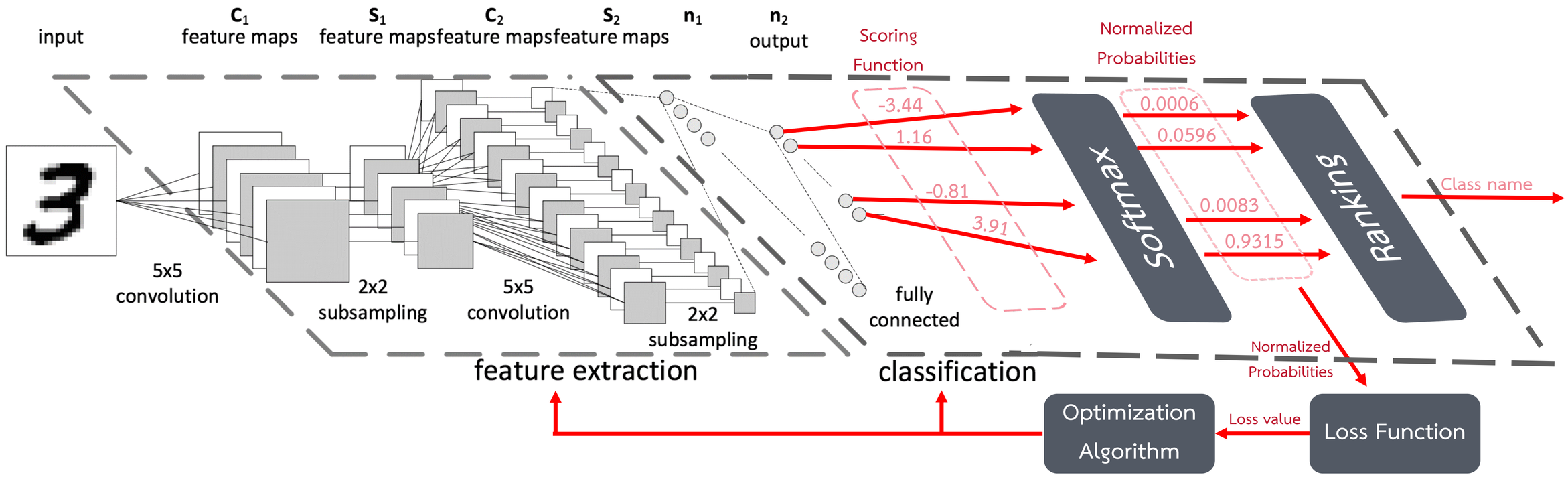
- **Epoch** คือ รอบของการฝึกสอนแบบจำลองด้วยข้อมูลทั้งหมดใน dataset ทุกตัว ดังนั้น 1 Epoch คือ การฝึกสอนแบบจำลอง โดยป้อนข้อมูลทั้งหมดใน dataset ทุกตัวเข้าแบบจำลอง

หลักการโครงข่ายประสาทแบบคอนโวลูชัน

Forward Propagation คือ กระบวนการที่ป้อน Input Image เข้าสู่แบบจำลองที่ฝึกสอนเรียบร้อยแล้ว และคืนค่าเป็นเป็น Classname

Backward Propagation คือ กระบวนการที่นำค่า Normalized Probabilities ไปใช้เพื่อปรับปรุง Weight ของแบบจำลองให้เกิดความแม่นยำมากยิ่งขึ้น

Forward Propagation : เพื่อรู้จำรูปภาพ



Backward Propagation : เพื่อปรับปรุงแบบจำลองจากข้อมูลฝึกสอน



การพัฒนาโปรแกรม

การโปรแกรมโครงข่ายประสาทแบบคอนโวลูชัน ด้วยภาษาไพธอนจากชุดคำสั่ง PyTorch ตอนที่ 2



หัวข้อ

- การสร้างโมเดล Image Classification แบบพื้นฐาน
- การสร้างโมเดลแบบกำหนดเอง (Custom CNN Model)
- การสร้าง Data Loader สำหรับชุดข้อมูลแบบกำหนดเอง
- การทำ Data Transformation และ Augmentation



หัวข้อ

- การสร้างโมเดล Image Classification แบบพื้นฐาน
- การสร้างโมเดลแบบกำหนดเอง (Custom CNN Model)
- การสร้าง Data Loader สำหรับชุดข้อมูลแบบกำหนดเอง
- การทำ Data Transformation และ Augmentation



การสร้างโมเดล Classification แบบพื้นฐาน

การสร้างโมเดล Classification แบบพื้นฐาน ประกอบไปด้วย 5 ขั้นตอนด้วยกัน ได้แก่

ข้อ	คำอธิบาย
1	กำหนด Dataset ที่จะนำมาใช้ในการฝึกสอนโมเดล
2	กำหนด Convolutional Neural Network สำหรับการฝึกสอนโมเดล
3	เลือก Loss Function และ Optimizer ที่เหมาะสมกับงานของเรา
4	การฝึกสอนโมเดล
5	การทดสอบและวัดผลโมเดล

ขั้นตอนดังกล่าว เป็นขั้นตอนในการสร้างโมเดล Classification แบบพื้นฐานของชุดคำสั่ง Pytorch ที่สามารถนำไปประยุกต์ใช้ หรือต่อยอดในงานประเภทอื่น ๆ ได้ เช่น Regression เป็นต้น

อ้างอิง : https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html

การสร้างโมเดล Classification แบบพื้นฐาน

```
import torch
import torchvision
import torchvision.transforms as transforms

transform = transforms.Compose([
     transforms.ToImage(),  # Convert to image format
     transforms.ConvertImageDtype(torch.float32),  # Convert image dtype
     transforms.Normalize(mean=(0.5, 0.5, 0.5), std=(0.5, 0.5, 0.5))
])
```

ขั้นตอนที่ 1 : กระบวนการ Pre-processing เป็นการรวบรวม pre-process ที่ต้องการทำกับข้อมูล อาทิเช่น data augmentation, data normalization, data standardization หรือ data transformation เป็นต้น โดยอาศัยคำสั่ง **transforms.Compose** จากตัวอย่างโค้ด (1) มีการแปลงชนิดข้อมูลให้อยู่ในdatatype ของ Pytorch ได้แก่ tensor และจากนั้น (2) ทำการ normalize ภาพให้อยู่ในช่วง -1 ถึง 1 เพื่อลดปัญหา bias ที่อาจจะเกิดขึ้นระหว่างการฝึกสอนแบบจำลอง (หรือโมเดล)

การสร้างโมเดล Classification แบบพื้นฐาน

```python
import torch
import torchvision
import torchvision.transforms as transforms

transform = transforms.Compose([
     transforms.ToImage(),  # Convert to image format
     transforms.ConvertImageDtype(torch.float32),  # Convert image dtype
     transforms.Normalize(mean=(0.5, 0.5, 0.5), std=(0.5, 0.5, 0.5))
])
```





อ้างอิง https://www.geeksforgeeks.org/how-to-normalize-images-in-pytorch/

ขั้นตอนที่ 2 : การปรับค่าของข้อมูลภาพให้อยู่ในช่วงที่เหมาะสมสำหรับการฝึกโมเดล โดยทำการลบค่าเฉลี่ยและหารด้วยค่าความเบี่ยงเบนมาตรฐานของข้อมูลแต่ละช่องสัญญาณ (channel) เช่น RGB โดยอาศัยคำสั่ง transforms.Normalize ใน Convolutional Neural Networks (CNN) ตัวอย่างการใช้

python Copy code

```python
import torchvision.transforms as transforms

# กำหนดค่าเฉลี่ยและค่าความเบี่ยงเบนมาตรฐานของแต่ละช่องสัญญาณ (เช่น RGB)
mean = [0.485, 0.456, 0.406]
std = [0.229, 0.224, 0.225]

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean, std)
])
```

ในตัวอย่างนี้ ค่าของช่องสัญญาณแต่ละช่อง (R, G, B) จะถูก normalize โดยลบค่าเฉลี่ยและหารด้วยค่าความเบี่ยงเบนมาตรฐานตามลำดับที่กำหนดไว้ การทำเช่นนี้ช่วยให้การเรียนรู้ของโมเดลมีประสิทธิภาพมากขึ้นและสามารถหาค่าพารามิเตอร์ที่เหมาะสมได้เร็วขึ้น

การสร้างโมเดล Classification แบบพื้นฐาน

```
import torch
import torchvision
import torchvision.transforms as transforms

transform = transforms.Compose([
     transforms.ToImage(),  # Convert to image format
     transforms.ConvertImageDtype(torch.float32),  # Convert image dtype
     transforms.Normalize(mean=(0.5, 0.5, 0.5), std=(0.5, 0.5, 0.5))
])

batch_size = 4

# Define training dataset and eval dataset
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
shuffle=True, num_workers=2)


testset = torchvision.datasets.CIFAR10(root='./data', train=False,
download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
shuffle=False, num_workers=2)

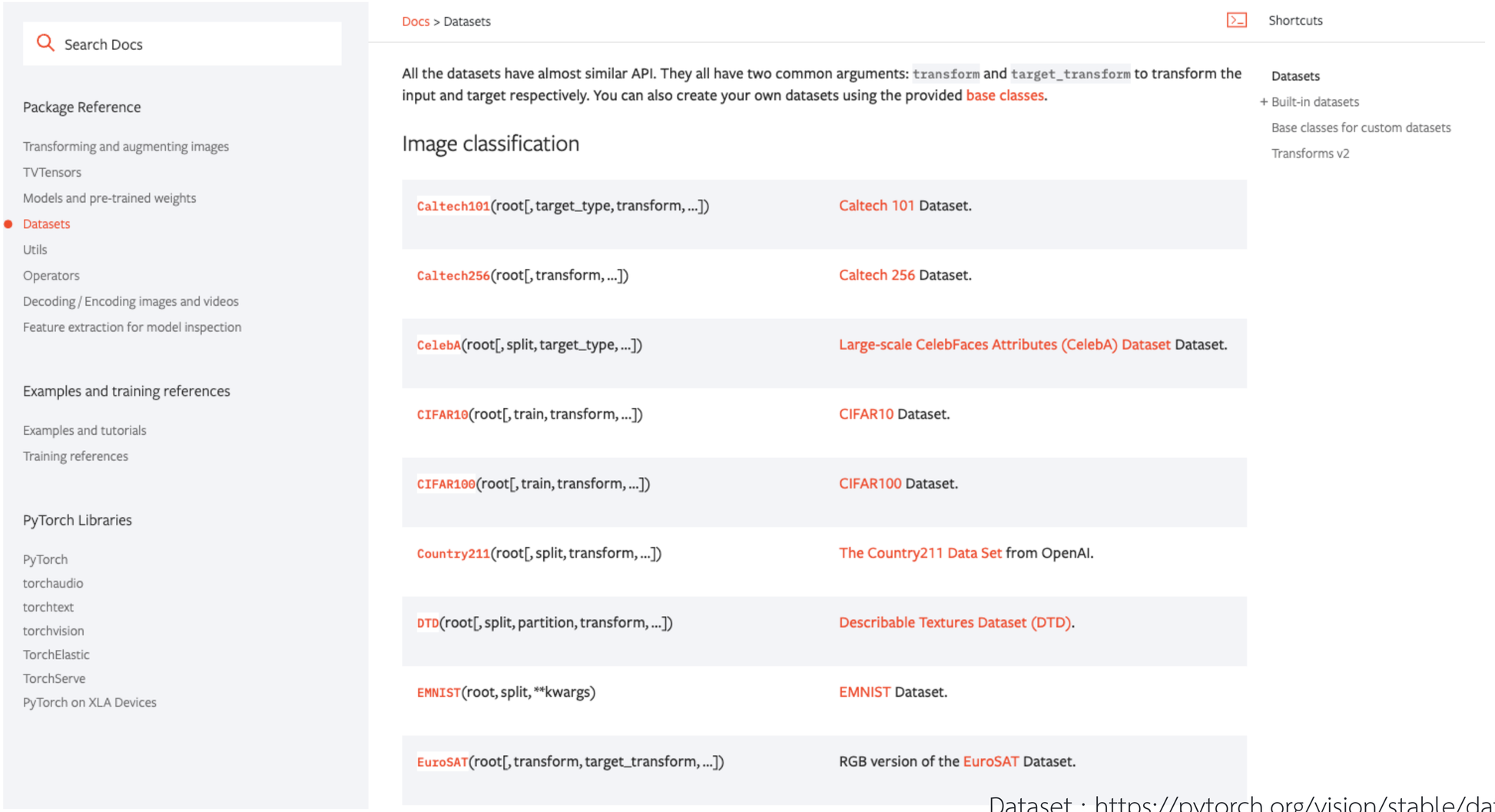
classes = ('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',
'ship', 'truck')
```

ขั้นตอนที่ 3 : การกำหนด Dataset ที่จะนำมาใช้ในการฝึกสอนโมเดล

สำหรับชุดคำสั่ง Pytorch มี Dataset สำเร็จรูปให้เลือกใช้มากมาย ไม่ว่าจะเป็น CIFAR10, FashionMNIST, ImageNet นอกจากนี้ยังมี Dataset สำหรับงานอื่น ๆ อีกด้วยโดยสามารถเข้าไปดูเพิ่มเติมได้ที่ https://pytorch.org/vision/stable/datasets.html สำหรับโค้ดตัวอย่างเป็นการนำ Dataset ที่มีอยู่แล้วมาประยุกต์ใช้งาน โดยใช้คำสั่ง **torchvision.datasets.XXX** โดยมีพารามิเตอร์ที่ต้องกำหนดดังนี้

พารามิเตอร์	คำอธิบาย
root	ตำแหน่งในการดาวน์โหลดเมื่อ download=true
train	ชุดในการนำข้อมูลไปใช้
download	ดาวน์โหลดชุดข้อมูลไปเก็บไว้ที่ root
transform	นำชุดข้อมูลไปผ่านกระบวนการบางอย่างก่อนไปทดสอบหรือฝึกโมเดล

การสร้างโมเดล Classification แบบพื้นฐาน



Search Docs

Package Reference

Transforming and augmenting images
TVTensors
Models and pre-trained weights
Datasets
Utils
Operators
Decoding / Encoding images and videos
Feature extraction for model inspection

Examples and training references

Examples and tutorials
Training references

PyTorch Libraries

PyTorch
torchaudio
torchtext
torchvision
TorchElastic
TorchServe
PyTorch on XLA Devices

Docs > Datasets

Shortcuts

All the datasets have almost similar API. They all have two common arguments: `transform` and `target_transform` to transform the input and target respectively. You can also create your own datasets using the provided base classes.

Image classification

`Caltech101`(root[, target_type, transform, ...])	Caltech 101 Dataset.
`Caltech256`(root[, transform, ...])	Caltech 256 Dataset.
`CelebA`(root[, split, target_type, ...])	Large-scale CelebFaces Attributes (CelebA) Dataset Dataset.
`CIFAR10`(root[, train, transform, ...])	CIFAR10 Dataset.
`CIFAR100`(root[, train, transform, ...])	CIFAR100 Dataset.
`Country211`(root[, split, transform, ...])	The Country211 Data Set from OpenAI.
`DTD`(root[, split, partition, transform, ...])	Describable Textures Dataset (DTD).
`EMNIST`(root, split, **kwargs)	EMNIST Dataset.
`EuroSAT`(root[, transform, target_transform, ...])	RGB version of the EuroSAT Dataset.

Datasets
+ Built-in datasets
Base classes for custom datasets
Transforms v2

Dataset : https://pytorch.org/vision/stable/datasets.html



การสร้างโมเดล Classification แบบพื้นฐาน

```python
import torch.nn as nn
import torch.nn.functional as F          # Recommend for Not Customer

class ClassificationNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        # nn.Conv2d(in_channels, out_channels, kernel_size)
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        # flatten all dimensions except batch
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

model = ClassificationNetwork()
# flatten() convert N-D to 1D but view() convert N-D to (N-x)D
```

ขั้นตอนที่ 4 : การสร้างแบบจำลองแบบกำหนดเอง สามารถทำได้โดยสร้างคลาสใหม่ขึ้นมา จากนั้น สืบทอด torch.nn. Module มายังคลาสที่สร้างเพื่อดึงสืบทอดคุณลักษณะ (Attribute) และฟังก์ชันที่จำเป็นมาใช้งาน ในการกำหนด Network จำเป็นต้องกำหนดฟังก์ชันดังนี้

- __init__() ใช้สำหรับสร้างหรือกำหนด (define) องค์ประกอบของโมเดลที่ต้องการสร้าง
- forward() ใช้สำหรับในการกำหนดลำดับการทำงานของโมเดลจากบนลงล่าง

ข้อมูลเพิ่มเติม torch.nn : https://pytorch.org/docs/stable/nn.html#convolution-layers



การสร้างโมเดล Classification แบบพื้นฐาน

```python
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```

ขั้นตอนที่ 5 : การกำหนด Loss Function และ Optimizer ที่เหมาะสมกับงานของเรา

- การเรียนรู้ของเครื่องจักรมีความหลากหลายของปัญหาหรือประเภทงานค่อนข้างมาก โดยแต่ละงานก็จะมี Loss Function ที่เหมาะสมแตกต่างกันดังนั้น เพื่อให้ได้รับแบบจำลองที่ดีและเหมะสม จึงจำเป็นต้องกำหนด Loss Function แตกต่างกันในงานแต่ละประเภท
- Optimizer เป็นการคำนวณหาจุด Optimal ที่ทำให้แบบจำลองทำนายได้มีความแม่นยำมากที่สุดตามหลักคณิตศาสตร์ โดยที่ Optimizer แต่ละตัวมีขั้นตอนและหลักการคำนวณที่ต่างกัน ทำให้พารามิเตอร์มีความแตกต่างกัน

ข้อมูลเพิ่มเติม

Loss Functions : https://pytorch.org/docs/stable/nn.html#loss-functions

Optimizer : https://pytorch.org/docs/stable/optim.html#algorithms



การสร้างโมเดล Classification แบบพื้นฐาน

```
num_epochs = 10
for epoch in range(num_epochs ): # loop over the dataset multiple times

    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # I is round number start from 0 to N.
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data

        # zero the parameter gradients
        optimizer.zero_grad()

        # forward + backward + optimize
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if i % 2000 == 1999:    # print every 2000 mini-batches
            print(f'[{epoch + 1}, {i + 1:5d}] loss: {running_loss /
            2000:.3f}')
            running_loss = 0.0

print('Finished Training')
```

ขั้นตอนที่ 6 : การฝึกสอนแบบจำลองโมเดล



การสร้างโมเดล Classification แบบพื้นฐาน

```
correct = 0
total = 0
# since we're not training, we don't need to calculate the gradients
for our outputs

model.eval()           # set disable all functions for training model
with torch.no_grad(): # return resource (or memory) for training

    for data in testloader:
        images, labels = data
        # calculate outputs by running images through the network
        outputs = model(images)
        # the class with the highest value is our prediction
        _, predicted = torch.max(outputs.data, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f'Accuracy of the network on the test images: {100 * correct //
total} %')
```

ขั้นตอนที่ 7 : ในการทดสอบและการวัดผลโมเดล ผู้พัฒนาควรระวัง ไม่ให้แบบจำลองมีการเรียนรู้จากชุดข้อมูลทดสอบเพิ่มเติม เพื่อไม่ให้แบบจำลองปรับปรุง Weight และ Bias ของแบบจำลองที่ได้รับการฝึกสอนเรียบร้อย โดยอาศัยคำสั่ง torch.no_grad() เพื่อกำหนดให้แบบจำลองไม่คำนวณหรือปรับปรุง Weight และ Bias ของแบบจำลอง และควรใส่อาศัยคำสั่ง [โมเดล Pytorch].eval() เช่น model.eval() เพื่อป้องกันไม่ให้เกิดการเรียนรู้ระหว่างทำการทดสอบกับชุดข้อมูลใหม่



หัวข้อ

- การสร้างโมเดล Image Classification แบบพื้นฐาน
- **การสร้างโมเดลแบบกำหนดเอง (Custom CNN Model)**
- การสร้าง Data Loader สำหรับชุดข้อมูลแบบกำหนดเอง
- การทำ Data Transformation และ Augmentation

การสร้างโมเดลแบบกำหนดเอง

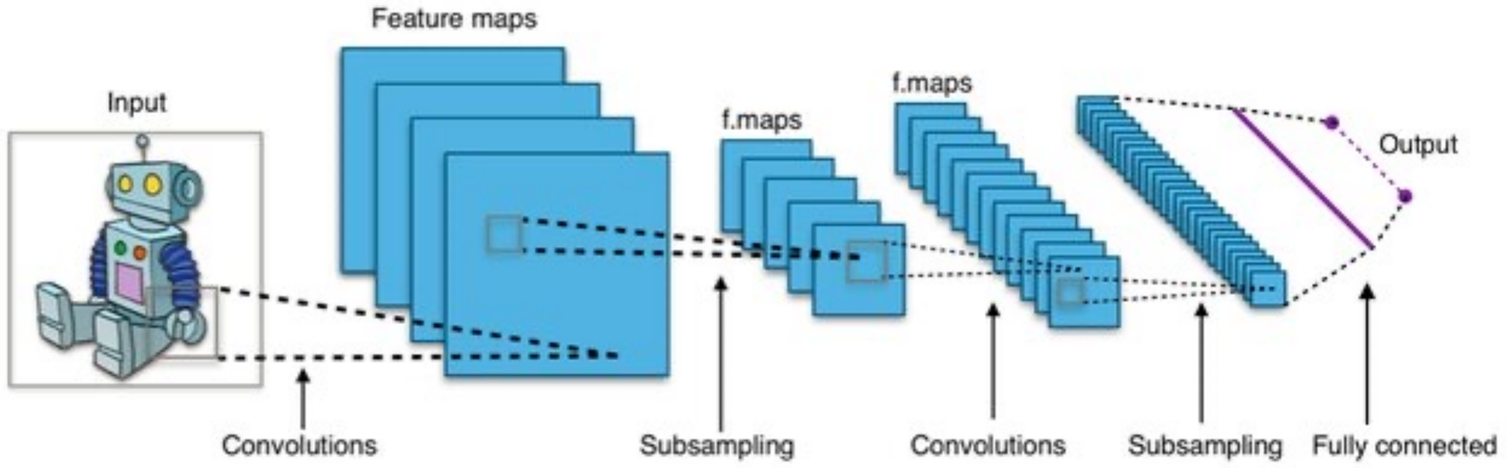


Image by Aphex34 (https://en.wikipedia.org/).

ในการสร้างโมเดลแบบกำหนดเองจากชุดคำสั่ง Pytorch จะเกี่ยวข้องกับคลาส torch.nn.Module และ torch.nn.Parameter ที่ทำงานร่วมกันอัตโนมัติเพียงแค่เราสร้างคลาสใหม่ขึ้นมา และสืบทอด (Inheritance) คลาส torch.nn.Module เท่านั้น ซึ่ง torch.nn.Module จะหน้าหน้าที่ในการจัดการเกี่ยวลักษณะของโมเดลรวมถึงพฤติกรรม (Behaviors) ในขณะที่ torch.nn.Parameter จะทำหน้าที่จัดการกับ weight และ bias ของแต่ละองค์ประกอบ (Component) ของโมเดล



การสร้างโมเดลแบบกำหนดเอง

```
import torch.nn as nn
import torch.nn.functional as F

class ClassificationNetwork(nn.Module):

    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

ในการสร้างโมเดลมีฟังก์ชันที่จำเป็น 2 ฟังก์ชัน ได้แก่

- **__init__()** ใช้สำหรับสร้างหรือกำหนด (define) องค์ประกอบของโมเดลที่ต้องการสร้าง เช่น Convolution Filter, Pooling หรือ Dense Layer ซึ่งหลักจากที่เรากำหนดเรียบร้อยแล้วพารามิเตอร์จากคลาส torch.nn.Parameter จะถูกนำไปลงทะเบียน (Register) ลงในตัวแปร self. parameters ของคลาส torch.nn.Module ให้อัตโนมัติ
- **forward()** ใช้สำหรับในการกำหนดลำดับการทำงานของโมเดลจากบนลงล่าง

การสร้างโมเดลแบบกำหนดเอง

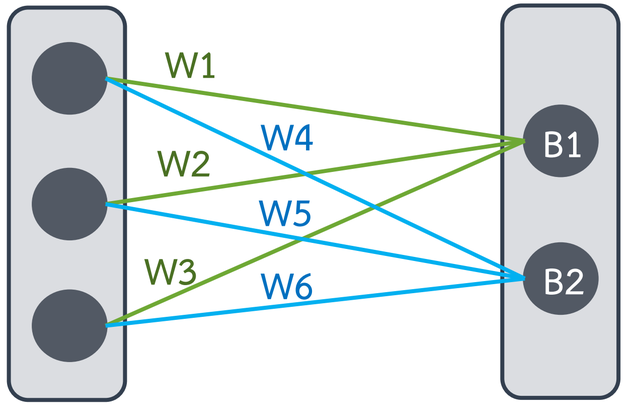
```
[Code]
linear = torch.nn.Linear(3, 2)
x = torch.rand(1, 3)
print('Input:')
print(x)

print('\n\nWeight and Bias parameters:')
for param in linear.parameters():
    print(param)

-------------------------------------------------
[Results]
Input:
tensor([[0.8790, 0.9774, 0.2547]])


Weight and Bias parameters:
Parameter containing:
tensor([[ 0.1656,  0.4969, -0.4972],
        [-0.2035, -0.2579, -0.3780]],
requires_grad=True)

Parameter containing:
tensor([0.3768, 0.3781], requires_grad=True)
```



จากตัวอย่างโค้ดทางด้านซ้ายมือเป็นการแสดงให้เห็น Weight และ Bias ขององค์ประกอบในชุดคำสั่ง Pytorch เริ่มจากสร้าง Linear Layer ที่รับอินพุธขนาด 1×3 และให้ผลลัพธ์ขนาด 1×2 โดยใช้คำสั่ง torch.nn.Linear(3, 2) จากนั้น เมื่อนำไปผ่านคำสั่ง print() เพื่อดูรายละเอียดพารามิเตอร์จะพบว่ามีการแสดงข้อมูลออกมา 2 ชุดด้วยกัน (Parameter containing: ....) ซึ่งมาจากค่า Weight ที่มีขนาด 2×3 และ Bias ขนาด 1×2



การสร้างโมเดลแบบกำหนดเอง

ตัวอย่างขององค์ประกอบของโมเดลในงานประเภทต่าง ๆ

Linear Layer

- **torch.nn.Linear**(in_features*, out_features*, bias=True, device=None, dtype=None)

Convolutional Layers

- **torch.nn.Conv1d**(in_channels*, out_channels*, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True, padding_mode='zeros', device=None, dtype=None)
- **torch.nn.Conv2d**(in_channels*, out_channels*, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True, padding_mode='zeros', device=None, dtype=None)
- **torch.nn.ConvTranspose2d**(in_channels*, out_channels*, kernel_size, stride=1, padding=0, output_padding=0, groups=1, bias=True, dilation=1, padding_mode='zeros', device=None, dtype=None)

ข้อมูลเพิ่มเติม torch.nn : https://pytorch.org/docs/stable/nn.html



การสร้างโมเดลแบบกำหนดเอง

ตัวอย่างขององค์ประกอบของโมเดลในงานประเภทต่าง ๆ

Recurrent Layers

- **torch.nn.RNN**(input_size*, hidden_size*, num_layers=1, nonlinearity='tanh', bias=True, batch_first=False, dropout=0.0, bidirectional=False, device=None, dtype=None)
- **torch.nn.GRU**(input_size*, hidden_size*, num_layers=1, bias=True, batch_first=False, dropout=0.0, bidirectional=False, device=None, dtype=None)
- **torch.nn.LSTM**(input_size*, hidden_size*, num_layers=1, bias=True, batch_first=False, dropout=0.0, bidirectional=False, proj_size=0, device=None, dtype=None)
- **torch.nn.RNNCell**(input_size*, hidden_size*, bias=True, nonlinearity='tanh', device=None, dtype=None)

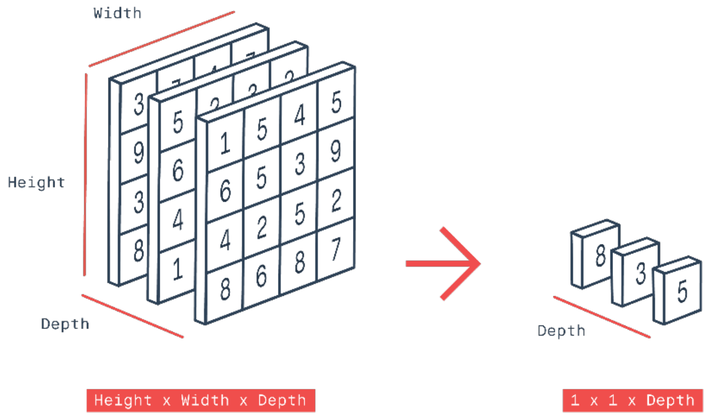
ข้อมูลเพิ่มเติม torch.nn : https://pytorch.org/docs/stable/nn.html

การสร้างโมเดลแบบกำหนดเอง

ตัวอย่างของ**องค์ประกอบ**ของโมเดลในงานประเภทต่าง ๆ

องค์ประกอบอื่นๆ

- `torch.nn.MaxPool2d(kernel_size*, stride=None, padding=0, dilation=1, return_indices=False, ceil_mode=False)`
- `torch.nn.AvgPool2d(kernel_size*, stride=None, padding=0, ceil_mode=False, count_include_pad=True, divisor_override=None)`
- `torch.nn.AdaptiveAvgPool2d(output_size*)` `# AdaptiveAvgPool2d like GlobalAveragePooling2D`
- `torch.nn.ZeroPad2d(padding*)`
- `torch.nn.ReLU(inplace=False)`
- `torch.nn.Sigmoid(*args, **kwargs)`
- `torch.nn.Tanh(*args, **kwargs)`



อ้างอิง : https://pytorch.org/docs/stable/nn.htm และ https://underflow101.tistory.com/41



การสร้างโมเดลแบบกำหนดเอง

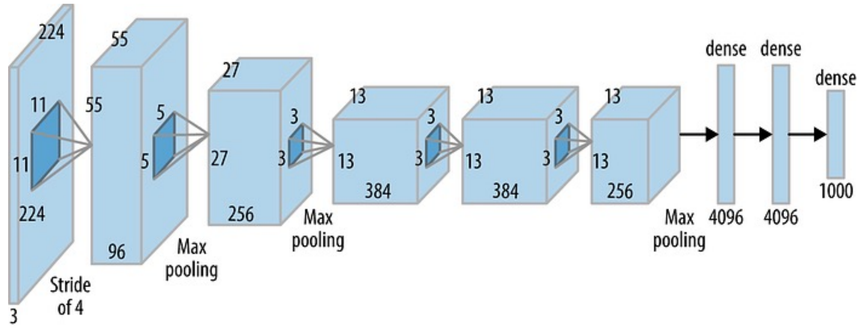
ตัวอย่างขององค์ประกอบของโมเดลในงานประเภทต่าง ๆ

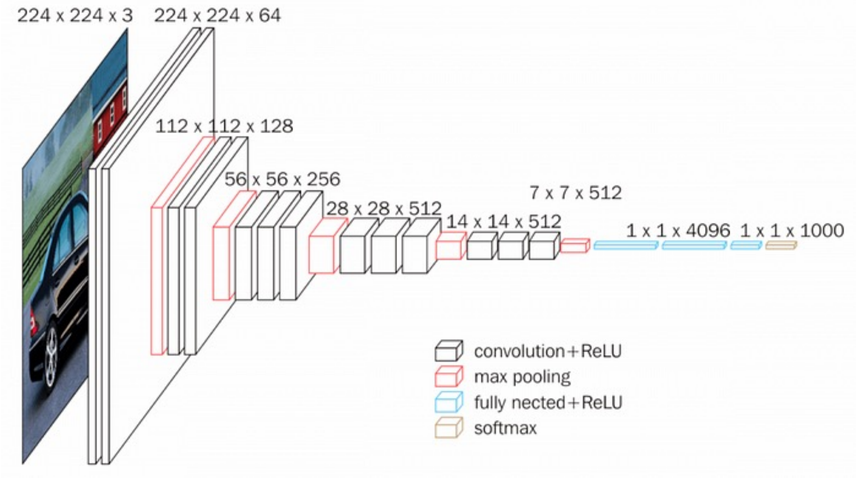
องค์ประกอบอื่นๆ

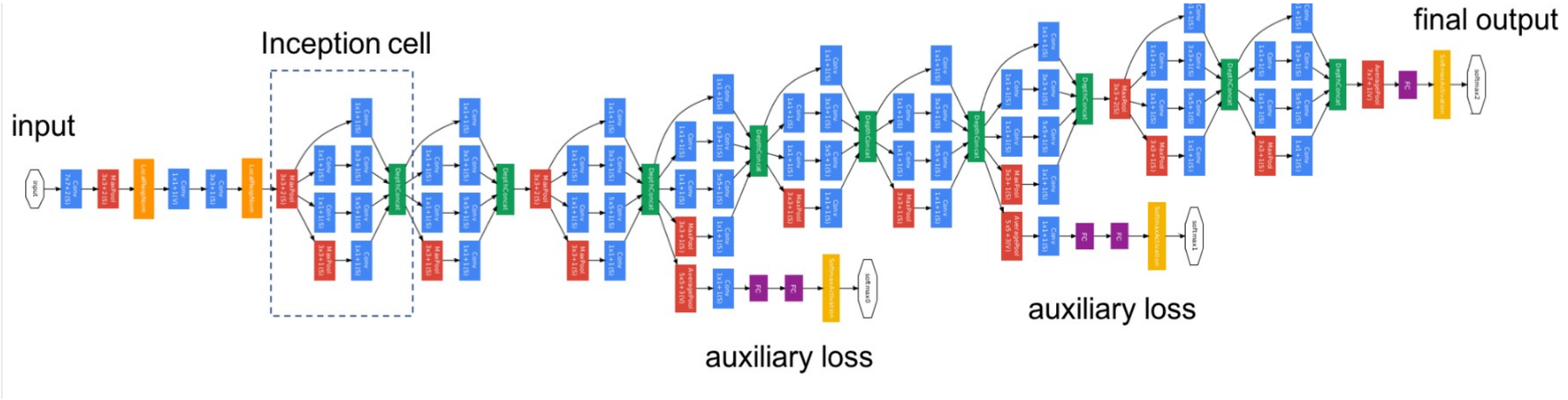
- **torch.nn.Softmax**(dim=None)
- **torch.nn.BatchNorm1d**(num_features*, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True, device=None, dtype=None)
- **torch.nn.BatchNorm2d**(num_features*, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True, device=None, dtype=None)
- **torch.nn.Dropout**(p=0.5, inplace=False)
- **torch.nn.Flatten**(start_dim=1, end_dim=-1)

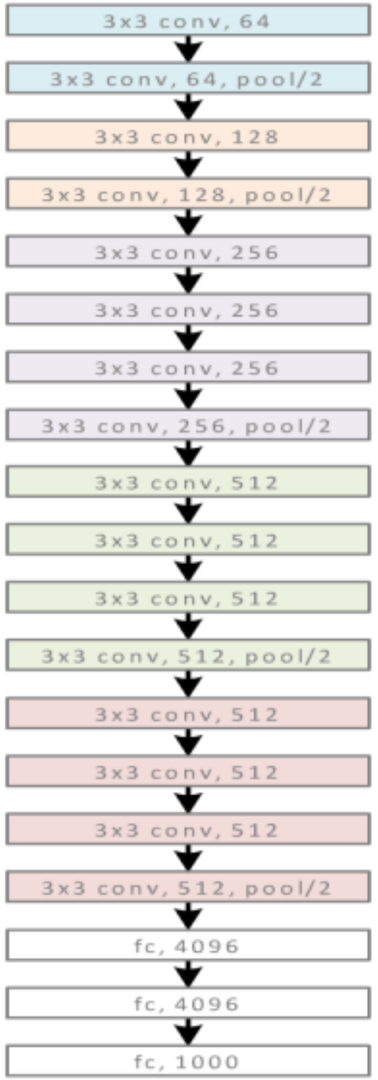
ข้อมูลเพิ่มเติม torch.nn : https://pytorch.org/docs/stable/nn.html

นอกจากนี้หากต้องการเลือกใช้โมเดล (Backbone) สำเร็จรูป ที่ถูกฝึกฝนเรียบร้อย (Pretrain) ด้วยชุดข้อมูล IMAGENET ไม่ว่าจะเป็น VGG, ResNet, SqueezeNet หรือ EfficientNet ก็สามารถเลือกใช้ได้จาก torchvision.models โดยตรง









นอกจากนี้ หากต้องการเลือกใช้โมเดล (Backbone) สำเร็จรูป ที่ถูกฝึกฝนเรียบร้อย (Pretrain) ด้วยชุดข้อมูล IMAGENET ไม่ว่าจะเป็น VGG, ResNet, SqueezeNet หรือ EfficientNet ก็สามารถเลือกใช้ได้จาก torchvision.models โดยตรง

Classification

The following classification models are available, with or without pre-trained weights:

- AlexNet
- ConvNeXt
- DenseNet
- EfficientNet
- EfficientNetV2
- GoogLeNet
- Inception V3
- MaxVit
- MNASNet
- MobileNet V2
- MobileNet V3
- RegNet
- ResNet
- ResNeXt
- ShuffleNet V2
- SqueezeNet
- SwinTransformer
- VGG
- VisionTransformer
- Wide ResNet

Weight	Acc@1	Acc@5	Params	GFLOPS	Recipe
AlexNet_Weights.IMAGENET1K_V1	56.522	79.066	61.1M	0.71	link
ConvNeXt_Base_Weights.IMAGENET1K_V1	84.062	96.87	88.6M	15.36	link
ConvNeXt_Large_Weights.IMAGENET1K_V1	84.414	96.976	197.8M	34.36	link
ConvNeXt_Small_Weights.IMAGENET1K_V1	83.616	96.65	50.2M	8.68	link
ConvNeXt_Tiny_Weights.IMAGENET1K_V1	82.52	96.146	28.6M	4.46	link
DenseNet121_Weights.IMAGENET1K_V1	74.434	91.972	8.0M	2.83	link
DenseNet161_Weights.IMAGENET1K_V1	77.138	93.56	28.7M	7.73	link
DenseNet169_Weights.IMAGENET1K_V1	75.6	92.806	14.1M	3.36	link
DenseNet201_Weights.IMAGENET1K_V1	76.896	93.37	20.0M	4.29	link
EfficientNet_B0_Weights.IMAGENET1K_V1	77.692	93.532	5.3M	0.39	link
EfficientNet_B1_Weights.IMAGENET1K_V1	78.642	94.186	7.8M	0.69	link
EfficientNet_B1_Weights.IMAGENET1K_V2	79.838	94.934	7.8M	0.69	link
EfficientNet_B2_Weights.IMAGENET1K_V1	80.608	95.31	9.1M	1.09	link
EfficientNet_B3_Weights.IMAGENET1K_V1	82.008	96.054	12.2M	1.83	link
EfficientNet_B4_Weights.IMAGENET1K_V1	83.384	96.594	19.3M	4.39	link
EfficientNet_B5_Weights.IMAGENET1K_V1	83.444	96.628	30.4M	10.27	link
EfficientNet_B6_Weights.IMAGENET1K_V1	84.008	96.916	43.0M	19.07	link
EfficientNet_B7_Weights.IMAGENET1K_V1	84.122	96.908	66.3M	37.75	link
EfficientNet_V2_L_Weights.IMAGENET1K_V1	85.808	97.788	118.5M	56.08	link
EfficientNet_V2_M_Weights.IMAGENET1K_V1	85.112	97.156	54.1M	24.58	link
EfficientNet_V2_S_Weights.IMAGENET1K_V1	84.228	96.878	21.5M	8.37	link
GoogLeNet_Weights.IMAGENET1K_V1	69.778	89.53	6.6M	1.5	link

https://pytorch.org/vision/stable/models.html



การสร้างโมเดลแบบกำหนดเอง

ตัวอย่างการใช้งาน

```
from torchvision.models import resnet50, googlenet, squeezenet1_0, densenet121, ResNet50_Weights,
GoogLeNet_Weights, SqueezeNet1_0_Weights, DenseNet121_Weights

# Using pretrained weights:

Model1 = resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)

Model2 = googlenet(weights=GoogLeNet_Weights.IMAGENET1K_V2)

Model3 = squeezenet1_0(weights=SqueezeNet1_0_Weights.IMAGENET1K_V2)

Model4 = densenet121 (weights=DenseNet121_Weights.IMAGENET1K_V2)

# Using no weights:

Model = resnet50()
```

สำหรับโมเดลสำเร็จรูปที่ชุดคำสั่ง torchvision.models พร้อมให้ใช้งานสามารถดูได้ที่ลิ้ง

- https://pytorch.org/vision/stable/models.html



หัวข้อ

- การสร้างโมเดล Image Classification แบบพื้นฐาน
- การสร้างโมเดลแบบกำหนดเอง (Custom CNN Model)
- **การสร้าง Data Loader สำหรับชุดข้อมูลแบบกำหนดเอง**
- การทำ Data Transformation และ Augmentation

การสร้าง Data Loader สำหรับชุดข้อมูลแบบกำหนดเอง



การประมวลผล (Process) ข้อมูลสามารถก่อให้เกิดความสับสนและดูแลรักษา (Maintain) ค่อนข้างลำบาก ดังนั้น ในชุดคำสั่ง PyTorch จึงทำการแยกการจัดการข้อมูลออกมาเป็นโมดูล เพื่อให้เกิดความสะดวกต่อการอ่านง่ายและทำให้โค้ดดูเรียบร้อยมากยิ่งขึ้น โดยชุดคำสั่ง PyTorch ได้ทำการแยกรูปแบบชุดข้อมูลพื้นฐาน (Primitive Data) ออกมาเป็น 2 ประเภท

- **torch.utils.data.Dataset** ทำหน้าที่ในการจัดการกับชุดข้อมูล รวมถึงจัดเก็บตัวอย่าง (Sample) และป้ายเฉลย (Label) ที่ต้องการนำมาฝึกฝนกับโมเดล นอกจากนี้ ยังสามารถเรียกใช้ชุดข้อมูล (Dataset) ต่าง ๆ ที่มีอยู่แล้วมาใช้ได้อีกด้วย
- **torch.utils.data.DataLoader** ทำหน้าที่ในการลำเลียงข้อมูลในขั้นตอนของการฝึกฝนและการทดสอบ โดยจะนำเอาข้อมูลจาก torch.utils.data.Dataset มาจัดให้เหมาะสม ตามขนาด Batch Size และลำเลียงไปฝึกฝนหรือทดสอบ



การสร้าง Data Loader สำหรับชุดข้อมูลแบบกำหนดเอง

```
import torch
from torch.utils.data import Dataset
from torchvision import datasets
from torchvision.transforms import ToTensor
import matplotlib.pyplot as plt

training_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor()
)

test_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=ToTensor()
)
```

กรณีที่ 1 สำหรับการเข้าถึงชุดข้อมูลสำเร็จรูป สามารถเข้าไปดูรายชื่อชุดข้อมูลที่ Pytorch มีให้ได้จากลิ้งในข้อมูลเพิ่มเติม ในการใช้งานเพียงนำเข้าชุดคำสั่ง **torch.utils.data** ตามด้วยชื่อชุดข้อมูลสำเร็จรูปที่ต้องการใช้งาน โดยมีพารามิเตอร์ที่ต้องกำหนดดังนี้

พารามิเตอร์	คำอธิบาย
root	ตำแหน่งในการดาวน์โหลดเมื่อ download=true
train	ชุดในการนำข้อมูลไปใช้
download	ดาวน์โหลดชุดข้อมูลไปเก็บไว้ที่ root
transform	นำชุดข้อมูลไปผ่านกระบวนการบางอย่างก่อนไปทดสอบหรือฝึกโมเดล

ข้อมูลเพิ่มเติม Dataset : https://pytorch.org/vision/stable/datasets.html

การสร้าง Data Loader สำหรับชุดข้อมูลแบบกำหนดเอง

```
import os
import pandas as pd
from torch.utils.data import Dataset
from torchvision.io import read_image

class CustomImageDataset(Dataset):
    def __init__(self, annotations_file, img_dir, transform=None,
                 target_transform=None):
        self.img_labels = pd.read_csv(annotations_file)
        self.img_dir = img_dir
        self.transform = transform
        self.target_transform = target_transform

    def __len__(self):
        return len(self.img_labels)

    def __getitem__(self, idx):
        img_path = os.path.join(self.img_dir,
                                self.img_labels.iloc[idx, 0])
        image = read_image(img_path)
        label = self.img_labels.iloc[idx, 1]
        if self.transform:
            image = self.transform(image)
        if self.target_transform:
            label = self.target_transform(label)
        return image, label
```

กรณีที่ 2 สำหรับข้อมูลแบบกำหนดเอง

ขั้นตอนที่ 1 การสร้างคลาสใหม่ขึ้นมา จากนั้น สืบทอดคลาส torch.utils. data.Dataset จากชุดคำสั่งของ Pytorch โดยจำเป็นต้องสร้าง (Implement) 3 ฟังก์ชันเพิ่มเติม ดังนี้

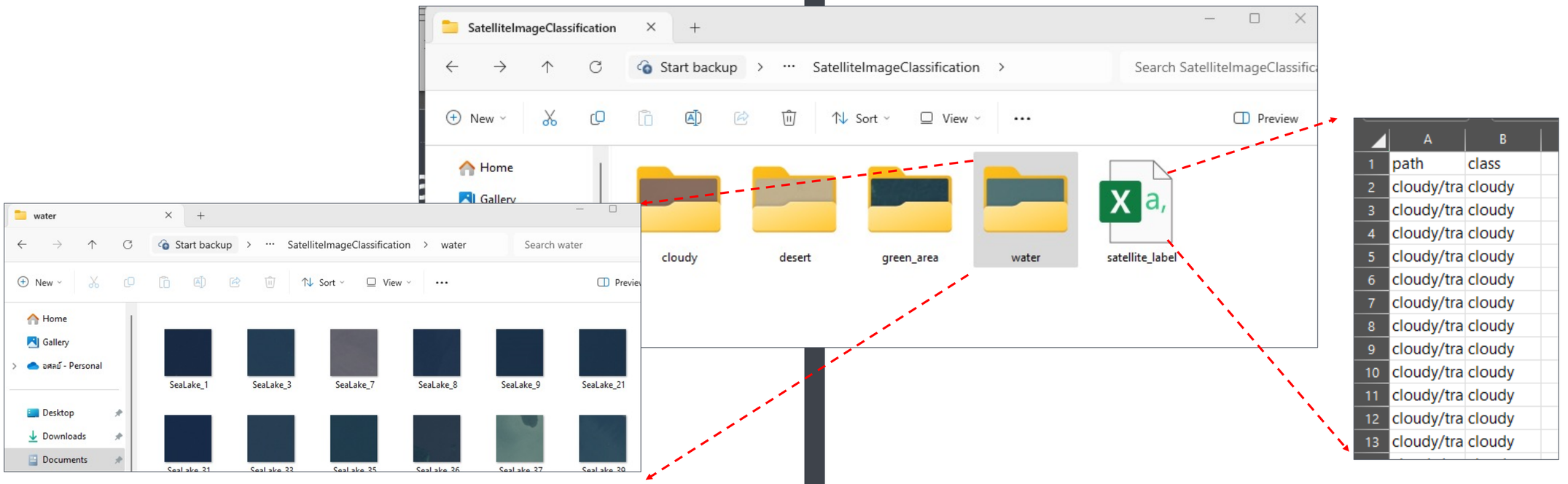
ฟังก์ชัน	คำอธิบาย
__init__	เป็นฟังก์ชันสำหรับ (1) กำหนดที่อยู่ หรือ ตำแหน่งเริ่มต้น (Root) ของชุดข้อมูล (2) กำหนดวิธีการแปลงข้อมูล (2) กำหนดตำแหน่งของเฉลย ซึ่งจะถูกเรียกใช้งานหนึ่งครั้งเมื่อ Object ของ Dataset ถูกสร้าง
__len__	เป็นฟังก์ชันสำหรับคืนค่า (Return) จำนวนข้อมูลในชุดข้อมูล
__getitem__	เป็นฟังก์ชันสำหรับจัดการและคืนค่าชุดข้อมูล และเฉลย โดยจะเรียกใช้ในขั้นตอนฝึกฝนและทดสอบ

การสร้าง Data Loader สำหรับชุดข้อมูลแบบกำหนดเอง

```python
from torch.utils.data import DataLoader


# Initialize Dataset
training_data = CustomImageDataset(annotations_file, img_dir)
test_data = CustomImageDataset(annotations_file, img_dir)
```

ขั้นตอนที่ 2 ให้สร้าง Object ของ Dataset ของชุดข้อมูลที่ต้องการ โดยการกำหนดพารามิเตอร์ต่อไปนี้



ข้อมูลเพิ่มเติม DataLoader : https://pytorch.org/docs/stable/data.html



การสร้าง Data Loader สำหรับชุดข้อมูลแบบกำหนดเอง

```python
from torch.utils.data import DataLoader


# Initialize Dataset
training_data = CustomImageDataset(annotations_file, img_dir)
test_data = CustomImageDataset(annotations_file, img_dir)


# Converts to Data Loader
train_dataloader = DataLoader (training_data, batch_size=64,
shuffle=True)
test_dataloader = DataLoader (test_data, batch_size=64,
shuffle=True)
```

ขั้นตอนที่ 3 ให้แปลง Object ของ Dataset ของชุดข้อมูลไปเป็น Object ของ DataLoader โดยใช้ชุดคำสั่ง torch.utils.data.DataLoader ซึ่งจะเป็นคำสั่งที่ใช้สำหรับลำเลียงข้อมูลไปฝึกฝนหรือทดสอบ ตามพารามิเตอร์ที่เรากำหนดไว้ พร้อมกับแปลงไปเป็นข้อมูล iterator ให้อัตโนมัติ ตัวอย่างของพารามิเตอร์ที่สามารถกำหนดได้

พารามิเตอร์	คำอธิบาย
batch_size	เป็นการกำหนดขนาดข้อมูลที่จะใช้ฝึกฝนหรือทดสอบ ในหนึ่งครั้ง
shuffle	เป็นการกำหนดว่าจะสุ่มข้อมูลหรือไม่ ซึ่งจะสุ่มใหม่ทุก ๆ eporch
num_workers	หากกำหนดเป็นจำนวนเต็มบวก Pytorch จะเปิดการโหลดข้อมูลแบบ multi-process ในการโหลดข้อมูลเข้ามายัง Ram ของระบบคอมพิวเตอร์ โดยที่ ค่าเริ่มต้นเป็น 0

ข้อมูลเพิ่มเติม DataLoader : https://pytorch.org/docs/stable/data.html



หัวข้อ

- การสร้างโมเดล Image Classification แบบพื้นฐาน
- การสร้างโมเดลแบบกำหนดเอง (Custom CNN Model)
- การสร้าง Data Loader สำหรับชุดข้อมูลแบบกำหนดเอง
- การทำ Data Transformation และ Augmentation

การทำ Data Transformation และ Augmentation



RandomPerspective



ElasticTransform



CenterCrop

ชุดคำสั่ง Pytorch มีฟังก์ชันสำเร็จรูปสำหรับการ Data Transformation และ Augmentation ได้โดยตรงเพียงนำเข้าชุดคำสั่ง torchvision.transforms.v2 ซึ่งภายในชุดคำสั่งประกอบไปด้วยฟังก์ชันมากมาย ไม่ว่าจะเป็นฟังก์ชันสำหรับแปลงรูปแบบข้อมูล ฟังก์ชันสำหรับการทำ Pre-process ข้อมูล รวมถึงเครื่องมือสำหรับการทำ Data Augmentation ด้วย

ภาพประกอบ : https://pytorch.org/vision/stable/auto_examples/transforms/plot_transforms_illustrations.html#sphx-glr-auto-examples-transforms-plot-transforms-illustrations-py



การทำ Data Transformation และ Augmentation

```
import torch
from torchvision.transforms import v2

preproc = v2.Compose([
    v2.PILToTensor(),
    v2.RandomResizedCrop(size=(224, 224), antialias=True),
    v2.RandomHorizontalFlip(p=0.5),

    # to float32 in [0, 1]
    v2.ToDtype(torch.float32, scale=True),

    # typically, from ImageNet
    v2.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224,
0.225)),])

trainset = torchvision.datasets.ImageNet(root='./data',
train=True, download=True, transform=preproc)
```

สำหรับการทำ Data Transformation และ Augmentation จะกำหนดลำดับการทำงานเป็นขั้นตอน โดยอาศัยใช้ฟังก์ชัน torchvision. transforms.v2.Compose()

จากโค้ดตัวอย่าง จะพบว่าในฟังก์ชัน Compose() จะเริ่มจากแปลงภาพจาก Object ของ Image มาอยู่ในรูปแบบ Object ของ Tensor ใน Pytorch จนจบฟังก์ชัน Compose() จากนั้น ให้นำผลลัพธ์ที่ได้ไปผ่านกระกวนการ Data Augmentation จากฟังก์ชันที่เรากำหนด ได้แก่ RandomResizedCrop, RandomResizedCrop และ Random HorizontalFlip ก่อนนำผลลัพธ์ที่ได้ไป ดำเนินการแปลงชนิดข้อมูลให้อยู่ในรูปแบบเลขทศนิยมในช่วง 0 ถึง 1 และผ่านกระบวนการ Normalize ข้อมูลเพื่อให้แบบจำลองมีประสิทธิภาพในการฝึกสอนแบบจำลองมากยิ่งขึ้น

ข้อมูลเพิ่มเติม DataLoader : https://pytorch.org/docs/stable/data.html



การทำ Data Transformation และ Augmentation

ตัวอย่างของฟังก์ชันในชุดคำสั่ง torchvision.transforms.v2

Geometry

- **torchvision.transforms.v2.Resize**(size*: Union[int, Sequence[int]], interpolation: Union[InterpolationMode, int] = InterpolationMode.BILINEAR, max_size: Optional[int] = None, antialias: Optional[bool] = True)
- **torchvision.transforms.v2.RandomResize**(min_size*: int, max_size*: int, interpolation: Union[InterpolationMode, int] = InterpolationMode.BILINEAR, antialias: Optional[bool] = True)
- **torchvision.transforms.v2.RandomCrop**(size*: Union[int, Sequence[int]], padding: Optional[Union[int, Sequence[int]]] = None, pad_if_needed: bool = False, fill: Union[int, float, Sequence[int], Sequence[float], None, Dict[Union[Type, str], Optional[Union[int, float, Sequence[int], Sequence[float]]]]] = 0, padding_mode: Literal['constant', 'edge', 'reflect', 'symmetric'] = 'constant’)
- **torchvision.transforms.v2.RandomHorizontalFlip**(p: float = 0.5)

ข้อมูลเพิ่มเติม torchvision.transforms.v2: https://pytorch.org/vision/stable/transforms.html



การทำ Data Transformation และ Augmentation

ตัวอย่างของฟังก์ชันในชุดคำสั่ง torchvision.transforms.v2

Geometry

- **torchvision.transforms.v2.RandomZoomOut**(fill: Union[int, float, Sequence[int], Sequence[float], None, Dict[Union[Type, str], Optional[Union[int, float, Sequence[int], Sequence[float]]]]] = 0, side_range: Sequence[float] = (1.0, 4.0), p: float = 0.5)
- **torchvision.transforms.v2.RandomRotation**(degrees*: Union[Number, Sequence], interpolation: Union[InterpolationMode, int] = InterpolationMode.NEAREST, expand: bool = False, center: Optional[List[float]] = None, fill: Union[int, float, Sequence[int], Sequence[float], None, Dict[Union[Type, str], Optional[Union[int, float, Sequence[int], Sequence[float]]]]] = 0)
- **torchvision.transforms.v2.Pad**(padding*: Union[int, Sequence[int]], fill: Union[int, float, Sequence[int], Sequence[float], None, Dict[Union[Type, str], Optional[Union[int, float, Sequence[int], Sequence[float]]]]] = 0, padding_mode: Literal['constant', 'edge', 'reflect', 'symmetric'] = 'constant')

ข้อมูลเพิ่มเติม torchvision.transforms.v2: https://pytorch.org/vision/stable/transforms.html



การทำ Data Transformation และ Augmentation

ตัวอย่างของฟังก์ชันในชุดคำสั่ง torchvision.transforms.v2

Color

- **torchvision.transforms.v2.Grayscale**(num_output_channels: int = 1)
- **torchvision.transforms.v2.RGB**()
- **torchvision.transforms.v2.GaussianBlur**(kernel_size*: Union[int, Sequence[int]], sigma: Union[int, float, Sequence[float]] = (0.1, 2.0))
- **torchvision.transforms.v2.RandomAdjustSharpness**(sharpness_factor*: float, p: float = 0.5)

Composition

- **torchvision.transforms.v2.RandomApply**(transforms*: Union[Sequence[Callable], ModuleList], p: float = 0.5)

ข้อมูลเพิ่มเติม torchvision.transforms.v2: https://pytorch.org/vision/stable/transforms.html



การทำ Data Transformation และ Augmentation

ตัวอย่างของฟังก์ชันในชุดคำสั่ง torchvision.transforms.v2

Miscellaneous

- ***torchvision.transforms.v2.Normalize****(mean*: Sequence[float], std*: Sequence[float], inplace: bool = False)*

Conversion

- ***torchvision.transforms.v2.PILToTensor****()*
- ***torchvision.transforms.v2.ToDtype****(dtype*: Union[dtype, Dict[Union[Type, str], Optional[dtype]]], scale: bool = False)*
- ***torchvision.transforms.v2.ToPILImage****(mode: Optional[str] = None)*
- *(**Deprecated**) torchvision.transforms.v2.ToTensor() -> v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True)])*

ข้อมูลเพิ่มเติม torchvision.transforms.v2: https://pytorch.org/vision/stable/transforms.html



การทำ Data Transformation และ Augmentation

ตัวอย่างของฟังก์ชันในชุดคำสั่ง torchvision.transforms.v2

Auto-Augmentation

- ***torchvision.transforms.v2.AugMix****(severity: int = 3, mixture_width: int = 3, chain_depth: int = - 1, alpha: float = 1.0, all_ops: bool = True, interpolation: Union[InterpolationMode, int] = InterpolationMode.BILINEAR, fill: Union[int, float, Sequence[int], Sequence[float], None, Dict[Union[Type, str], Optional[Union[int, float, Sequence[int], Sequence[float]]]]] = None)*

CutMix

- ***torchvision.transforms.v2.CutMix****(*, alpha: float = 1.0, num_classes: int, labels_getter='default’)*

ข้อมูลเพิ่มเติม https://pytorch.org/vision/stable/auto_examples/transforms/plot_cutmix_mixup.html#sphx-glr-auto-examples-transforms-plot-cutmix-mixup-py และ https://pytorch.org/vision/stable/transforms.html



หัวข้อ

- หลักการและการสร้างโมเดลสำหรับ Multiclass Image Classification
- หลักการและการสร้างโมเดลสำหรับ Multilabel Image Classification
- หลักการและการสร้างโมเดลสำหรับ Image Regression
- หลักการ Multi-task Learning

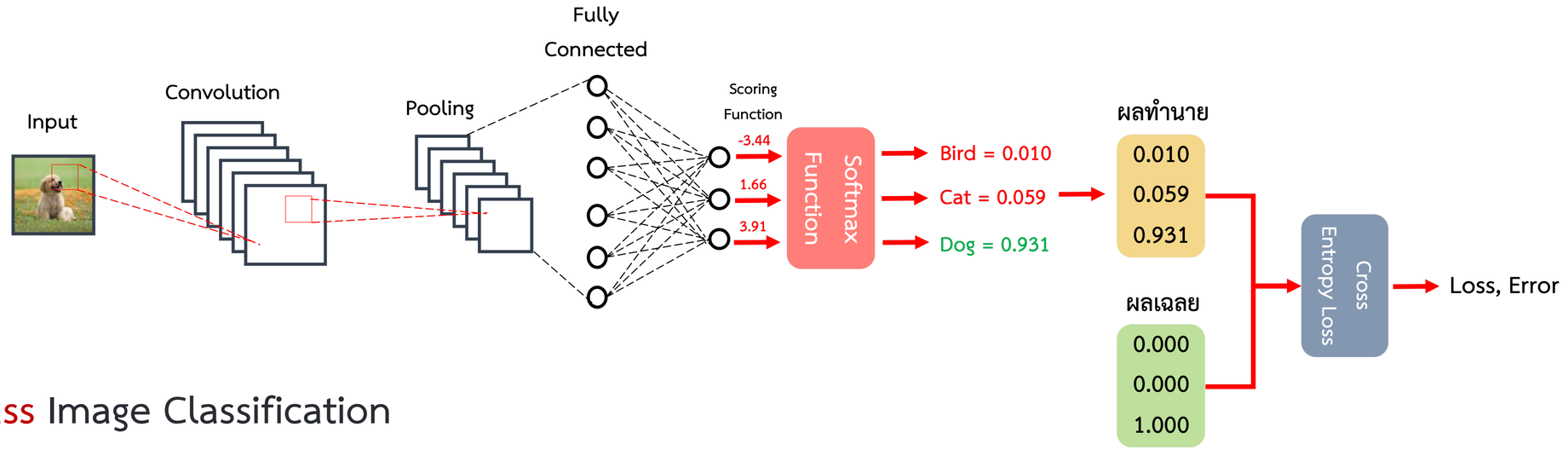


หัวข้อ

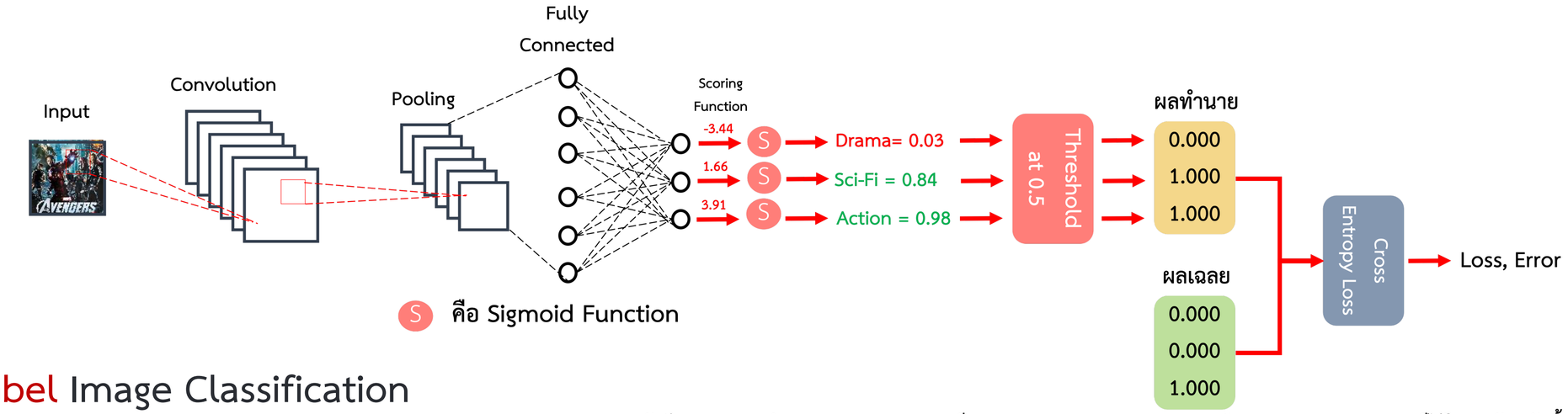
- หลักการและการสร้างโมเดลสำหรับ Multiclass Image Classification
- **หลักการและการสร้างโมเดลสำหรับ Multilabel Image Classification**
- หลักการและการสร้างโมเดลสำหรับ Image Regression
- หลักการ Multi-task Learning

การสร้างโมเดลสำหรับ Multilabel Image Classification

การพิจารณาผลลัพธ์ของแบบจำลองและการคำนวณข้อผิดพลาด



Multiclass Image Classification



Multilabel Image Classification

นักศึกษาสามารถกำหนด Loss Function เป็น Binary Cross Entropy Loss หรือ Cross Entropy Loss ก็ได้ใน ณ ปัจจุบันนี้



การสร้างโมเดล Multilabel Image Classification

กำหนดโครงสร้าง Convolutional Neural Network สำหรับ Multilabel Image Classification

```
from torchvision.models import resnet50, ResNet50_Weights
import torch.nn as nn

# Load Backborn
model = resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)
# Edit Model
model.fc = nn.Linear(2048, 24)
print(model)
```

กำหนด Loss function และ Optimizer

```
import torch.optim as optim

# This loss combines a Sigmoid layer and the Binary Cross Entropy (BCELoss) in one single class.
criterion = nn.BCEWithLogitsLoss()
# optimizer function
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```



การสร้างโมเดล Multilabel Image Classification

ตัวอย่างโปรแกรมเรียกใช้งานแบบจำลอง

```
from PIL import Image
from torch.nn.functional import sigmoid

img_path = 'Multi_Label_dataset/tt0084867.jpg'
image = Image.open(img_path).convert('RGB')
image = transform(image)

# Check if GPU (cuda) is available
device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

# Move our model to device (GPU or CPU)
model.to(device)
model.eval()

image = image.to(device)
image = image.unsqueeze(0)

output = model(image)

# Normalize range
output = sigmoid(output)

# get predict class based on threshold
output = output.squeeze(0)
output = torch.nonzero(output > 0.3, as_tuple=True)[0].cpu()
print("The genre of this oster is ", classes[output].to_list())
```

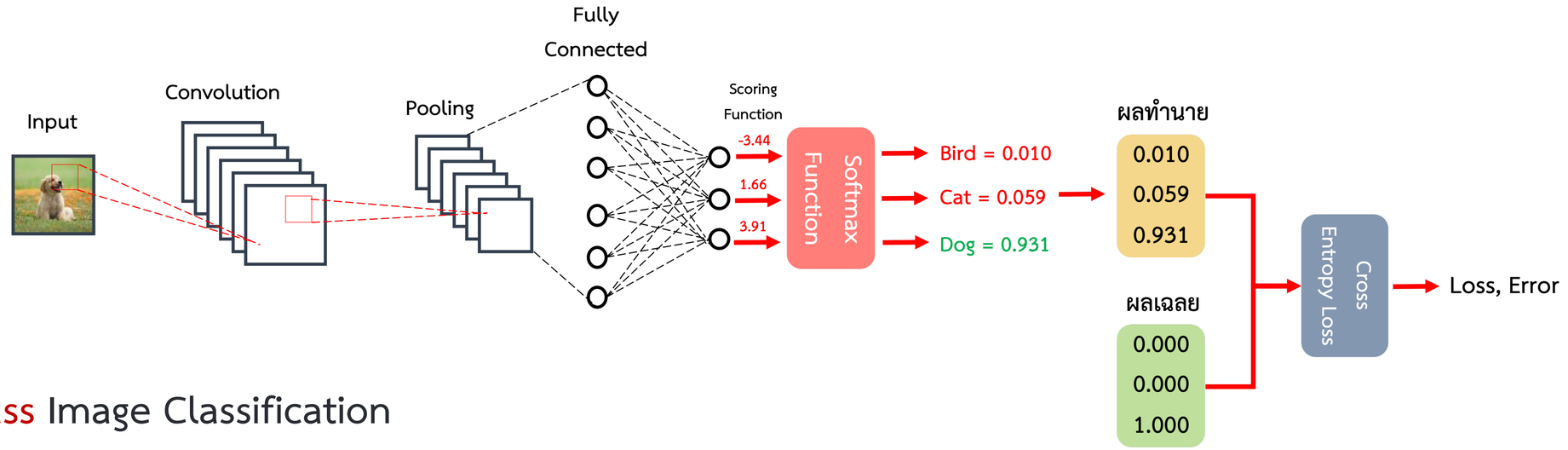


หัวข้อ

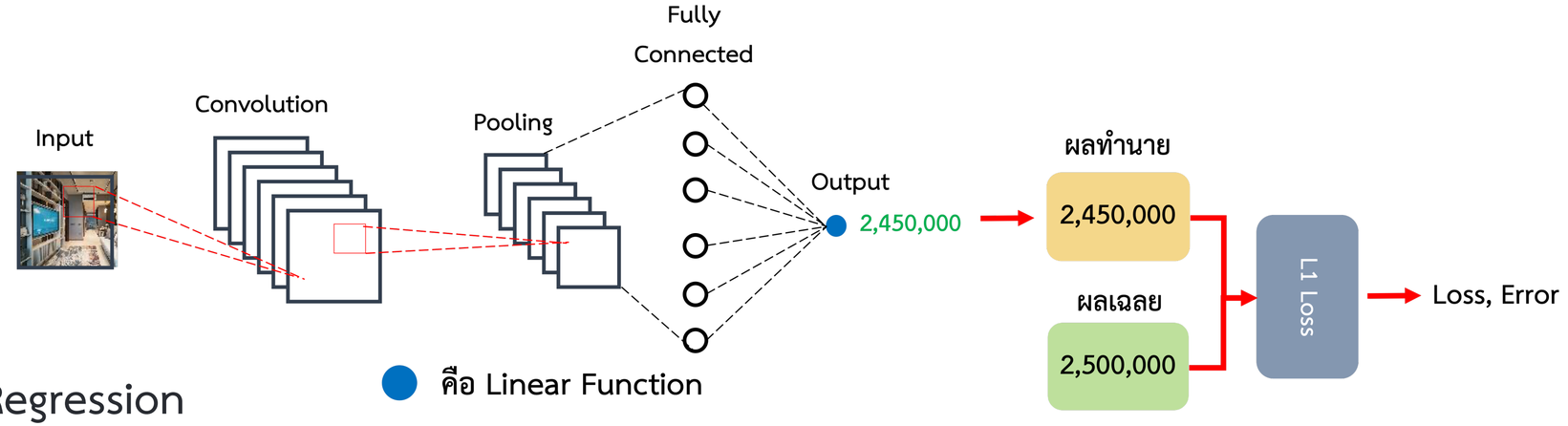
- หลักการและการสร้างโมเดลสำหรับ Multiclass Image Classification
- หลักการและการสร้างโมเดลสำหรับ Multilabel Image Classification
- **หลักการและการสร้างโมเดลสำหรับ Image Regression**
- หลักการ Multi-task Learning

การสร้างโมเดลสำหรับ Image Regression

การพิจารณาผลลัพธ์ของแบบจำลองและการคำนวณข้อผิดพลาด



Multiclass Image Classification



คือ Linear Function

Image Regression

นักศึกษาสามารถกำหนด Loss Function เป็น Mean Absolute Error (MAE) หรือ L1 loss, Mean Squared Error (MSE) หรือ L2 loss และอื่น ๆ ก็ได้ใน



การสร้างโมเดล Image Regression

กำหนดโครงสร้าง Convolutional Neural Network สำหรับ Image Regression

```
from torchvision.models import resnet50, ResNet50_Weights
import torch.nn as nn

# Load Backborn
model = resnet50()
# Edit Model
model.fc = nn.Linear(2048, 1)
```

กำหนด Loss function และ optimizer

```
import torch.optim as optim

# loss function
criterion = nn.L1Loss()
# optimizer function
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```



การสร้างโมเดล Image Regression

ตัวอย่างโปรแกรมเรียกใช้งานแบบจำลอง

```
from PIL import Image
from torch.nn.functional import sigmoid

img_path = 'Multi_Label_dataset/tt0084867.jpg'
image = Image.open(img_path).convert('RGB')
image = transform(image)

# Check if GPU (cuda) is available
device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

# Move our model to device (GPU or CPU)
model.to(device)
model.eval()

image = image.to(device)
image = image.unsqueeze(0)

output = model(image)

# Normalize range
output = sigmoid(output)

# get predict class based on threshold
output = output.squeeze(0)
output = torch.nonzero(output > 0.3, as_tuple=True)[0].cpu()
print("The genre of this oster is ", classes[output].to_list())
```



การสร้างโมเดล Image Regression

ตัวอย่างโปรแกรมเรียกใช้งานแบบจำลอง

```
from PIL import Image
img_path = 'AgeDetection/30_1_2_20170116192249012.jpg'
image = Image.open(img_path).convert('RGB')
image = transform(image)

# Check if GPU (cuda) is available
device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

# Move our model to device (GPU or CPU)
model.to(device)
model.eval()

image = image.to(device)
image = image.unsqueeze(dim=0)

outputs = model(image)
predicted = outputs.squeeze(dim=0)
print(f'Predicted: {predicted.item():,.2f} years old', )
```



การสร้างโมเดล Image Regression

ตัวอย่างโปรแกรมการสร้างแบบจำลองสำหรับ Image Regression

```
import torch.nn as nn
import torch.nn.functional as F

class RegressionNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        # nn.Conv2d(in_channels, out_channels, kernel_size)
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, 1)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        # flatten all dimensions except batch
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

model = RegressionNetwork()
```